DESARROLLO MÓVIL · INGENIERÍA EN SOFTWARE · GUÍA 2 DE 2: CONSTRUYE ESTA DESPUÉS DE LA API

Construye la app móvil de la Biblioteca

Tutorial paso a paso con React Native, Expo y SQLite. Unidad 03: semanas 9 a 12

Asignatura	Desarrollo Móvil (PREISESP17015)
Unidad	03 · Integración con API, seguridad y servicios del dispositivo
Qué construirás	Una app Expo que consume la API, guarda la sesión de forma segura y funciona sin conexión con SQLite
Requisito	La Guía 1 terminada: la API y Firebase funcionando (`pnpm smoke` en verde)
Tecnologías	Expo SDK 57, React Native 0.86, TypeScript, Expo Router, expo-sqlite, expo-secure-store, NetInfo, expo-image-picker, Jest
Resultado	Proyecto `library_app` completo, con pruebas, listo para el hito 3

Contenido

1. Antes de empezar 4

1.1 Qué vas a construir 4

1.2 Cómo usar esta guía 4

1.3 Requisitos 5

1.4 La URL de la API según dónde corras la app 5

1.5 Ruta de trabajo 6

2. Paso 0 · Preparar tu app 7

2.1 Punto de partida 7

2.2 Retira el acceso directo a Firebase 7

2.3 Instalar los módulos que usaremos 7

2.4 Eliminar los archivos de ejemplo de la plantilla 8

2.5 Configuración del proyecto 8

2.6 Base de interfaz: tema, texto, pantalla y botón 9

3. Semana 9 · Consumo de API REST 14

3.1 Conceptos clave 14

3.2 Paso a paso 14

3.3 Cómo probar en el teléfono 27

3.4 Evidencia y criterios 27

4. Semana 10 · Autenticación, autorización y seguridad 28

4.1 Conceptos clave 28

4.2 Paso a paso 29

4.3 Cómo probar en el teléfono 41

4.4 Evidencia y criterios 41

5. Semana 11 · SQLite, sincronización y trabajo sin conexión 42

5.1 Conceptos clave 42

5.2 Paso a paso 43

5.3 Escenarios de prueba en el teléfono 79

5.4 Evidencia y criterios 79

6. Semana 12 · Servicio del dispositivo integrado con la API 80

6.1 Conceptos clave 80

6.2 Paso a paso 80

6.3 Manejo de fallos: qué ve el usuario . . . 85
6.4 Trazabilidad . . . 85
6.5 Pruebas en dispositivo (lista de verificación) . . . 85
6.6 Evidencia del hito 3 . . . 86

7. Pruebas de integración y calidad . . . 87
7.1 La app real contra la API real . . . 87
7.2 Verificación completa . . . 89
7.3 Qué cubren las pruebas . . . 90

8. Problemas frecuentes . . . 91

9. Bitácora de uso de IA generativa . . . 92

10. Referencias . . . 93

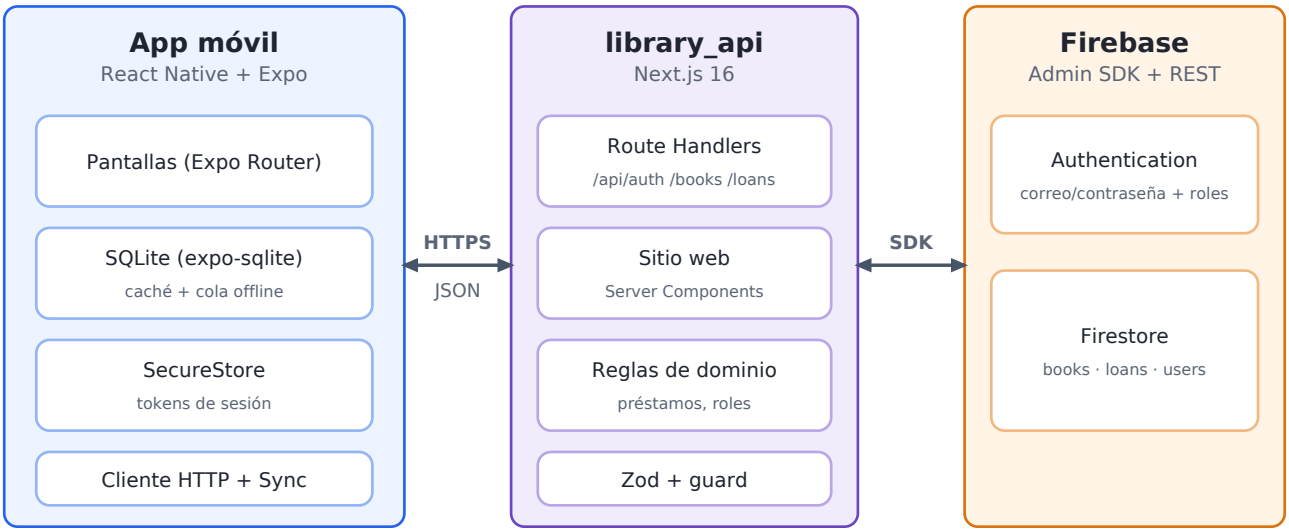
1. Antes de empezar

1.1 Qué vas a construir

Biblioteca es una app móvil para consultar el catálogo de una biblioteca y gestionar préstamos, con dos roles y sin límite de usuarios:

Rol	Qué puede hacer
Estudiante	Crear su cuenta, listar y buscar libros, ver el detalle y **solicitar el préstamo**. Ve el estado de sus solicitudes.
Bibliotecario	Ver todas las solicitudes, **prestar**, rechazar o registrar la devolución, y cambiar la portada de un libro con la cámara.

Hasta ahora tu app hablaba **directamente con Firebase**. En esta guía la conviertes en una app que habla solo con **tu API** (Guía 1) y que **sigue funcionando sin internet**:



La app nunca habla con Firebase: todo pasa por la API.

Tres decisiones explican casi todo el código:

- **La app nunca habla con Firebase.** Solo conoce la API REST; así las credenciales de administrador y las reglas de negocio viven en un único lugar.
- **La interfaz siempre lee de SQLite.** La red solo sirve para sincronizar. Por eso la app es rápida y funciona en el metro o en un ascensor.
- **El servidor es la fuente de verdad.** Si lo que hizo el teléfono sin conexión choca con el servidor, gana el servidor y se le explica al usuario.

1.2 Cómo usar esta guía

Es un **tutorial de construcción**: al terminarlo tienes el proyecto completo y funcionando. Cada paso tiene una etiqueta de color:

Etiqueta	Qué debes hacer
CREAR	Crea el archivo en esa ruta (relativa a la raíz del proyecto) con **exactamente** ese contenido. Si ya existe, reemplázalo completo.

Etiqueta	Qué debes hacer
AGREGAR A	Agrega ese fragmento JSON a un archivo que ya existe (sin borrar lo demás).
ELIMINAR	Borra ese archivo.
TERMINAL	Ejecuta esos comandos dentro de la carpeta del proyecto.

Algunos archivos aparecen **dos veces** (por ejemplo, `src/app/_layout.tsx`): la primera es una versión provisional que sirve para la semana en curso y la segunda la reemplaza por la versión definitiva. Es a propósito: así cada semana termina con una app que compila y se puede probar.

Después de cada semana hay un **punto de control**: ejecuta `npm run typecheck`, `npx expo lint` y `npm test`. Si pasan, sigues; si no, revisa el último archivo que creaste.

Idea. Trabaja con Git y haz un commit al terminar cada semana (`git add -A && git commit -m "semana 9"`). Si algo se rompe, `git diff` muestra qué cambió.

Nota. Los comentarios dentro del código están en español y explican el porqué de cada decisión. Léelos: son parte de la guía.

1.3 Requisitos

- **La Guía 1 terminada**: la API de `library_api` corriendo, con `pnpm seed` ejecutado y **al menos un estudiante y un bibliotecario creados** con `pnpm create-user` (Guía 1, sección 4.4). Compruébalo con `curl http://localhost:3000/api/health`.
- **Node.js 24 LTS** (mínimo 22.5), npm, Git y Visual Studio Code.
- La app **Expo Go** en tu teléfono (Android o iOS), en la misma red Wi-Fi que tu computador.

1.4 La URL de la API según dónde corras la app

Dónde corre la app	Valor de `EXPO_PUBLIC_API_URL`
Teléfono físico con Expo Go	`http://IP-DE-TU-PC:3000` (ej. `http://192.168.1.20:3000`)
Emulador de Android	`http://10.0.2.2:3000`
Simulador de iOS	`http://localhost:3000`

Ojo. En un teléfono físico `localhost` **es el propio teléfono**, no tu computador. Es el error de conexión más común. Además, arranca la API con `pnpm dev:lan` para que acepte conexiones de la red.

Ojo. Si usas **WSL2**, el puerto 3000 de WSL no es visible desde la red del teléfono por defecto. Opciones: activar `networkingMode=mirrored` en `%UserProfile%\.wslconfig`, o crear un reenvío de puertos en PowerShell de administrador: `netsh interface portproxy add v4tov4 listenport=3000 listenaddress=0.0.0.0 connectport=3000 connectaddress=IP-DE-WSL` y permitir el puerto 3000 en el firewall de Windows.

1.5 Ruta de trabajo

Semana	Guía 1 (API y web)	Esta guía (app móvil)	Lo que ya funciona al terminar
9	Capítulo 3: REST, validación, catálogo	Capítulo 3: cliente HTTP y estados de pantalla	La app lista libros desde la API
10	Capítulo 4: Firebase Auth, roles	Capítulo 4: sesión segura y rutas protegidas	Registro, login y roles
11	Capítulo 5: préstamos, idempotencia	Capítulo 5: SQLite offline y sincronización	Solicitudes sin conexión
12	Capítulo 6: portadas	Capítulo 6: cámara y galería	**Hito 3**
—	Capítulo 8: pruebas finales	Capítulo 7: pruebas de integración	Entrega

2. Paso 0 · Preparar tu app

2.1 Punto de partida

La guía parte de un proyecto **Expo con Expo Router y TypeScript** (carpeta `src/app`), como el que crea `npx create-expo-app@latest` con la plantilla por defecto del SDK 57. Si tu app actual tiene otra estructura, crea un proyecto limpio con ese comando y trabaja allí; después podrás migrar lo que quieras conservar.

> **Ojo.** Antes de empezar, **protege tu trabajo actual**: crea una rama (`git checkout -b biblioteca-api`). Esta guía reemplaza las pantallas de la plantilla (`index`, `explore`…) y la configuración de rutas. Si tienes pantallas propias que quieras conservar, muévelas a otra carpeta.

2.2 Retira el acceso directo a Firebase

Tu app actual usa el SDK de Firebase en el cliente. Con esta arquitectura **ya no debe hacerlo**: todo pasa por la API. Cuando termines la semana 10, haz esto en tu app original (si la conservas):

- Elimina la configuración e inicialización de Firebase (`firebaseConfig`, `initializeApp`) y los imports de `firebase/auth` y `firebase/firestore`.
- Reemplaza cada llamada de autenticación por las funciones de `src/services/auth/auth-service.ts` (semana 10) y cada lectura de Firestore por los hooks de `src/hooks/use-library-data.ts` (semana 11).
- Desinstala el paquete: `npm uninstall firebase`.

> **Idea.** La razón: una app móvil se puede descomprimir; cualquier clave o regla de negocio que viva en ella es pública y modificable. Con la API, las claves de administrador nunca salen del servidor y las reglas (ejemplares, roles, conflictos) no se pueden saltar.

2.3 Instalar los módulos que usaremos

Instala **siempre** con `npx expo install` (no con `npm install`): Expo elige la versión compatible con tu SDK.

▸ **TERMINAL** · **library_app**

```
npx expo install expo-sqlite expo-secure-store @react-native-community/netinfo
npx expo install expo-image-picker expo-image-manipulator expo-crypto
npx expo install jest-expo jest @types/jest --dev
npx expo install @testing-library/react-native --dev
npx expo install eslint eslint-config-expo --dev
```

Módulo	Para qué
`expo-sqlite`	Base de datos SQLite local (caché y cola offline)
`expo-secure-store`	Guardar los tokens de sesión en el Keychain / Keystore
`@react-native-community/netinfo`	Detectar si hay conexión
`expo-image-picker` · `expo-image-manipulator`	Cámara/galería y reducción de imágenes
`expo-crypto`	Generar identificadores únicos (UUID)
`jest-expo`, `jest`, `@testing-library/react-native`	Pruebas automáticas

Módulo	Para qué
`eslint`, `eslint-config-expo`	Reglas de calidad de código

> **Nota.** En Windows, si tu terminal no acepta `--dev`, escríbelo como `"--" --dev` (con las comillas), según la documentación de Expo.

2.4 Eliminar los archivos de ejemplo de la plantilla

La plantilla trae pantallas y componentes de ejemplo que no usaremos. Elimínalos (si tu proyecto no tiene alguno, sigue adelante):

▸ ELIMINAR · `library_app/src/app/explore.tsx`

▸ ELIMINAR · `library_app/src/app/index.tsx`

▸ ELIMINAR · `library_app/src/components/animated-icon.module.css`

▸ ELIMINAR · `library_app/src/components/animated-icon.tsx`

▸ ELIMINAR · `library_app/src/components/animated-icon.web.tsx`

▸ ELIMINAR · `library_app/src/components/app-tabs.tsx`

▸ ELIMINAR · `library_app/src/components/app-tabs.web.tsx`

▸ ELIMINAR · `library_app/src/components/external-link.tsx`

▸ ELIMINAR · `library_app/src/components/hint-row.tsx`

▸ ELIMINAR · `library_app/src/components/web-badge.tsx`

▸ ELIMINAR · `library_app/src/components/ui/collapsible.tsx`

▸ ELIMINAR · `library_app/src/components/themed-view.tsx`

▸ ELIMINAR · `library_app/src/global.css`

▸ ELIMINAR · `library_app/src/hooks/use-color-scheme.web.ts`

> **Nota.** Esta app es **solo móvil** (Android e iOS): por eso también eliminamos los archivos `.web` y los estilos web de la plantilla. `expo-sqlite` no se usa en web.

2.5 Configuración del proyecto

`app.json`**.** Pon el nombre de tu app e identificadores únicos (reemplaza `estudiante` por tu usuario). Se usan al compilar el instalable en la semana 15:

▸ **AGREGAR A · library_app/app.json**

```
{
  "expo": {
    "name": "Biblioteca",
    "android": { "package": "com.estudiante.biblioteca" },
    "ios": { "bundleIdentifier": "com.estudiante.biblioteca" }
  }
}
```

`package.json`**.** Agrega los scripts y la configuración de Jest (el alias `@/` debe resolverse también en las pruebas):

▸ **AGREGAR A · library_app/package.json**

```
{
  "scripts": {
    "test": "jest",
    "typecheck": "tsc --noEmit"
  },
  "jest": {
    "preset": "jest-expo",
    "moduleNameMapper": {
      "^@/assets/(.*)$": "<rootDir>/assets/$1",
      "^@/(.*)$": "<rootDir>/src/$1"
    },
    "testPathIgnorePatterns": ["/node_modules/", "/.expo/"]
  }
}
```

`tsconfig.json`**.** Agrega los tipos de Jest y de Node (las pruebas usan `node:sqlite`):

▸ **AGREGAR A · library_app/tsconfig.json**

```
{
  "compilerOptions": {
    "types": ["jest", "node"]
  }
}
```

ESLint. Crea la configuración (la misma que genera `npx expo lint`):

▸ **CREAR · library_app/eslint.config.js**

```
// https://docs.expo.dev/guides/using-eslint/
const { defineConfig } = require('eslint/config');
const expoConfig = require("eslint-config-expo/flat");

module.exports = defineConfig([
  expoConfig,
  {
    ignores: ["dist/*"],
  }
]);
```

Variables de entorno. Crea la plantilla `.env.example` y cópiala a `.env` con la URL de tu API (ver 1.4):

▸ **CREAR · library_app/.env.example**

```
# Copia este archivo a .env y ajusta la URL de library_api (semana 9: configuración por ambientes).
# Expo expone al bundle SOLO las variables con prefijo EXPO_PUBLIC_ → nunca pongas secretos aquí.
#
#  - Dispositivo físico (Expo Go): IP local de tu PC en la misma red Wi-Fi, ej. http://192.168.1.20:3000
#    (y levanta la API con `pnpm dev:lan` para que acepte conexiones de la red)
#  - Emulador Android:             http://10.0.2.2:3000
#  - Simulador iOS:                http://localhost:3000
EXPO_PUBLIC_API_URL=http://192.168.1.20:3000

cp .env.example .env        # en PowerShell: Copy-Item .env.example .env
```

2.6 Base de interfaz: tema, texto, pantalla y botón

Estos cinco archivos son la base visual de todas las pantallas. No tienen lógica de negocio, pero todo lo demás depende de ellos.

El **tema** define colores claros y oscuros, espaciado y medidas:

▸ **CREAR · library_app/src/constants/theme.ts**

```
import { Platform } from 'react-native';

/** Paleta clara y oscura. Los componentes la leen con el hook useTheme(). */
export const Colors = {
  light: {
    text: '#000000',
    background: '#ffffff',
    backgroundElement: '#F0F0F3',
    backgroundSelected: '#E0E1E6',
    textSecondary: '#60646C',
    primary: '#2563EB',
    onPrimary: '#FFFFFF',
    border: '#D4D6DC',
    danger: '#B42318',
    success: '#067647',
    warning: '#B54708',
  },
  dark: {
    text: '#ffffff',
    background: '#000000',
    backgroundElement: '#212225',
    backgroundSelected: '#2E3135',
    textSecondary: '#B0B4BA',
    primary: '#60A5FA',
    onPrimary: '#0B1220',
    border: '#3A3D44',
    danger: '#F97066',
    success: '#47CD89',
    warning: '#FDB022',
  },
} as const;

export type ThemeColor = keyof typeof Colors.light & keyof typeof Colors.dark;

/** Escala de espaciado (múltiplos de 4 px). */
export const Spacing = {
  half: 2,
  one: 4,
  two: 8,
  three: 16,
  four: 24,
  five: 32,
  six: 64,
} as const;

/** Espacio inferior para que el contenido no quede tapado por la barra de pestañas. */
export const BottomTabInset = Platform.select({ ios: 50, android: 80 }) ?? 0;

/** Ancho máximo del contenido en pantallas grandes (tabletas). */
export const MaxContentWidth = 800;
```

Dos hooks leen el esquema de color del sistema y devuelven la paleta correcta:

▸ **CREAR · library_app/src/hooks/use-color-scheme.ts**

```
export { useColorScheme } from 'react-native';
```

▸ **CREAR · library_app/src/hooks/use-theme.ts**

```
/**
 * Learn more about light and dark modes:
 * https://docs.expo.dev/guides/color-schemes/
 */

import { Colors } from '@/constants/theme';
import { useColorScheme } from '@/hooks/use-color-scheme';

export function useTheme() {
  const scheme = useColorScheme();
  const theme = scheme === 'unspecified' ? 'light' : scheme;

  return Colors[theme];
}
```

`ThemedText` es un texto que toma su color del tema y ofrece los estilos tipográficos:

▸ **CREAR · library_app/src/components/themed-text.tsx**

```
import { StyleSheet, Text, type TextProps } from 'react-native';

import type { ThemeColor } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

export type ThemedTextProps = TextProps & {
  type?: 'default' | 'title' | 'subtitle' | 'small' | 'smallBold' | 'link' | 'linkPrimary';
  themeColor?: ThemeColor;
};

/** Texto que toma su color del tema (claro/oscuro) y ofrece los estilos tipográficos de la app. */
export function ThemedText({ style, type = 'default', themeColor, ...rest }: ThemedTextProps) {
  const theme = useTheme();

  return (
    <Text
      style={[
        { color: theme[themeColor ?? 'text'] },
        type === 'default' && styles.default,
        type === 'title' && styles.title,
        type === 'subtitle' && styles.subtitle,
        type === 'small' && styles.small,
        type === 'smallBold' && styles.smallBold,
        type === 'link' && styles.link,
        type === 'linkPrimary' && styles.linkPrimary,
        style,
      ]}
      {...rest}
    />
  );
}

const styles = StyleSheet.create({
  default: { fontSize: 16, lineHeight: 24, fontWeight: 500 },
  title: { fontSize: 48, fontWeight: 600, lineHeight: 52 },
  subtitle: { fontSize: 32, lineHeight: 44, fontWeight: 600 },
  small: { fontSize: 14, lineHeight: 20, fontWeight: 500 },
  smallBold: { fontSize: 14, lineHeight: 20, fontWeight: 700 },
  link: { lineHeight: 30, fontSize: 14 },
  linkPrimary: { lineHeight: 30, fontSize: 14, color: '#3c87f7' },
});
```

`Screen` respeta las zonas seguras (notch, barra de gestos) y limita el ancho en pantallas grandes. `Button` es accesible: rol de botón, estado `busy`/`disabled` y área táctil mínima de 48 px.

▸ **CREAR · library_app/src/components/screen.tsx**

```
import { StyleSheet, View, type ViewProps } from 'react-native';
import { SafeAreaView, type Edge } from 'react-native-safe-area-context';

import { MaxContentWidth, Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

interface ScreenProps extends ViewProps {
  edges?: Edge[];
}

/** Contenedor base: respeta las zonas seguras (notch, barra de gestos) y limita el ancho en pantallas grandes. */
export function Screen({
  children,
  edges = ['top', 'left', 'right'],
  style,
  ...rest
}: ScreenProps) {
  const theme = useTheme();
  return (
    <SafeAreaView edges={edges} style={[styles.safe, { backgroundColor: theme.background }]}>
      <View style={[styles.content, style]} {...rest}>
        {children}
      </View>
    </SafeAreaView>
  );
```

```
}

const styles = StyleSheet.create({
  safe: { flex: 1 },
  content: {
    flex: 1,
    width: '100%',
    maxWidth: MaxContentWidth,
    alignSelf: 'center',
    paddingHorizontal: Spacing.three,
  },
});
```

▸ **CREAR · library_app/src/components/button.tsx**

```
import { ActivityIndicator, Pressable, StyleSheet, type PressableProps } from 'react-native';

import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

import { ThemedText } from './themed-text';

type Variant = 'primary' | 'secondary' | 'danger';

interface ButtonProps extends Omit<PressableProps, 'children' | 'style'> {
  title: string;
  variant?: Variant;
  loading?: boolean;
}

export function Button({
  title,
  variant = 'primary',
  loading = false,
  disabled,
  ...rest
}: ButtonProps) {
  const theme = useTheme();
  const isDisabled = disabled || loading;

  const background =
    variant === 'primary' ? theme.primary : variant === 'danger' ? theme.danger : 'transparent';
  const color =
    variant === 'secondary' ? theme.text : variant === 'primary' ? theme.onPrimary : '#FFFFFF';

  return (
    <Pressable
      accessibilityRole="button"
      accessibilityState={{ disabled: isDisabled, busy: loading }}
      disabled={isDisabled}
      style={({ pressed }) => [
        styles.base,
        {
          backgroundColor: background,
          borderColor: variant === 'secondary' ? theme.border : background,
        },
        pressed && styles.pressed,
        isDisabled && styles.disabled,
      ]}
      {...rest}>
      {loading ? (
        <ActivityIndicator color={color} />
      ) : (
        <ThemedText type="smallBold" style={{ color, fontSize: 16 }}>
          {title}
        </ThemedText>
      )}
    </Pressable>
  );
}

const styles = StyleSheet.create({
  base: {
    minHeight: 48, // área táctil mínima recomendada (accesibilidad)
    paddingHorizontal: Spacing.four,
    borderRadius: 12,
    borderWidth: 1,
    alignItems: 'center',
```

```
    justifyContent: 'center',
  },
  pressed: { opacity: 0.8 },
  disabled: { opacity: 0.5 },
});
```

Por último, un **layout raíz provisional**: solo declara las dos pantallas que crearemos en la semana 9. En la semana 10 lo reemplazaremos por uno que protege las rutas con la sesión.

▸ **CREAR · library_app/src/app/_layout.tsx**

```
import { Stack } from 'expo-router';

/** Versión provisional (semana 9): solo navegación. En la semana 10 se protege con la sesión. */
export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: 'Catálogo' }} />
      <Stack.Screen name="book/[id]" options={{ title: 'Detalle del libro' }} />
    </Stack>
  );
}
```

Nota. Punto de control 0. `npm run typecheck` y `npx expo lint` sin errores. Si el typecheck dice que no encuentra rutas o `expo-env.d.ts`, ejecuta `npx expo start` unos segundos: Expo genera esos archivos de tipos la primera vez.

3. Semana 9 · Consumo de API REST

Del microcurrículo: Consumo de API REST desde React Native: HTTP, JSON, asincronía, configuración por ambientes y estados de carga, vacío, error y reintento. Presencial: comparación guiada del consumo de API en web y móvil. Independiente: integrar la app con la API del proyecto y probar respuestas válidas, errores y pérdida de conectividad.

3.1 Conceptos clave

- **HTTP y JSON.** La app envía una petición (método + ruta + cabeceras + cuerpo) y recibe un código de estado y un cuerpo JSON. `2xx` es éxito, `4xx` un problema de la petición y `5xx` del servidor.
- **Asincronía.** `fetch` devuelve una `Promise`; usamos `async/await`. Hay tres desenlaces: respuesta correcta, respuesta con error y ninguna respuesta.
- **Timeout.** Sin tiempo máximo, una petición puede quedarse colgada para siempre. Usamos `AbortController` para cancelarla a los 10 segundos.
- **Configuración por ambientes.** La URL de desarrollo no es la de producción. Expo lee `.env` y expone al código las variables con prefijo `EXPO_PUBLIC_`.

> **Ojo.** Todo lo que lleve el prefijo `EXPO_PUBLIC_` queda **visible dentro de la app instalada**. Allí van URLs, nunca claves secretas.

Comparación guiada: consumir una API en web y en móvil

Aspecto	Web (React en el navegador)	Móvil (React Native)
Llamada	`fetch`	`fetch` (el mismo)
CORS	El navegador lo exige: la API debe permitir el origen	**No existe**: la app no es un navegador
`localhost`	El propio computador	**El propio teléfono**
HTTP sin cifrar	Los navegadores lo toleran en `localhost`	Android bloquea HTTP en apps publicadas; iOS exige excepciones (ATS)
Variables de entorno	`NEXT_PUBLIC_*` / `VITE_*`	`EXPO_PUBLIC_*`
Dónde guardar sesión	Cookie `httpOnly`	`expo-secure-store` (Keychain / Keystore)
Estado sin red	`navigator.onLine`, caché del navegador	NetInfo y base de datos local (SQLite)

3.2 Paso a paso

Paso 9.1 · Configuración por ambiente

▸ **CREAR · library_app/src/config/env.ts**

```
/**
 * Configuración por ambiente (semana 9).
 *
 * Expo inyecta en el bundle las variables que empiezan con EXPO_PUBLIC_ leyéndolas de
 * `.env`, `.env.development`, `.env.production`, etc. Deben referenciarse de forma literal
 * (process.env.EXPO_PUBLIC_X), no desestructuradas.
 *
 * ⚠ Todo lo que lleve prefijo EXPO_PUBLIC_ queda visible dentro de la app: nunca pongas secretos aquí.
 */
const rawApiUrl = process.env.EXPO_PUBLIC_API_URL;

/** URL base de library_api, sin "/" final. En un dispositivo físico usa la IP de tu PC, no localhost. */
export const API_URL = (
  rawApiUrl && rawApiUrl.length > 0 ? rawApiUrl : 'http://localhost:3000'
).replace(/\/+$/, '');

/** Tiempo máximo de espera por petición antes de considerarla un fallo de red. */
export const REQUEST_TIMEOUT_MS = 10_000;
```

Paso 9.2 · El contrato de datos (tipos)

Los tipos espejan lo que devuelve la API (Guía 1, paso 9.1). Si la API cambia, TypeScript te avisa en toda la app.

▸ **CREAR · library_app/src/types/index.ts**

```
export type Role = 'student' | 'librarian';

export interface User {
  uid: string;
  email: string;
  name: string;
  role: Role;
}

export interface TokenSet {
  idToken: string;
  refreshToken: string;
  /** Segundos de vigencia del idToken. */
  expiresIn: number;
}

export interface AuthResult {
  tokens: TokenSet;
  user: User;
}

export interface Book {
  id: string;
  title: string;
  author: string;
  category: string;
  year: number;
  description: string;
  totalCopies: number;
  availableCopies: number;
  /** Ruta relativa a la API o null. */
  coverUrl: string | null;
  updatedAt: string;
}

export type LoanStatus = 'requested' | 'approved' | 'rejected' | 'returned';
export type LoanAction = 'approve' | 'reject' | 'return';

/**
 * Estado de sincronización de un préstamo guardado en SQLite:
 * - synced:   coincide con el servidor
 * - pending:  creado sin conexión, esperando en la cola (outbox)
 * - conflict: el servidor lo rechazó al sincronizar (gana el servidor); ver `syncError`
 */
export type SyncState = 'synced' | 'pending' | 'conflict';

export interface Loan {
  id: string;
  bookId: string;
```

```
  bookTitle: string;
  userId: string;
  userEmail: string;
  userName: string;
  status: LoanStatus;
  clientRequestId: string;
  requestedAt: string;
  resolvedAt: string | null;
  dueDate: string | null;
  returnedAt: string | null;
  updatedAt: string;
}

export interface LocalLoan extends Loan {
  syncState: SyncState;
  syncError: string | null;
}
```

Paso 9.3 · El cliente HTTP

Es la pieza más importante de la semana. Distingue dos clases de fallo, porque la app reacciona distinto a cada una:

- `NetworkError` → no hubo respuesta (sin internet, servidor apagado, timeout). **Se puede reintentar** más tarde.
- `ApiRequestError` → el servidor respondió con un error. Trae `status` y un `code` estable (`NO_COPIES`, `ALREADY_ACTIVE`…).

Otros puntos: el `catch` de `fetch` solo se ejecuta cuando **no hay respuesta**; el timeout limpia su temporizador en el `finally`; y el bloque del `401` (renovar la sesión y reintentar una vez) lo usaremos en la semana 10. El cliente recibe `fetchImpl` por parámetro para poder reemplazarlo en las pruebas.

▸ **CREAR · library_app/src/services/api/client.ts**

```
/**
 * Cliente HTTP de la app (semana 9) con soporte de sesión (semana 10).
 *
 * Distingue dos clases de fallo, porque la app reacciona distinto a cada una:
 *  - NetworkError:    no hubo respuesta (sin internet, servidor caído, timeout) → se puede reintentar luego.
 *  - ApiRequestError: el servidor respondió con un error (4xx/5xx) → trae `status` y `code` estables.
 */

export type ApiErrorCode =
  | 'VALIDATION_ERROR'
  | 'UNAUTHENTICATED'
  | 'INVALID_CREDENTIALS'
  | 'FORBIDDEN'
  | 'NOT_FOUND'
  | 'EMAIL_IN_USE'
  | 'ALREADY_ACTIVE'
  | 'NO_COPIES'
  | 'INVALID_STATE'
  | 'RATE_LIMITED'
  | 'CONFIG_ERROR'
  | 'INTERNAL_ERROR'
  | 'UNKNOWN';

export class ApiRequestError extends Error {
  constructor(
    readonly status: number,
    readonly code: ApiErrorCode,
    message: string,
  ) {
    super(message);
    this.name = 'ApiRequestError';
  }
}

export class NetworkError extends Error {
  constructor(message = 'No hay conexión con el servidor') {
    super(message);
```

```
    this.name = 'NetworkError';
  }
}

export interface RequestOptions {
  method?: 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE';
  body?: unknown;
  /** false para endpoints públicos (login, registro, refresh). Por defecto true. */
  auth?: boolean;
  signal?: AbortSignal;
}

export interface AuthHooks {
  /** Devuelve el idToken vigente (o null si no hay sesión). */
  getAccessToken: () => Promise<string | null>;
  /** Intenta renovar la sesión. true si se renovó y vale la pena reintentar la petición. */
  refreshSession: () => Promise<boolean>;
}

export interface ApiClientOptions {
  baseUrl: string;
  timeoutMs: number;
  fetchImpl?: typeof fetch;
  auth?: AuthHooks;
}

export interface ApiClient {
  request<T>(path: string, options?: RequestOptions): Promise<T>;
  get<T>(path: string, options?: Omit<RequestOptions, 'method' | 'body'>): Promise<T>;
  post<T>(
    path: string,
    body?: unknown,
    options?: Omit<RequestOptions, 'method' | 'body'>,
  ): Promise<T>;
  put<T>(
    path: string,
    body?: unknown,
    options?: Omit<RequestOptions, 'method' | 'body'>,
  ): Promise<T>;
  patch<T>(
    path: string,
    body?: unknown,
    options?: Omit<RequestOptions, 'method' | 'body'>,
  ): Promise<T>;
}

function toApiError(status: number, body: unknown): ApiRequestError {
  const err = (body as { error?: { code?: string; message?: string } } | null)?.error;
  return new ApiRequestError(
    status,
    (err?.code as ApiErrorCode | undefined) ?? 'UNKNOWN',
    err?.message ?? `Error del servidor (${status})`,
  );
}

export function createApiClient({
  baseUrl,
  timeoutMs,
  fetchImpl,
  auth,
}: ApiClientOptions): ApiClient {
  const doFetch = fetchImpl ?? fetch;

  async function send(
    path: string,
    options: RequestOptions,
    token: string | null,
  ): Promise<Response> {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), timeoutMs);
    // Si el llamador también cancela (ej. al desmontar una pantalla), se propaga.
    options.signal?.addEventListener('abort', () => controller.abort());

    try {
      return await doFetch(`${baseUrl}${path}`, {
        method: options.method ?? 'GET',
        headers: {
          Accept: 'application/json',
```

```
          ...(options.body !== undefined ? { 'Content-Type': 'application/json' } : {}),
          ...(token ? { Authorization: `Bearer ${token}` } : {}),
        },
        body: options.body !== undefined ? JSON.stringify(options.body) : undefined,
        signal: controller.signal,
      });
    } catch {
      // fetch solo rechaza cuando no hay respuesta: red caída, DNS, timeout o cancelación.
      throw new NetworkError();
    } finally {
      clearTimeout(timer);
    }
  }

  async function request<T>(path: string, options: RequestOptions = {}): Promise<T> {
    const useAuth = options.auth !== false && auth !== undefined;
    let token = useAuth ? await auth!.getAccessToken() : null;
    let res = await send(path, options, token);

    // 401 con sesión: el idToken venció → renovar una vez y reintentar.
    if (res.status === 401 && useAuth) {
      if (await auth!.refreshSession()) {
        token = await auth!.getAccessToken();
        res = await send(path, options, token);
      }
    }

    let body: unknown = null;
    try {
      body = await res.json();
    } catch {
      // Respuesta sin cuerpo JSON (ej. 502 de un proxy).
    }
    if (!res.ok) throw toApiError(res.status, body);
    return body as T;
  }

  return {
    request,
    get: (path, options) => request(path, { ...options, method: 'GET' }),
    post: (path, body, options) => request(path, { ...options, method: 'POST', body }),
    put: (path, body, options) => request(path, { ...options, method: 'PUT', body }),
    patch: (path, body, options) => request(path, { ...options, method: 'PATCH', body }),
  };
}

/** Mensaje legible para mostrar al usuario a partir de cualquier error. */
export function describeError(err: unknown): string {
  if (err instanceof NetworkError)
    return 'No hay conexión con el servidor. Revisa tu internet e inténtalo de nuevo.';
  if (err instanceof ApiRequestError) return err.message;
  return 'Ocurrió un error inesperado.';
}
```

Paso 9.4 · Conectar el cliente con la API (versión provisional)

La API exige sesión, pero todavía no tenemos pantallas de inicio de sesión (son de la semana 10). Mientras tanto, la app entra sola con **tu estudiante de prueba** (el que creaste en la Guía 1). Es un andamio: lo eliminamos en la semana 10.

Para no escribir la contraseña en el código, agrega estas dos líneas a tu archivo `.env` (que no se sube a git; usa tu propio correo y contraseña):

```
EXPO_PUBLIC_DEV_EMAIL=estudiante@tucorreo.com
EXPO_PUBLIC_DEV_PASSWORD=TuClaveEstudiante
```

> **Ojo.** Todo `EXPO_PUBLIC_*` queda **visible dentro de la app**. Por eso esto es solo provisional y para desarrollo: en la semana 10 las borras y la sesión pasa a ser la del formulario de inicio de sesión.

▸ **CREAR · library_app/src/services/api/index.ts**

```
import { API_URL, REQUEST_TIMEOUT_MS } from '@/config/env';
import type { AuthResult } from '@/types';

import { createApiClient } from './client';

/**
 * VERSIÓN PROVISIONAL (semana 9). La API exige un token pero todavía no hay pantallas de inicio de
 * sesión, así que la app entra con tu estudiante de prueba (variables del .env) la primera vez que
 * lo necesita. ⚠ Se elimina en la semana 10: nunca dejes credenciales en una app real.
 */
const publicApi = createApiClient({ baseUrl: API_URL, timeoutMs: REQUEST_TIMEOUT_MS });

let token: string | null = null;

async function demoLogin(): Promise<string> {
  const email = process.env.EXPO_PUBLIC_DEV_EMAIL;
  const password = process.env.EXPO_PUBLIC_DEV_PASSWORD;
  if (!email || !password) {
    throw new Error('Define EXPO_PUBLIC_DEV_EMAIL y EXPO_PUBLIC_DEV_PASSWORD en el archivo .env');
  }
  const result = await publicApi.post<AuthResult>(
    '/api/auth/login',
    { email, password },
    { auth: false },
  );
  token = result.tokens.idToken;
  return token;
}

export const api = createApiClient({
  baseUrl: API_URL,
  timeoutMs: REQUEST_TIMEOUT_MS,
  auth: {
    getAccessToken: async () => token ?? demoLogin(),
    refreshSession: async () => {
      await demoLogin();
      return true;
    },
  },
});
```

Paso 9.5 · Los cuatro estados de una pantalla

Toda pantalla que depende de datos remotos debe estar siempre en uno de estos estados: **cargando**, **listo**, **vacío** o **error con reintento**. Los componentes son pequeños y reutilizables. Fíjate en que solo el texto del error se agrupa como alerta, para que el botón Reintentar siga siendo enfocable con lectores de pantalla.

▸ **CREAR · library_app/src/components/state-views.tsx**

```
import { ActivityIndicator, StyleSheet, View } from 'react-native';

import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

import { Button } from './button';
import { ThemedText } from './themed-text';

/** Los estados de una pantalla que consume datos (semana 9): cargando, vacío y error con reintento. */

export function LoadingView({ message = 'Cargando…' }: { message?: string }) {
  const theme = useTheme();
  return (
    <View style={styles.center} accessibilityRole="progressbar" accessibilityLabel={message}>
      <ActivityIndicator size="large" color={theme.primary} />
      <ThemedText themeColor="textSecondary">{message}</ThemedText>
    </View>
  );
}

export function EmptyView({ title, message }: { title: string; message?: string }) {
  return (
    <View style={styles.center}>
```

```
      <ThemedText type="subtitle" style={styles.centerText}>
        {title}
      </ThemedText>
      {message ? (
        <ThemedText themeColor="textSecondary" style={styles.centerText}>
          {message}
        </ThemedText>
      ) : null}
    </View>
  );
}

export function ErrorView({ message, onRetry }: { message: string; onRetry: () => void }) {
  const theme = useTheme();
  return (
    <View style={styles.center}>
      {/* Solo el texto se agrupa como alerta; el botón queda aparte para que siga siendo enfocable. */}
      <View accessible accessibilityRole="alert" style={styles.alert}>
        <ThemedText type="subtitle" style={[styles.centerText, { color: theme.danger }]}>
          No pudimos cargar los datos
        </ThemedText>
        <ThemedText themeColor="textSecondary" style={styles.centerText}>
          {message}
        </ThemedText>
      </View>
      <Button title="Reintentar" onPress={onRetry} />
    </View>
  );
}

const styles = StyleSheet.create({
  center: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    gap: Spacing.three,
    padding: Spacing.four,
  },
  centerText: { textAlign: 'center' },
  alert: { gap: Spacing.three, alignItems: 'center' },
});
```

La decisión de **cuál mostrar** se aísla en una función pura, fácil de probar. Observa la regla central: si hay datos guardados, siempre se muestran, aunque la red falle; el error se comunica con un aviso discreto. Esa regla convierte la app en offline-first en la semana 11.

▸ **CREAR · library_app/src/lib/list-status.ts**

```
export type ListStatus = 'loading' | 'ready' | 'empty' | 'error';

export interface ListStatusInput {
  /** La consulta local a SQLite ya respondió al menos una vez. */
  loaded: boolean;
  count: number;
  syncing: boolean;
  /** Mensaje del último intento de sincronización fallido (null si el último salió bien). */
  error: string | null;
  /** Ya hubo al menos una sincronización exitosa en este dispositivo. */
  everSynced: boolean;
}

/**
 * Los cuatro estados de una lista que consume API (semana 9): cargando, listo, vacío y error.
 * Con caché local (semana 11) hay una regla clave: si hay datos guardados, SIEMPRE se muestran,
 * aunque la red falle; el fallo solo se comunica con un aviso (OfflineBanner).
 */
export function deriveListStatus({
  loaded,
  count,
  syncing,
  error,
  everSynced,
}: ListStatusInput): ListStatus {
  if (!loaded) return 'loading';
  if (count > 0) return 'ready';
  if (syncing) return 'loading';
```

```
  if (error) return 'error';
  // Sin datos ni error: o aún no se sincronizó nunca (se mostrará cargando) o el catálogo está realmente vacío.
  return everSynced ? 'empty' : 'loading';
}
```

Paso 9.6 · Un hook para cargar con los cuatro estados

Primera iteración: un hook que ejecuta la petición y devuelve `{ state, retry }`. Funciona, pero tiene un defecto que descubrirás al probar con el modo avión: **sin internet no hay catálogo**. En la semana 11 reemplazamos esta fuente por SQLite sin cambiar las pantallas.

▸ **CREAR · library_app/src/hooks/use-resource.ts**

```
import { useCallback, useEffect, useState } from 'react';

import { describeError } from '@/services/api/client';

export type ResourceState<T> =
  | { status: 'loading' }
  | { status: 'error'; message: string }
  | { status: 'empty' }
  | { status: 'ready'; data: T };

/**
 * Carga un recurso remoto y expone sus cuatro estados (semana 9).
 * `requestKey` identifica la petición (incluye el texto de búsqueda): cuando cambia, se vuelve a cargar.
 */
export function useResource<T>(
  requestKey: string,
  load: () => Promise<T>,
  isEmpty: (data: T) => boolean,
) {
  const [attempt, setAttempt] = useState(0);
  const key = `${requestKey}#${attempt}`;
  const [result, setResult] = useState<{ key: string; state: ResourceState<T> } | null>(null);

  useEffect(() => {
    let active = true; // evita actualizar el estado si la pantalla ya se cerró o la petición quedó obsoleta
    load()
      .then((data) => {
        if (active)
          setResult({
            key,
            state: isEmpty(data) ? { status: 'empty' } : { status: 'ready', data },
          });
      })
      .catch((err) => {
        if (active) setResult({ key, state: { status: 'error', message: describeError(err) } });
      });
    return () => {
      active = false;
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [key]);

  // Mientras no haya resultado para ESTA petición, el estado es "cargando": no hace falta un setState síncrono.
  const state: ResourceState<T> = result?.key === key ? result.state : { status: 'loading' };
  const retry = useCallback(() => setAttempt((n) => n + 1), []);
  return { state, retry };
}
```

Paso 9.7 · Tarjeta y portada del libro

▸ **CREAR · library_app/src/components/book-views.tsx**

```
import { Image } from 'expo-image';
import { Link } from 'expo-router';
import { Pressable, StyleSheet, View } from 'react-native';

import { API_URL } from '@/config/env';
import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import type { Book } from '@/types';

import { ThemedText } from './themed-text';
```

```
export function BookCover({ book, width, height }: { book: Book; width: number; height: number }) {
  const theme = useTheme();
  const size = { width, height, borderRadius: 10 };

  if (book.coverUrl) {
    return (
      <Image
        source={{ uri: `${API_URL}${book.coverUrl}` }}
        style={size}
        contentFit="cover"
        transition={150}
        accessibilityLabel={`Portada de ${book.title}`}
      />
    );
  }
  return (
    <View
      accessible={false}
      style={[size, styles.placeholder, { backgroundColor: theme.backgroundSelected }]}>
      <ThemedText type="subtitle" themeColor="textSecondary">
        {book.title.charAt(0)}
      </ThemedText>
    </View>
  );
}

export function availabilityLabel(book: Pick<Book, 'availableCopies' | 'totalCopies'>): string {
  return book.availableCopies > 0
    ? `${book.availableCopies} de ${book.totalCopies} disponibles`
    : 'Sin ejemplares disponibles';
}

export function BookCard({ book }: { book: Book }) {
  const theme = useTheme();
  const available = book.availableCopies > 0;

  return (
    <Link href={{ pathname: '/book/[id]', params: { id: book.id } }} asChild>
      <Pressable
        accessibilityRole="button"
        accessibilityLabel={`${book.title}, de ${book.author}. ${availabilityLabel(book)}`}
        accessibilityHint="Abre el detalle del libro"
        style={({ pressed }) => [
          styles.card,
          { backgroundColor: theme.backgroundElement, borderColor: theme.border },
          pressed && { opacity: 0.85 },
        ]}>
        <BookCover book={book} width={64} height={88} />
        <View style={styles.info}>
          <ThemedText type="smallBold" numberOfLines={2} style={{ fontSize: 16 }}>
            {book.title}
          </ThemedText>
          <ThemedText type="small" themeColor="textSecondary" numberOfLines={1}>
            {book.author}
          </ThemedText>
          <ThemedText type="small" themeColor="textSecondary">
            {book.category} · {book.year}
          </ThemedText>
          <ThemedText type="smallBold" style={{ color: available ? theme.success : theme.danger }}>
            {availabilityLabel(book)}
          </ThemedText>
        </View>
      </Pressable>
    </Link>
  );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    gap: Spacing.three,
    padding: Spacing.three,
    borderRadius: 14,
    borderWidth: StyleSheet.hairlineWidth,
  },
  info: { flex: 1, gap: Spacing.half, justifyContent: 'center' },
  placeholder: { alignItems: 'center', justifyContent: 'center' },
```

```
});
```

Paso 9.8 · Las pantallas: catálogo y detalle (versión de la semana 9)

El catálogo consume la API directamente. Detalles de calidad móvil ya aplicados: `FlatList` (solo renderiza lo visible), `keyboardShouldPersistTaps="handled"` para que el teclado no se robe el primer toque, `useDeferredValue` para mantener fluida la escritura y accesibilidad en cada elemento.

▸ **CREAR · library_app/src/app/index.tsx**

```
import { useDeferredValue, useState } from 'react';
import { FlatList, StyleSheet, TextInput, View } from 'react-native';

import { BookCard } from '@/components/book-views';
import { Screen } from '@/components/screen';
import { EmptyView, ErrorView, LoadingView } from '@/components/state-views';
import { Spacing } from '@/constants/theme';
import { useResource } from '@/hooks/use-resource';
import { useTheme } from '@/hooks/use-theme';
import { api } from '@/services/api';
import type { Book } from '@/types';

/** Catálogo (semana 9): consume la API directamente y maneja los cuatro estados. */
export default function BooksScreen() {
  const theme = useTheme();
  const [search, setSearch] = useState('');
  const q = useDeferredValue(search);

  const { state, retry } = useResource(
    `books:${q}`,
    () => api.get<{ books: Book[] }>(`/api/books?q=${encodeURIComponent(q.trim())}`),
    (res) => res.books.length === 0,
  );

  return (
    <Screen edges={['left', 'right', 'bottom']}>
      <View style={styles.header}>
        <TextInput
          value={search}
          onChangeText={setSearch}
          placeholder="Buscar por título o autor"
          placeholderTextColor={theme.textSecondary}
          accessibilityLabel="Buscar libros por título o autor"
          returnKeyType="search"
          autoCorrect={false}
          style={[
            styles.search,
            {
              color: theme.text,
              borderColor: theme.border,
              backgroundColor: theme.backgroundElement,
            },
          ]}
        />
      </View>

      {state.status === 'loading' ? (
        <LoadingView message="Cargando catálogo…" />
      ) : state.status === 'error' ? (
        <ErrorView message={state.message} onRetry={retry} />
      ) : state.status === 'empty' ? (
        <EmptyView title="Sin resultados" message="Prueba con otro título o autor." />
      ) : (
        <FlatList
          data={state.data.books}
          keyExtractor={(b) => b.id}
          renderItem={({ item }) => <BookCard book={item} />}
          ItemSeparatorComponent={() => <View style={{ height: Spacing.two }} />}
          contentContainerStyle={styles.list}
          keyboardShouldPersistTaps="handled"
        />
      )}
    </Screen>
  );
}
```

```
const styles = StyleSheet.create({
  header: { paddingVertical: Spacing.two },
  search: {
    minHeight: 48,
    borderWidth: 1,
    borderRadius: 12,
    paddingHorizontal: Spacing.three,
    fontSize: 16,
  },
  list: { paddingBottom: Spacing.four },
});
```

▸ **CREAR · library_app/src/app/book/[id].tsx**

```
import { useLocalSearchParams } from 'expo-router';
import { ScrollView, StyleSheet, View } from 'react-native';

import { availabilityLabel, BookCover } from '@/components/book-views';
import { Screen } from '@/components/screen';
import { ErrorView, LoadingView } from '@/components/state-views';
import { ThemedText } from '@/components/themed-text';
import { Spacing } from '@/constants/theme';
import { useResource } from '@/hooks/use-resource';
import { useTheme } from '@/hooks/use-theme';
import { api } from '@/services/api';
import type { Book } from '@/types';

/** Detalle del libro (semana 9): solo lectura. El préstamo se agrega en la semana 11. */
export default function BookDetailScreen() {
  const theme = useTheme();
  const { id } = useLocalSearchParams<{ id: string }>();
  const { state, retry } = useResource(
    `book:${id}`,
    () => api.get<{ book: Book }>(`/api/books/${id}`),
    () => false,
  );

  if (state.status === 'loading') return <LoadingView />;
  if (state.status === 'error') return <ErrorView message={state.message} onRetry={retry} />;
  if (state.status === 'empty') return null; // no ocurre: isEmpty siempre devuelve false

  const { book } = state.data;
  return (
    <Screen edges={['left', 'right', 'bottom']}>
      <ScrollView contentContainerStyle={styles.content}>
        <View style={styles.cover}>
          <BookCover book={book} width={160} height={220} />
        </View>
        <ThemedText type="subtitle" accessibilityRole="header">
          {book.title}
        </ThemedText>
        <ThemedText themeColor="textSecondary">{book.author}</ThemedText>
        <ThemedText type="small" themeColor="textSecondary">
          {book.category} · {book.year}
        </ThemedText>
        <ThemedText
          type="smallBold"
          style={{ color: book.availableCopies > 0 ? theme.success : theme.danger }}>
          {availabilityLabel(book)}
        </ThemedText>
        <ThemedText style={styles.description}>{book.description}</ThemedText>
      </ScrollView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  content: { gap: Spacing.two, paddingVertical: Spacing.three },
  cover: { alignItems: 'center', paddingBottom: Spacing.two },
  description: { marginTop: Spacing.two, lineHeight: 24 },
});
```

Paso 9.9 · Pruebas de la semana

El cliente HTTP es la pieza más crítica, así que tiene pruebas unitarias: el Bearer token, los errores del servidor, las respuestas sin JSON (un `502` de un proxy), el timeout, la diferencia entre `NetworkError` y `ApiRequestError`, y el reintento tras `401`. La segunda prueba cubre la decisión de qué estado mostrar.

▸ **CREAR · library_app/__tests__/api-client.test.ts**

```
import {
  ApiRequestError,
  createApiClient,
  describeError,
  NetworkError,
} from '@/services/api/client';

const json = (status: number, body: unknown) =>
  new Response(JSON.stringify(body), { status, headers: { 'Content-Type': 'application/json' } });

describe('createApiClient', () => {
  it('envía el Bearer token y parsea la respuesta JSON', async () => {
    const fetchImpl = jest.fn().mockResolvedValue(json(200, { ok: true }));
    const client = createApiClient({
      baseUrl: 'http://api',
      timeoutMs: 1000,
      fetchImpl,
      auth: { getAccessToken: async () => 'tok', refreshSession: async () => false },
    });

    await expect(client.get('/api/books')).resolves.toEqual({ ok: true });
    const [url, init] = fetchImpl.mock.calls[0];
    expect(url).toBe('http://api/api/books');
    expect(init.headers.Authorization).toBe('Bearer tok');
  });

  it('no envía Authorization en endpoints públicos (auth: false)', async () => {
    const fetchImpl = jest.fn().mockResolvedValue(json(200, {}));
    const getAccessToken = jest.fn(async () => 'tok');
    const client = createApiClient({
      baseUrl: 'http://api',
      timeoutMs: 1000,
      fetchImpl,
      auth: { getAccessToken, refreshSession: async () => false },
    });
    await client.post('/api/auth/login', { a: 1 }, { auth: false });
    expect(fetchImpl.mock.calls[0][1].headers.Authorization).toBeUndefined();
    expect(getAccessToken).not.toHaveBeenCalled();
  });

  it('convierte errores del servidor en ApiRequestError con status y code', async () => {
    const fetchImpl = jest
      .fn()
      .mockResolvedValue(json(409, { error: { code: 'NO_COPIES', message: 'Sin ejemplares' } }));
    const client = createApiClient({ baseUrl: 'http://api', timeoutMs: 1000, fetchImpl });

    await expect(client.post('/api/loans', {})).rejects.toMatchObject({
      status: 409,
      code: 'NO_COPIES',
      message: 'Sin ejemplares',
    });
  });

  it('tolera respuestas de error sin JSON (ej. 502 de un proxy)', async () => {
    const fetchImpl = jest
      .fn()
      .mockResolvedValue(new Response('<html>Bad gateway</html>', { status: 502 }));
    const client = createApiClient({ baseUrl: 'http://api', timeoutMs: 1000, fetchImpl });
    await expect(client.get('/x')).rejects.toBeInstanceOf(ApiRequestError);
  });

  it('distingue "sin red" (NetworkError) de un error del servidor', async () => {
    const fetchImpl = jest.fn().mockRejectedValue(new TypeError('Network request failed'));
    const client = createApiClient({ baseUrl: 'http://api', timeoutMs: 1000, fetchImpl });
    await expect(client.get('/x')).rejects.toBeInstanceOf(NetworkError);
  });
```

```
  it('cancela la petición al superar el timeout', async () => {
    const fetchImpl = jest.fn(
      (_url: string, init: RequestInit) =>
        new Promise<Response>((_resolve, reject) => {
          init.signal!.addEventListener('abort', () => reject(new Error('aborted')));
        }),
    );
    const client = createApiClient({
      baseUrl: 'http://api',
      timeoutMs: 20,
      fetchImpl: fetchImpl as unknown as typeof fetch,
    });
    await expect(client.get('/x')).rejects.toBeInstanceOf(NetworkError);
  });

  it('ante 401 renueva la sesión y reintenta UNA vez con el token nuevo', async () => {
    const fetchImpl = jest
      .fn()
      .mockResolvedValueOnce(json(401, { error: { code: 'UNAUTHENTICATED', message: 'expirado' } }))
      .mockResolvedValueOnce(json(200, { ok: true }));
    let token = 'viejo';
    const client = createApiClient({
      baseUrl: 'http://api',
      timeoutMs: 1000,
      fetchImpl,
      auth: {
        getAccessToken: async () => token,
        refreshSession: async () => {
          token = 'nuevo';
          return true;
        },
      },
    });

    await expect(client.get('/api/loans')).resolves.toEqual({ ok: true });
    expect(fetchImpl).toHaveBeenCalledTimes(2);
    expect(fetchImpl.mock.calls[1][1].headers.Authorization).toBe('Bearer nuevo');
  });

  it('si la renovación falla, devuelve el 401 sin reintentar en bucle', async () => {
    const fetchImpl = jest
      .fn()
      .mockResolvedValue(json(401, { error: { code: 'UNAUTHENTICATED', message: 'x' } }));
    const client = createApiClient({
      baseUrl: 'http://api',
      timeoutMs: 1000,
      fetchImpl,
      auth: { getAccessToken: async () => 't', refreshSession: async () => false },
    });
    await expect(client.get('/x')).rejects.toMatchObject({ status: 401 });
    expect(fetchImpl).toHaveBeenCalledTimes(1);
  });
});

describe('describeError', () => {
  it('da mensajes legibles', () => {
    expect(describeError(new NetworkError())).toMatch(/conexión/i);
    expect(describeError(new ApiRequestError(400, 'VALIDATION_ERROR', 'Correo inválido'))).toBe(
      'Correo inválido',
    );
    expect(describeError(new Error('boom'))).toMatch(/inesperado/i);
  });
});
```

▸ **CREAR · library_app/__tests__/list-status.test.ts**

```
import { deriveListStatus } from '@/lib/list-status';

const base = { loaded: true, count: 0, syncing: false, error: null, everSynced: true };

describe('deriveListStatus', () => {
  it('cargando hasta que SQLite responda', () => {
    expect(deriveListStatus({ ...base, loaded: false })).toBe('loading');
  });
  it('con datos locales los muestra SIEMPRE, aunque haya error de red (offline-first)', () => {
    expect(deriveListStatus({ ...base, count: 3, error: 'Sin conexión' })).toBe('ready');
    expect(deriveListStatus({ ...base, count: 3, syncing: true })).toBe('ready');
  });
```

```
  it('sin datos y sincronizando → cargando', () => {
    expect(deriveListStatus({ ...base, syncing: true })).toBe('loading');
  });
  it('sin datos y con error → error (con reintento)', () => {
    expect(deriveListStatus({ ...base, error: 'Sin conexión', everSynced: false })).toBe('error');
  });
  it('sin datos tras sincronizar bien → vacío', () => {
    expect(deriveListStatus(base)).toBe('empty');
  });
  it('sin datos y aún sin primera sincronización ni error → cargando', () => {
    expect(deriveListStatus({ ...base, everSynced: false })).toBe('loading');
  });
});
```

Nota. **Punto de control de la semana 9.** `npm run typecheck`, `npx expo lint` y `npm test` en verde.

3.3 Cómo probar en el teléfono

Con la API encendida (`pnpm dev:lan`) y la URL correcta en `.env`, ejecuta `npx expo start` y escanea el QR con Expo Go. Comprueba cada fila y anótala en tu bitácora:

Escenario	Cómo provocarlo	Resultado esperado
Respuesta válida	Abrir la app	Aparecen los 10 libros
Estado vacío	Buscar `zzzz`	Mensaje Sin resultados
Servidor apagado	Detener la API y reabrir la app	Estado de error con botón Reintentar
Reintento exitoso	Encender la API y tocar Reintentar	La lista aparece
URL incorrecta	Cambiar `EXPO_PUBLIC_API_URL` a una IP inexistente (y `npx expo start -c`)	Error de conexión en ≤ 10 s, no se congela
Detalle	Tocar un libro	Portada, autor, año y disponibilidad

3.4 Evidencia y criterios

Evidencia para el docente. Repositorio con la lista y el detalle consumiendo la API; configuración por ambiente (`.env.example`); los cuatro estados visibles y probados; tabla de pruebas manuales completada; `npm test` en verde; entrada de la bitácora de IA.

- ☐ La URL de la API no está escrita en ningún archivo `.ts`
- ☐ Cada fallo de red muestra un mensaje comprensible y nunca deja la pantalla en blanco
- ☐ Hay botón de reintento
- ☐ La petición no queda colgada más de 10 segundos

4. Semana 10 · Autenticación, autorización y seguridad

Del microcurrículo: Autenticación, autorización y protección de rutas. Manejo de sesión y tokens con expo-secure-store; secretos, datos personales y registro seguro. Presencial: modelado de amenazas y laboratorio de autenticación. Independiente: implementar el flujo de acceso y permisos, y contrastar las sugerencias de IA con requisitos de seguridad y pruebas.

4.1 Conceptos clave

- **Autenticación** responde ¿quién eres?; **autorización** responde ¿qué puedes hacer? (estudiante o bibliotecario).
- **Token de acceso (`idToken`).** JWT firmado por Firebase que la app envía en cada petición. Dura **1 hora** e incluye el rol.
- **Token de renovación (`refreshToken`).** Larga duración; solo sirve para pedir un `idToken` nuevo. Es **el dato más sensible** de la app.
- **¿Dónde guardar los tokens?** `AsyncStorage` y SQLite guardan texto plano legible en un teléfono con root o en una copia de seguridad. `expo-secure-store` usa el Keychain (iOS) y el Keystore (Android), cifrados por el sistema.
- **El cliente nunca es de fiar.** Ocultar un botón es solo experiencia de usuario: la API vuelve a comprobar el rol en cada petición.

Modelado de amenazas (laboratorio)

Para cada activo: qué podría salir mal y qué control lo reduce. Referencia: OWASP MASVS.

Activo	Amenaza	Control en este proyecto
Tokens de sesión	Lectura desde una copia de seguridad o un teléfono con root	Solo en `expo-secure-store`, cada token en su propia llave
Contraseña	Se guarda o se imprime en logs	Nunca se persiste; solo viaja en el login por HTTPS
Rol del usuario	Un estudiante modifica la app para verse como bibliotecario	La API valida el rol del token en cada petición (`403`)
Préstamos de otro usuario	Un estudiante pide por id un préstamo ajeno	La API responde `404` como si no existiera
Datos de otra cuenta en el mismo teléfono	Un segundo usuario inicia sesión y hereda la cola de solicitudes	Al cambiar de cuenta se descartan préstamos y cola locales (semana 11)
Token robado en tránsito	Red Wi-Fi hostil	HTTPS en producción (HTTP solo en desarrollo)
Sesión eterna	Un `refreshToken` robado vive para siempre	La API puede revocarlo; al recibir `401/403` la app cierra la sesión
Registro abusivo	Alguien se registra como bibliotecario	El registro público **siempre** crea estudiantes

4.2 Paso a paso

Paso 10.1 · El gestor de sesión

Toda la lógica de sesión vive en una clase **sin dependencias de React**, lo que permite probarla fácilmente. Tres decisiones importantes:

- **Cada token en su propia llave:** iOS puede rechazar valores de SecureStore de más de ~2 KB y un `idToken` ronda 1 KB.
- **Renovación proactiva y única:** se renueva un minuto antes de que venza y, si cinco peticiones reciben `401` juntas, **comparten una sola renovación** (el `refreshToken` rota: enviar cinco rompería algunas).
- **Sin red no se cierra la sesión.** Un fallo de red no significa que la sesión sea inválida; solo un `401/403` del servidor la cierra. Restaurarla al abrir la app tampoco necesita internet.

▸ **CREAR · library_app/src/services/auth/session-manager.ts**

```
/**
 * Gestión de sesión (semana 10).
 *
 * - Los tokens viven SOLO en almacenamiento seguro (Keychain en iOS, Keystore en Android),
 *   nunca en AsyncStorage/SQLite ni en logs.
 * - Se guardan en llaves separadas: iOS puede rechazar valores de SecureStore mayores a ~2 KB
 *   y un idToken de Firebase (JWT) ronda 1 KB.
 * - Si no hay red al renovar, la sesión NO se cierra: la app sigue funcionando sin conexión.
 */
import type { ApiClient } from '@/services/api/client';
import { ApiRequestError, NetworkError } from '@/services/api/client';
import type { AuthResult, User } from '@/types';

/** Subconjunto de expo-secure-store; permite sustituirlo por una versión en memoria en pruebas. */
export interface SecureStorage {
  getItemAsync(key: string): Promise<string | null>;
  setItemAsync(key: string, value: string): Promise<void>;
  deleteItemAsync(key: string): Promise<void>;
}

const KEYS = {
  idToken: 'auth.idToken',
  refreshToken: 'auth.refreshToken',
  expiresAt: 'auth.expiresAt',
  user: 'auth.user',
} as const;

/** Renovar el token un poco antes de que venza para evitar peticiones que fallen por 401. */
const EXPIRY_MARGIN_MS = 60_000;

export type SessionListener = (user: User | null) => void;

export class SessionManager {
  private idToken: string | null = null;
  private refreshToken: string | null = null;
  private expiresAt = 0;
  private user: User | null = null;
  private refreshing: Promise<boolean> | null = null;
  private listeners = new Set<SessionListener>();

  /**
   * @param storage almacenamiento seguro
   * @param publicApi cliente SIN hooks de autenticación (el refresh no puede depender de sí mismo)
   */
  constructor(
    private readonly storage: SecureStorage,
    private readonly publicApi: Pick<ApiClient, 'post'>,
    private readonly now: () => number = Date.now,
  ) {}

  subscribe(listener: SessionListener): () => void {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
  }
```

```
  private emit() {
    for (const l of this.listeners) l(this.user);
  }

  getUser(): User | null {
    return this.user;
  }

  /** Restaura la sesión guardada al abrir la app (funciona sin conexión). */
  async restore(): Promise<User | null> {
    try {
      const [idToken, refreshToken, expiresAt, user] = await Promise.all([
        this.storage.getItemAsync(KEYS.idToken),
        this.storage.getItemAsync(KEYS.refreshToken),
        this.storage.getItemAsync(KEYS.expiresAt),
        this.storage.getItemAsync(KEYS.user),
      ]);
      if (!idToken || !refreshToken || !user) return null;
      this.idToken = idToken;
      this.refreshToken = refreshToken;
      this.expiresAt = Number(expiresAt) || 0;
      this.user = JSON.parse(user) as User;
      return this.user;
    } catch {
      // Datos corruptos o Keystore invalidado: se parte de cero.
      await this.clear();
      return null;
    }
  }

  async save({ tokens, user }: AuthResult): Promise<void> {
    this.idToken = tokens.idToken;
    this.refreshToken = tokens.refreshToken;
    this.expiresAt = this.now() + tokens.expiresIn * 1000;
    this.user = user;
    await Promise.all([
      this.storage.setItemAsync(KEYS.idToken, tokens.idToken),
      this.storage.setItemAsync(KEYS.refreshToken, tokens.refreshToken),
      this.storage.setItemAsync(KEYS.expiresAt, String(this.expiresAt)),
      this.storage.setItemAsync(KEYS.user, JSON.stringify(user)),
    ]);
    this.emit();
  }

  async clear(): Promise<void> {
    this.idToken = null;
    this.refreshToken = null;
    this.expiresAt = 0;
    this.user = null;
    await Promise.all(
      Object.values(KEYS).map((k) => this.storage.deleteItemAsync(k).catch(() => {})),
    );
    this.emit();
  }

  /** Para AuthHooks: devuelve un token vigente, renovándolo si está por vencer. */
  async getAccessToken(): Promise<string | null> {
    if (!this.idToken) return null;
    if (this.now() >= this.expiresAt - EXPIRY_MARGIN_MS) await this.refresh();
    // Si el refresh falló por red, se devuelve el token actual: el servidor decidirá.
    return this.idToken;
  }

  /**
   * Renueva la sesión con el refreshToken. Una sola renovación en vuelo a la vez:
   * si varias peticiones reciben 401 juntas, comparten la misma promesa.
   */
  refresh(): Promise<boolean> {
    if (!this.refreshToken) return Promise.resolve(false);
    this.refreshing ??= this.doRefresh().finally(() => {
      this.refreshing = null;
    });
    return this.refreshing;
  }

  private async doRefresh(): Promise<boolean> {
    try {
```

```
      const result = await this.publicApi.post<AuthResult>(
        '/api/auth/refresh',
        { refreshToken: this.refreshToken },
        { auth: false },
      );
      await this.save(result);
      return true;
    } catch (err) {
      if (err instanceof NetworkError) return false; // sin red: conservar la sesión
      if (err instanceof ApiRequestError && (err.status === 401 || err.status === 403)) {
        await this.clear(); // refreshToken inválido o cuenta deshabilitada
      }
      return false;
    }
  }
}
```

Paso 10.2 · Conectar la sesión con el cliente HTTP

Reemplaza el archivo provisional de la semana 9. Ahora hay **dos** clientes: uno público (login, registro y refresh) y uno autenticado que agrega el Bearer, renueva el token en `401` y reintenta una vez. El de renovación no puede depender de sí mismo.

▸ **CREAR · library_app/src/services/api/index.ts**

```
import * as SecureStore from 'expo-secure-store';

import { API_URL, REQUEST_TIMEOUT_MS } from '@/config/env';
import { SessionManager } from '@/services/auth/session-manager';

import { createApiClient } from './client';

/** Cliente sin sesión: login, registro y refresh. */
const publicApi = createApiClient({ baseUrl: API_URL, timeoutMs: REQUEST_TIMEOUT_MS });

/** Única instancia de sesión de la app (tokens en SecureStore). */
export const session = new SessionManager(SecureStore, publicApi);

/** Cliente autenticado: agrega Bearer, renueva el token en 401 y reintenta una vez. */
export const api = createApiClient({
  baseUrl: API_URL,
  timeoutMs: REQUEST_TIMEOUT_MS,
  auth: {
    getAccessToken: () => session.getAccessToken(),
    refreshSession: () => session.refresh(),
  },
});

export { publicApi };
```

> **Ojo.** Ya no existe el inicio de sesión automático: **borra del** `.env` **las líneas** `EXPO_PUBLIC_DEV_EMAIL` **y** `EXPO_PUBLIC_DEV_PASSWORD` (ya no se usan) y nunca dejes credenciales en una app. Desde ahora entras con el formulario de inicio de sesión, con tus usuarios de prueba.

Paso 10.3 · Servicio de autenticación y validación

Validamos en el cliente para dar respuesta inmediata, pero la API vuelve a validar todo.

▸ **CREAR · library_app/src/services/auth/auth-service.ts**

```
import type { ApiClient } from '@/services/api/client';
import type { AuthResult } from '@/types';

import type { SessionManager } from './session-manager';

/** Validación en el cliente: mejora la experiencia, pero la API vuelve a validar todo. */
export function validateCredentials(
  input: { email: string; password: string; name?: string },
  mode: 'signIn' | 'signUp',
) {
  const errors: { name?: string; email?: string; password?: string } = {};
```

```
  if (mode === 'signUp' && (input.name ?? '').trim().length < 2) {
    errors.name = 'Escribe tu nombre (mínimo 2 letras)';
  }
  if (!/^\S+@\S+\.\S+$/.test(input.email.trim())) {
    errors.email = 'Escribe un correo válido';
  }
  if (mode === 'signUp' ? input.password.length < 6 : input.password.length === 0) {
    errors.password =
      mode === 'signUp'
        ? 'La contraseña debe tener al menos 6 caracteres'
        : 'Escribe tu contraseña';
  }
  return errors;
}

export function createAuthService(publicApi: Pick<ApiClient, 'post'>, session: SessionManager) {
  return {
    async signIn(email: string, password: string) {
      const result = await publicApi.post<AuthResult>(
        '/api/auth/login',
        { email: email.trim(), password },
        { auth: false },
      );
      await session.save(result);
      return result.user;
    },

    async signUp(name: string, email: string, password: string) {
      const result = await publicApi.post<AuthResult>(
        '/api/auth/register',
        { displayName: name.trim(), email: email.trim(), password },
        { auth: false },
      );
      await session.save(result);
      return result.user;
    },

    signOut: () => session.clear(),
  };
}
```

Paso 10.4 · Proveedor de sesión

La suscripción a `session` es la clave: **cualquier** cambio de sesión (login, logout o un cierre forzado porque el `refreshToken` ya no sirve) actualiza el estado y las rutas reaccionan solas.

▸ **CREAR · library_app/src/providers/auth-provider.tsx**

```
import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from 'react';

import { publicApi, session } from '@/services/api';
import { createAuthService } from '@/services/auth/auth-service';
import type { User } from '@/types';

export type AuthStatus = 'loading' | 'signedOut' | 'signedIn';

interface AuthContextValue {
  status: AuthStatus;
  user: User | null;
  signIn: (email: string, password: string) => Promise<void>;
  signUp: (name: string, email: string, password: string) => Promise<void>;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextValue | null>(null);
const authService = createAuthService(publicApi, session);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [status, setStatus] = useState<AuthStatus>('loading');
```

```
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    let active = true;

    // Cualquier cambio de sesión (login, logout, o cierre forzado porque el refreshToken ya no sirve)
    // actualiza el estado; las rutas protegidas reaccionan solas.
    const unsubscribe = session.subscribe((next) => {
      setUser(next);
      setStatus(next ? 'signedIn' : 'signedOut');
    });

    session.restore().then((restored) => {
      if (!active) return;
      setUser(restored);
      setStatus(restored ? 'signedIn' : 'signedOut');
    });

    return () => {
      active = false;
      unsubscribe();
    };
  }, []);

  const signIn = useCallback(async (email: string, password: string) => {
    await authService.signIn(email, password);
  }, []);
  const signUp = useCallback(async (name: string, email: string, password: string) => {
    await authService.signUp(name, email, password);
  }, []);
  const signOut = useCallback(() => authService.signOut(), []);

  const value = useMemo(
    () => ({ status, user, signIn, signUp, signOut }),
    [status, user, signIn, signUp, signOut],
  );
  return <AuthContext value={value}>{children}</AuthContext>;
}

export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth debe usarse dentro de <AuthProvider>');
  return ctx;
}
```

Paso 10.5 · Campo de texto accesible

▸ **CREAR · library_app/src/components/text-field.tsx**

```
import type { Ref } from 'react';
import { StyleSheet, TextInput, View, type TextInputProps } from 'react-native';

import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

import { ThemedText } from './themed-text';

interface TextFieldProps extends TextInputProps {
  label: string;
  error?: string;
  ref?: Ref<TextInput>;
}

export function TextField({ label, error, style, ref, ...rest }: TextFieldProps) {
  const theme = useTheme();
  return (
    <View style={styles.wrapper}>
      <ThemedText type="smallBold">{label}</ThemedText>
      <TextInput
        ref={ref}
        accessibilityLabel={label}
        placeholderTextColor={theme.textSecondary}
        style={[
          styles.input,
          {
            color: theme.text,
            borderColor: error ? theme.danger : theme.border,
```

```
            backgroundColor: theme.backgroundElement,
          },
          style,
        ]}
        {...rest}
      />
      {error ? (
        <ThemedText type="small" style={{ color: theme.danger }} accessibilityLiveRegion="polite">
          {error}
        </ThemedText>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  wrapper: { gap: Spacing.one },
  input: {
    minHeight: 48,
    borderWidth: 1,
    borderRadius: 12,
    paddingHorizontal: Spacing.three,
    fontSize: 16,
  },
});
```

Paso 10.6 · Rutas protegidas con `Stack.Protected`

Expo Router permite declarar qué pantallas **existen** según una condición (`guard`). Con la sesión cerrada solo existen `sign-in` y `sign-up`; con la sesión abierta, el catálogo y el detalle. La pantalla de carga nativa se mantiene hasta saber si hay una sesión guardada, para que no parpadee el login.

▸ **CREAR · library_app/src/app/_layout.tsx**

```
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { useEffect } from 'react';
import { Pressable } from 'react-native';

import { ThemedText } from '@/components/themed-text';
import { AuthProvider, useAuth } from '@/providers/auth-provider';

// La pantalla de carga nativa permanece hasta saber si hay una sesión guardada.
SplashScreen.preventAutoHideAsync();

function RootStack() {
  const { status, signOut } = useAuth();

  useEffect(() => {
    if (status !== 'loading') SplashScreen.hideAsync();
  }, [status]);

  if (status === 'loading') return null;
  const signedIn = status === 'signedIn';

  return (
    <Stack>
      {/* Rutas protegidas (semana 10): el guard decide qué pantallas existen según la sesión. */}
      <Stack.Protected guard={!signedIn}>
        <Stack.Screen name="sign-in" options={{ headerShown: false }} />
        <Stack.Screen name="sign-up" options={{ title: 'Crear cuenta' }} />
      </Stack.Protected>
      <Stack.Protected guard={signedIn}>
        <Stack.Screen
          name="index"
          options={{
            title: 'Catálogo',
            headerRight: () => (
              <Pressable
                onPress={() => void signOut()}
                accessibilityRole="button"
                accessibilityLabel="Cerrar sesión">
                <ThemedText type="linkPrimary">Salir</ThemedText>
              </Pressable>
            ),
```

```
          }}
        />
        <Stack.Screen name="book/[id]" options={{ title: 'Detalle del libro' }} />
      </Stack.Protected>
    </Stack>
  );
}

export default function RootLayout() {
  return (
    <AuthProvider>
      <RootStack />
    </AuthProvider>
  );
}
```

> **Ojo.** Las rutas protegidas son una comodidad de **navegación**, no una barrera de seguridad. La seguridad real está en la API: sin un token válido no se obtiene ningún dato.

Paso 10.7 · Pantallas de inicio de sesión y registro

El botón queda en estado `loading` (evita el doble envío), los errores se anuncian con `accessibilityRole="alert"` y la navegación **no** está aquí: al cambiar la sesión, `Stack.Protected` hace el resto.

▸ **CREAR · library_app/src/app/sign-in.tsx**

```
import { Link } from 'expo-router';
import { useRef, useState } from 'react';
import {
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StyleSheet,
  TextInput,
  View,
} from 'react-native';

import { Button } from '@/components/button';
import { Screen } from '@/components/screen';
import { TextField } from '@/components/text-field';
import { ThemedText } from '@/components/themed-text';
import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import { useAuth } from '@/providers/auth-provider';
import { describeError } from '@/services/api/client';
import { validateCredentials } from '@/services/auth/auth-service';

export default function SignInScreen() {
  const theme = useTheme();
  const { signIn } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState<{ email?: string; password?: string }>({});
  const [formError, setFormError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const passwordRef = useRef<TextInput>(null);

  async function submit() {
    const found = validateCredentials({ email, password }, 'signIn');
    setErrors(found);
    setFormError(null);
    if (Object.keys(found).length > 0) return;

    setLoading(true);
    try {
      await signIn(email, password);
      // Sin navegación manual: al cambiar la sesión, Stack.Protected lleva a las pestañas.
    } catch (err) {
      setFormError(describeError(err));
    } finally {
      setLoading(false);
    }
```

```
  }

  return (
    <Screen>
      <KeyboardAvoidingView
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
        style={styles.flex}>
        <ScrollView contentContainerStyle={styles.form} keyboardShouldPersistTaps="handled">
          <ThemedText type="title" style={styles.title} accessibilityRole="header">
            Biblioteca
          </ThemedText>
          <ThemedText themeColor="textSecondary">
            Inicia sesión para ver y solicitar libros.
          </ThemedText>

          <TextField
            label="Correo"
            value={email}
            onChangeText={setEmail}
            error={errors.email}
            keyboardType="email-address"
            autoCapitalize="none"
            autoComplete="email"
            autoCorrect={false}
            textContentType="emailAddress"
            returnKeyType="next"
            onSubmitEditing={() => passwordRef.current?.focus()}
          />
          <TextField
            ref={passwordRef}
            label="Contraseña"
            value={password}
            onChangeText={setPassword}
            error={errors.password}
            secureTextEntry
            autoComplete="current-password"
            textContentType="password"
            returnKeyType="go"
            onSubmitEditing={submit}
          />

          {formError ? (
            <ThemedText accessibilityRole="alert" style={{ color: theme.danger }}>
              {formError}
            </ThemedText>
          ) : null}

          <Button title="Iniciar sesión" onPress={submit} loading={loading} />

          <View style={styles.footer}>
            <ThemedText themeColor="textSecondary">¿No tienes cuenta?</ThemedText>
            <Link href="/sign-up" accessibilityRole="link">
              <ThemedText type="linkPrimary">Regístrate</ThemedText>
            </Link>
          </View>
        </ScrollView>
      </KeyboardAvoidingView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  flex: { flex: 1 },
  form: {
    flexGrow: 1,
    justifyContent: 'center',
    gap: Spacing.three,
    paddingVertical: Spacing.four,
  },
  title: { fontSize: 40, lineHeight: 46 },
  footer: {
    flexDirection: 'row',
    gap: Spacing.two,
    justifyContent: 'center',
    alignItems: 'center',
  },
});
```

▸ **CREAR · library_app/src/app/sign-up.tsx**

```
import { useRef, useState } from 'react';
import { KeyboardAvoidingView, Platform, ScrollView, StyleSheet, TextInput } from 'react-native';

import { Button } from '@/components/button';
import { Screen } from '@/components/screen';
import { TextField } from '@/components/text-field';
import { ThemedText } from '@/components/themed-text';
import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import { useAuth } from '@/providers/auth-provider';
import { describeError } from '@/services/api/client';
import { validateCredentials } from '@/services/auth/auth-service';

export default function SignUpScreen() {
  const theme = useTheme();
  const { signUp } = useAuth();
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState<{ name?: string; email?: string; password?: string }>({});
  const [formError, setFormError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const emailRef = useRef<TextInput>(null);
  const passwordRef = useRef<TextInput>(null);

  async function submit() {
    const found = validateCredentials({ name, email, password }, 'signUp');
    setErrors(found);
    setFormError(null);
    if (Object.keys(found).length > 0) return;

    setLoading(true);
    try {
      await signUp(name, email, password);
    } catch (err) {
      setFormError(describeError(err));
    } finally {
      setLoading(false);
    }
  }

  return (
    <Screen edges={['left', 'right']}>
      <KeyboardAvoidingView
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
        style={styles.flex}>
        <ScrollView contentContainerStyle={styles.form} keyboardShouldPersistTaps="handled">
          <ThemedText themeColor="textSecondary">
            Las cuentas nuevas son de estudiante. El rol de bibliotecario lo asigna la
            administración.
          </ThemedText>

          <TextField
            label="Nombre"
            value={name}
            onChangeText={setName}
            error={errors.name}
            autoComplete="name"
            textContentType="name"
            returnKeyType="next"
            onSubmitEditing={() => emailRef.current?.focus()}
          />
          <TextField
            ref={emailRef}
            label="Correo"
            value={email}
            onChangeText={setEmail}
            error={errors.email}
            keyboardType="email-address"
            autoCapitalize="none"
            autoComplete="email"
            autoCorrect={false}
            textContentType="emailAddress"
            returnKeyType="next"
            onSubmitEditing={() => passwordRef.current?.focus()}
          />
```

```
          <TextField
            ref={passwordRef}
            label="Contraseña (mínimo 6 caracteres)"
            value={password}
            onChangeText={setPassword}
            error={errors.password}
            secureTextEntry
            autoComplete="new-password"
            textContentType="newPassword"
            returnKeyType="go"
            onSubmitEditing={submit}
          />

          {formError ? (
            <ThemedText accessibilityRole="alert" style={{ color: theme.danger }}>
              {formError}
            </ThemedText>
          ) : null}

          <Button title="Crear cuenta" onPress={submit} loading={loading} />
        </ScrollView>
      </KeyboardAvoidingView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  flex: { flex: 1 },
  form: { flexGrow: 1, gap: Spacing.three, paddingVertical: Spacing.four },
});
```

Paso 10.8 · Pruebas de la semana

Sesión: restauración sin red, tokens en llaves separadas, `clear()` notifica a los suscriptores, renovación proactiva, **una sola** renovación en vuelo, sesión conservada sin red y cierre de sesión ante `401`. Componentes: estados de pantalla, botones y campos, incluida la accesibilidad.

▸ **CREAR · library_app/__tests__/session-manager.test.ts**

```
import { ApiRequestError, NetworkError } from '@/services/api/client';
import { SessionManager, type SecureStorage } from '@/services/auth/session-manager';
import type { AuthResult } from '@/types';

class MemoryStorage implements SecureStorage {
  data = new Map<string, string>();
  async getItemAsync(k: string) {
    return this.data.get(k) ?? null;
  }
  async setItemAsync(k: string, v: string) {
    this.data.set(k, v);
  }
  async deleteItemAsync(k: string) {
    this.data.delete(k);
  }
}

const result = (idToken: string, expiresIn = 3600): AuthResult => ({
  tokens: { idToken, refreshToken: 'refresh-1', expiresIn },
  user: { uid: 'u1', email: 'a@b.co', name: 'Ana', role: 'student' },
});

describe('SessionManager', () => {
  it('guarda la sesión y la restaura tras reiniciar la app (sin red)', async () => {
    const storage = new MemoryStorage();
    await new SessionManager(storage, { post: jest.fn() }).save(result('tok-1'));

    const restarted = new SessionManager(storage, { post: jest.fn() });
    expect(await restarted.restore()).toMatchObject({ uid: 'u1', role: 'student' });
    expect(await restarted.getAccessToken()).toBe('tok-1');
  });

  it('guarda cada token en su propia llave (límite de tamaño de SecureStore en iOS)', async () => {
    const storage = new MemoryStorage();
    await new SessionManager(storage, { post: jest.fn() }).save(result('tok-1'));
    expect([...storage.data.keys()].sort()).toEqual([
      'auth.expiresAt',
```

```
      'auth.idToken',
      'auth.refreshToken',
      'auth.user',
    ]);
  });

  it('clear() borra todo y notifica a los suscriptores', async () => {
    const storage = new MemoryStorage();
    const session = new SessionManager(storage, { post: jest.fn() });
    const listener = jest.fn();
    session.subscribe(listener);
    await session.save(result('tok-1'));
    await session.clear();
    expect(storage.data.size).toBe(0);
    expect(listener).toHaveBeenLastCalledWith(null);
  });

  it('renueva proactivamente un token por vencer', async () => {
    let now = 1_000_000;
    const post = jest.fn().mockResolvedValue(result('tok-2'));
    const session = new SessionManager(new MemoryStorage(), { post }, () => now);
    await session.save(result('tok-1', 3600));

    now += 3600 * 1000 - 30_000; // faltan 30 s: dentro del margen de 60 s
    expect(await session.getAccessToken()).toBe('tok-2');
    expect(post).toHaveBeenCalledWith(
      '/api/auth/refresh',
      { refreshToken: 'refresh-1' },
      { auth: false },
    );
  });

  it('varias renovaciones simultáneas comparten una sola petición', async () => {
    const post = jest.fn().mockResolvedValue(result('tok-2'));
    const session = new SessionManager(new MemoryStorage(), { post });
    await session.save(result('tok-1'));
    await Promise.all([session.refresh(), session.refresh(), session.refresh()]);
    expect(post).toHaveBeenCalledTimes(1);
  });

  it('sin red NO cierra la sesión (la app sigue usable offline)', async () => {
    const post = jest.fn().mockRejectedValue(new NetworkError());
    const session = new SessionManager(new MemoryStorage(), { post });
    await session.save(result('tok-1'));
    expect(await session.refresh()).toBe(false);
    expect(session.getUser()).not.toBeNull();
  });

  it('si el refreshToken es rechazado (401) cierra la sesión', async () => {
    const post = jest.fn().mockRejectedValue(new ApiRequestError(401, 'INVALID_CREDENTIALS', 'x'));
    const session = new SessionManager(new MemoryStorage(), { post });
    await session.save(result('tok-1'));
    expect(await session.refresh()).toBe(false);
    expect(session.getUser()).toBeNull();
  });
});
```

▸ **CREAR · library_app/__tests__/components.test.tsx**

```
import { fireEvent, render, screen } from '@testing-library/react-native';

import { Button } from '@/components/button';
import { EmptyView, ErrorView, LoadingView } from '@/components/state-views';
import { TextField } from '@/components/text-field';

describe('estados de pantalla (semana 9)', () => {
  it('ErrorView muestra el mensaje y permite reintentar', async () => {
    const onRetry = jest.fn();
    await render(<ErrorView message="No hay conexión con el servidor." onRetry={onRetry} />);

    expect(screen.getByText('No hay conexión con el servidor.')).toBeTruthy();
    await fireEvent.press(screen.getByRole('button', { name: 'Reintentar' }));
    expect(onRetry).toHaveBeenCalledTimes(1);
  });

  it('ErrorView se anuncia como alerta para lectores de pantalla', async () => {
    await render(<ErrorView message="x" onRetry={() => {}} />);
    expect(screen.getByRole('alert')).toBeTruthy();
```

```
  });

  it('LoadingView expone un indicador de progreso accesible', async () => {
    await render(<LoadingView message="Cargando catálogo…" />);
    expect(screen.getByLabelText('Cargando catálogo…')).toBeTruthy();
  });

  it('EmptyView muestra título y mensaje', async () => {
    await render(<EmptyView title="Sin resultados" message="Prueba con otro título." />);
    expect(screen.getByText('Sin resultados')).toBeTruthy();
    expect(screen.getByText('Prueba con otro título.')).toBeTruthy();
  });
});

describe('Button', () => {
  it('ejecuta onPress', async () => {
    const onPress = jest.fn();
    await render(<Button title="Solicitar préstamo" onPress={onPress} />);
    await fireEvent.press(screen.getByRole('button', { name: 'Solicitar préstamo' }));
    expect(onPress).toHaveBeenCalledTimes(1);
  });

  it('no dispara onPress cuando está deshabilitado', async () => {
    const onPress = jest.fn();
    await render(<Button title="Sin ejemplares" disabled onPress={onPress} />);
    await fireEvent.press(screen.getByText('Sin ejemplares'));
    expect(onPress).not.toHaveBeenCalled();
  });

  it('en estado de carga se bloquea y no dispara onPress (evita doble envío)', async () => {
    const onPress = jest.fn();
    await render(<Button title="Iniciar sesión" loading onPress={onPress} />);
    await fireEvent.press(screen.getByRole('button'));
    expect(onPress).not.toHaveBeenCalled();
  });
});

describe('TextField', () => {
  it('asocia la etiqueta al campo y muestra el error', async () => {
    await render(
      <TextField
        label="Correo"
        error="Escribe un correo válido"
        value=""
        onChangeText={() => {}}
      />,
    );
    expect(screen.getByLabelText('Correo')).toBeTruthy();
    expect(screen.getByText('Escribe un correo válido')).toBeTruthy();
  });

  it('notifica los cambios de texto', async () => {
    const onChangeText = jest.fn();
    await render(<TextField label="Correo" value="" onChangeText={onChangeText} />);
    await fireEvent.changeText(screen.getByLabelText('Correo'), 'ana@correo.co');
    expect(onChangeText).toHaveBeenCalledWith('ana@correo.co');
  });
});
```

Nota. Punto de control de la semana 10. `npm run typecheck`, `npx expo lint` y `npm test` en verde.

4.3 Cómo probar en el teléfono

Escenario	Resultado esperado
Abrir la app sin sesión	Pantalla de inicio de sesión
Registrar un estudiante nuevo	Entra al catálogo
Registrar un correo ya usado	Mensaje Ya existe una cuenta con ese correo
Contraseña de 5 caracteres en registro	Error de validación en el campo, sin llamar a la API
Iniciar sesión, cerrar la app por completo y abrirla	Sigue con la sesión iniciada (también sin internet)
Tocar Salir	Vuelve al login
Credenciales erróneas	Mensaje Correo o contraseña incorrectos
Con la API apagada, intentar iniciar sesión	No hay conexión con el servidor…

4.4 Evidencia y criterios

Evidencia para el docente. Flujo de registro, inicio y cierre de sesión funcionando; tokens en `expo-secure-store`; rutas protegidas; tabla de modelado de amenazas completada para tu proyecto; pruebas de sesión en verde; bitácora de IA con al menos una sugerencia **contrastada** contra estos requisitos de seguridad.

- ☐ Ningún token ni contraseña aparece en la consola ni en SQLite
- ☐ La sesión sobrevive al reinicio de la app
- ☐ Un `401` provoca una renovación y un reintento, no una pantalla de error
- ☐ No queda ninguna credencial en el código ni en el `.env` de la app

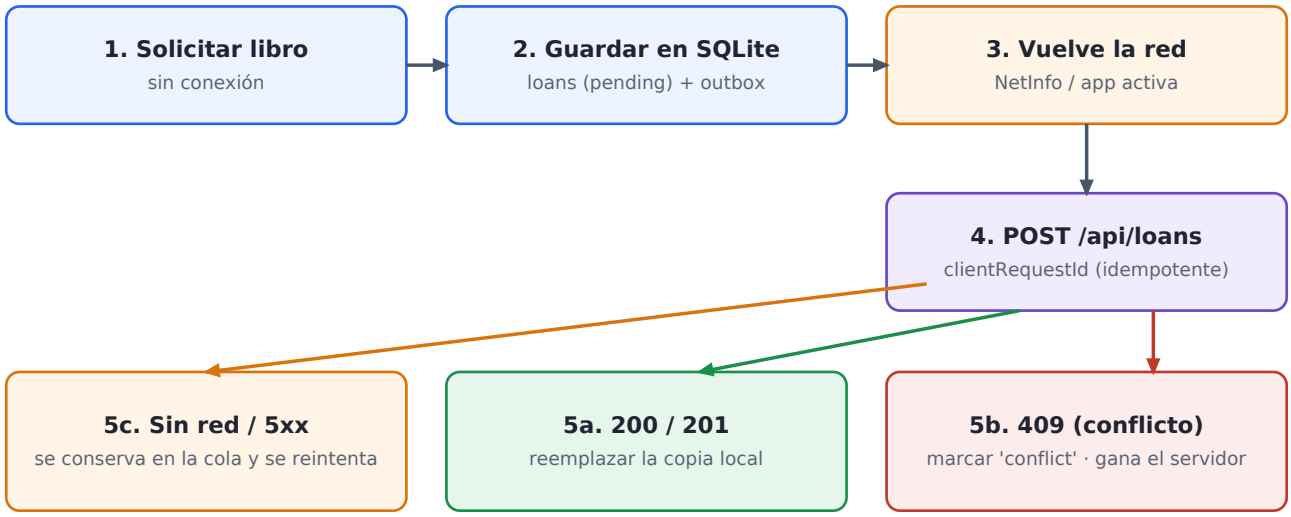
5. Semana 11 · SQLite, sincronización y trabajo sin conexión

Del microcurrículo: Sincronización local-remota: caché, cola de cambios, detección de conectividad, conflictos y consistencia eventual. Estrategias offline-first básicas. Presencial: estudio de casos y laboratorio de sincronización. Independiente: implementar lectura local, actualización remota y recuperación; diseñar y probar escenarios de reconexión y conflicto.

5.1 Conceptos clave

- **Offline-first.** La interfaz lee y escribe en la base de datos local; la red es un detalle que se resuelve en segundo plano.
- **Caché.** Copia local del catálogo para mostrarlo al instante y sin conexión.
- **Cola de cambios (outbox).** Tabla donde se anotan los cambios hechos sin conexión, en orden, hasta enviarlos.
- **Sincronización incremental.** En vez de descargar todo, se envía al servidor un cursor (`updatedSince`) y este devuelve solo lo que cambió.
- **Idempotencia.** Enviar la **misma** solicitud dos veces no debe crear dos préstamos. Cada solicitud lleva un `clientRequestId` (UUID) generado en el teléfono; si el servidor ya lo vio, devuelve la existente. Hace seguro reintentar tras perder la respuesta.
- **Consistencia eventual.** Durante un rato el teléfono y el servidor pueden diferir; al sincronizar convergen.
- **Resolución de conflictos.** Aquí aplicamos "gana el servidor": es simple, predecible y suficiente, porque el servidor es quien conoce los ejemplares reales.

El ciclo completo



Después se descargan los cambios del servidor: libros desde el cursor updatedSince y los préstamos.

Modelo de datos local

Tabla	Para qué sirve	Detalle clave
`books`	Caché del catálogo	`search_text` (título + autor sin tildes) para buscar sin importar acentos
`loans`	Préstamos, incluidos los aún no enviados	`sync_state`: `synced`, `pending` o `conflict`; `client_request_id` único
`outbox`	Cola de cambios sin enviar	`attempts` y `last_error` para reintentos
`sync_meta`	Clave/valor	Cursor de libros, fecha de última sincronización, dueño de los datos
`activity_log`	Bitácora de eventos	Trazabilidad (semana 12); solo los últimos 200

5.2 Paso a paso

Paso 11.1 · Utilidades de texto

`normalizeSearch` quita tildes y mayúsculas; `escapeLike` hace que un `%` escrito por el usuario se busque como texto y no como comodín de SQL.

▸ **CREAR · library_app/src/lib/text.ts**

```
/** Minúsculas y sin tildes: "Cólera" → "colera". Sirve para búsquedas que ignoran acentos. */
export function normalizeSearch(value: string): string {
  return value
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .toLowerCase()
    .trim();
}

/** Escapa % y _ para usarlos de forma literal dentro de un LIKE ... ESCAPE '\'. */
export function escapeLike(value: string): string {
  return value.replace(/[\\%_]/g, (c) => `\\${c}`);
}

export function formatDate(iso: string | null): string {
  if (!iso) return '—';
  return new Date(iso).toLocaleDateString('es-CO', {
    day: '2-digit',
    month: 'short',
    year: 'numeric',
  });
}

export function formatDateTime(iso: string | null): string {
  if (!iso) return 'nunca';
  return new Date(iso).toLocaleString('es-CO', {
    day: '2-digit',
    month: 'short',
    hour: '2-digit',
    minute: '2-digit',
  });
}
```

Paso 11.2 · Migraciones con `PRAGMA user_version`

SQLite guarda un número de versión dentro del archivo. Al abrir la app lo comparamos con `DATABASE_VERSION` y aplicamos, en orden, **solo lo que falta**. Así un teléfono con la versión 1 de la app se actualiza sin perder datos al instalar la versión 2.

▸ **CREAR · library_app/src/db/migrations.ts**

```
import type { SQLiteDatabase } from 'expo-sqlite';

/**
 * Migraciones básicas con PRAGMA user_version.
 *
 * SQLite guarda un entero (user_version) dentro del archivo de la base de datos.
 * Al abrir la app comparamos ese número con DATABASE_VERSION y aplicamos, en orden,
 * solo las migraciones que falten. Cada migración nueva se agrega como un `if` más y
 * se sube DATABASE_VERSION; NUNCA se edita una migración que ya salió en una versión publicada.
 */
export const DATABASE_VERSION = 2;

export async function migrateDbIfNeeded(db: SQLiteDatabase): Promise<void> {
  const row = await db.getFirstAsync<{ user_version: number }>('PRAGMA user_version');
  let version = row?.user_version ?? 0;
  if (version >= DATABASE_VERSION) return;

  // WAL permite leer mientras se escribe (la UI consulta mientras la sincronización inserta).
  await db.execAsync('PRAGMA journal_mode = WAL;');
  await db.execAsync('PRAGMA foreign_keys = ON;');

  if (version < 1) {
    await db.withTransactionAsync(async () => {
      await db.execAsync(`
        CREATE TABLE books (
          id               TEXT PRIMARY KEY NOT NULL,
          title            TEXT NOT NULL,
          author           TEXT NOT NULL,
          category         TEXT NOT NULL,
          year             INTEGER NOT NULL,
          description      TEXT NOT NULL,
          total_copies     INTEGER NOT NULL,
          available_copies INTEGER NOT NULL,
          cover_url        TEXT,
          updated_at       TEXT NOT NULL,
          search_text      TEXT NOT NULL  -- título + autor sin tildes ni mayúsculas, para buscar
        );

        CREATE TABLE loans (
          id                TEXT PRIMARY KEY NOT NULL,
          book_id           TEXT NOT NULL,
          book_title        TEXT NOT NULL,
          user_id           TEXT NOT NULL,
          user_email        TEXT NOT NULL,
          user_name         TEXT NOT NULL,
          status            TEXT NOT NULL CHECK (status IN ('requested','approved','rejected','returned')),
          client_request_id TEXT NOT NULL UNIQUE,
          requested_at      TEXT NOT NULL,
          resolved_at       TEXT,
          due_date          TEXT,
          returned_at       TEXT,
          updated_at        TEXT NOT NULL,
          sync_state        TEXT NOT NULL DEFAULT 'synced' CHECK (sync_state IN ('synced','pending','conflict')),
          sync_error        TEXT
        );

        -- Cola de cambios hechos sin conexión, en orden de creación.
        CREATE TABLE outbox (
          id          INTEGER PRIMARY KEY AUTOINCREMENT,
          kind        TEXT NOT NULL,
          payload     TEXT NOT NULL,     -- JSON
          attempts    INTEGER NOT NULL DEFAULT 0,
          last_error  TEXT,
          created_at  TEXT NOT NULL
        );

        -- Clave/valor: cursor de sincronización, etc.
        CREATE TABLE sync_meta (
          key   TEXT PRIMARY KEY NOT NULL,
          value TEXT NOT NULL
        );
      `);
    });
    version = 1;
  }
```

```
  if (version < 2) {
    await db.withTransactionAsync(async () => {
      await db.execAsync(`
        CREATE INDEX idx_books_category ON books (category);
        CREATE INDEX idx_loans_user ON loans (user_id, requested_at DESC);
        CREATE INDEX idx_loans_book ON loans (book_id);

        -- Trazabilidad (semana 12): qué hizo la app con servicios del dispositivo y de sincronización.
        CREATE TABLE activity_log (
          id         INTEGER PRIMARY KEY AUTOINCREMENT,
          kind       TEXT NOT NULL,
          detail     TEXT NOT NULL,
          created_at TEXT NOT NULL
        );
      `);
    });
    version = 2;
  }

  await db.execAsync(`PRAGMA user_version = ${DATABASE_VERSION}`);
}
```

> **Ojo. Nunca edites una migración que ya se publicó.** Agrega una nueva (`if (version < 3)`) y sube `DATABASE_VERSION`. Cada migración corre dentro de una transacción: o se aplica completa o no se aplica.

Paso 11.3 · Repositorios: toda la SQL en un solo lugar

Las pantallas nunca arman consultas. Los repositorios exponen operaciones con nombre de negocio y **todas las consultas son parametrizadas** (`?`): nunca se concatena texto del usuario. Puntos para leer con calma:

- `upsertMany` usa una **transacción** (entran todos los libros o ninguno).
- `upsertFromServer` reemplaza la copia local (mismo `client_request_id`) por la del servidor: así un préstamo creado sin conexión **nunca queda duplicado**.
- La cola `outbox` es FIFO con conteo de intentos.

▸ **CREAR · library_app/src/db/repositories.ts**

```
/**
 * Repositorios: toda la SQL de la app vive aquí. Las pantallas y servicios nunca arman consultas.
 * Reglas: consultas SIEMPRE parametrizadas (?), nunca concatenar datos del usuario en el SQL.
 */
import type { SQLiteDatabase } from 'expo-sqlite';

import { escapeLike, normalizeSearch } from '@/lib/text';
import type { Book, LocalLoan, Loan, LoanStatus, SyncState } from '@/types';

// ─────────────────────────────── Libros ───────────────────────────────

interface BookRow {
  id: string;
  title: string;
  author: string;
  category: string;
  year: number;
  description: string;
  total_copies: number;
  available_copies: number;
  cover_url: string | null;
  updated_at: string;
}

const toBook = (r: BookRow): Book => ({
  id: r.id,
  title: r.title,
  author: r.author,
  category: r.category,
  year: r.year,
  description: r.description,
```

```
  totalCopies: r.total_copies,
  availableCopies: r.available_copies,
  coverUrl: r.cover_url,
  updatedAt: r.updated_at,
});

export interface BookFilters {
  q?: string;
  category?: string | null;
}

export function createBooksRepo(db: SQLiteDatabase) {
  return {
    /** Inserta o actualiza (upsert) en una sola transacción: o entran todos o ninguno. */
    async upsertMany(books: Book[]): Promise<void> {
      if (books.length === 0) return;
      await db.withTransactionAsync(async () => {
        for (const b of books) {
          await db.runAsync(
            `INSERT INTO books (id, title, author, category, year, description, total_copies,
                                available_copies, cover_url, updated_at, search_text)
             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
             ON CONFLICT(id) DO UPDATE SET
               title = excluded.title, author = excluded.author, category = excluded.category,
               year = excluded.year, description = excluded.description,
               total_copies = excluded.total_copies, available_copies = excluded.available_copies,
               cover_url = excluded.cover_url, updated_at = excluded.updated_at,
               search_text = excluded.search_text`,
            b.id,
            b.title,
            b.author,
            b.category,
            b.year,
            b.description,
            b.totalCopies,
            b.availableCopies,
            b.coverUrl,
            b.updatedAt,
            normalizeSearch(`${b.title} ${b.author}`),
          );
        }
      });
    },

    async list({ q, category }: BookFilters = {}): Promise<Book[]> {
      const term = q ? normalizeSearch(q) : '';
      const rows = await db.getAllAsync<BookRow>(
        `SELECT * FROM books
          WHERE (? = '' OR search_text LIKE ? ESCAPE '\\')
            AND (? IS NULL OR category = ?)
          ORDER BY title COLLATE NOCASE`,
        term,
        `%${escapeLike(term)}%`,
        category ?? null,
        category ?? null,
      );
      return rows.map(toBook);
    },

    async getById(id: string): Promise<Book | null> {
      const row = await db.getFirstAsync<BookRow>('SELECT * FROM books WHERE id = ?', id);
      return row ? toBook(row) : null;
    },

    async categories(): Promise<string[]> {
      const rows = await db.getAllAsync<{ category: string }>(
        'SELECT DISTINCT category FROM books ORDER BY category COLLATE NOCASE',
      );
      return rows.map((r) => r.category);
    },

    async clear(): Promise<void> {
      await db.runAsync('DELETE FROM books');
    },
  };
}

// ──────────────────────────── Préstamos ────────────────────────────
```

```
interface LoanRow {
  id: string;
  book_id: string;
  book_title: string;
  user_id: string;
  user_email: string;
  user_name: string;
  status: LoanStatus;
  client_request_id: string;
  requested_at: string;
  resolved_at: string | null;
  due_date: string | null;
  returned_at: string | null;
  updated_at: string;
  sync_state: SyncState;
  sync_error: string | null;
}

const toLoan = (r: LoanRow): LocalLoan => ({
  id: r.id,
  bookId: r.book_id,
  bookTitle: r.book_title,
  userId: r.user_id,
  userEmail: r.user_email,
  userName: r.user_name,
  status: r.status,
  clientRequestId: r.client_request_id,
  requestedAt: r.requested_at,
  resolvedAt: r.resolved_at,
  dueDate: r.due_date,
  returnedAt: r.returned_at,
  updatedAt: r.updated_at,
  syncState: r.sync_state,
  syncError: r.sync_error,
});

export function createLoansRepo(db: SQLiteDatabase) {
  async function insert(loan: Loan, syncState: SyncState, syncError: string | null) {
    await db.runAsync(
      `INSERT OR REPLACE INTO loans (id, book_id, book_title, user_id, user_email, user_name, status,
         client_request_id, requested_at, resolved_at, due_date, returned_at, updated_at, sync_state, sync_error)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
      loan.id,
      loan.bookId,
      loan.bookTitle,
      loan.userId,
      loan.userEmail,
      loan.userName,
      loan.status,
      loan.clientRequestId,
      loan.requestedAt,
      loan.resolvedAt,
      loan.dueDate,
      loan.returnedAt,
      loan.updatedAt,
      syncState,
      syncError,
    );
  }

  return {
    /** Préstamos del estudiante, o todos si userId es null (bibliotecario). */
    async list(userId: string | null): Promise<LocalLoan[]> {
      const rows = userId
        ? await db.getAllAsync<LoanRow>(
            'SELECT * FROM loans WHERE user_id = ? ORDER BY requested_at DESC',
            userId,
          )
        : await db.getAllAsync<LoanRow>('SELECT * FROM loans ORDER BY requested_at DESC');
      return rows.map(toLoan);
    },

    /** Solicitud creada sin confirmación del servidor (sync_state = 'pending'). */
    async insertPending(loan: Loan): Promise<void> {
      await insert(loan, 'pending', null);
    },
```

```
    /**
     * Guarda la versión del servidor. Si existía una copia local creada offline con el mismo
     * clientRequestId (id "local-..."), se reemplaza: así nunca queda duplicada.
     */
    async upsertFromServer(loan: Loan): Promise<void> {
      await db.withTransactionAsync(async () => {
        await db.runAsync(
          'DELETE FROM loans WHERE client_request_id = ? AND id <> ?',
          loan.clientRequestId,
          loan.id,
        );
        await insert(loan, 'synced', null);
      });
    },

    async upsertManyFromServer(loans: Loan[]): Promise<void> {
      if (loans.length === 0) return;
      await db.withTransactionAsync(async () => {
        for (const loan of loans) {
          await db.runAsync(
            'DELETE FROM loans WHERE client_request_id = ? AND id <> ?',
            loan.clientRequestId,
            loan.id,
          );
          await insert(loan, 'synced', null);
        }
      });
    },

    /** El servidor rechazó la solicitud al sincronizar: "gana el servidor", se informa al usuario. */
    async markConflict(clientRequestId: string, message: string): Promise<void> {
      await db.runAsync(
        `UPDATE loans SET sync_state = 'conflict', sync_error = ? WHERE client_request_id = ?`,
        message,
        clientRequestId,
      );
    },

    async remove(id: string): Promise<void> {
      await db.runAsync('DELETE FROM loans WHERE id = ?', id);
    },

    /** ¿Tiene el usuario una solicitud/préstamo vigente (incluida una aún sin enviar) de este libro? */
    async hasActiveForBook(userId: string, bookId: string): Promise<boolean> {
      const row = await db.getFirstAsync<{ n: number }>(
        `SELECT COUNT(*) AS n FROM loans
          WHERE user_id = ? AND book_id = ? AND status IN ('requested','approved') AND sync_state <> 'conflict'`,
        userId,
        bookId,
      );
      return (row?.n ?? 0) > 0;
    },

    async latestForBook(userId: string, bookId: string): Promise<LocalLoan | null> {
      const row = await db.getFirstAsync<LoanRow>(
        'SELECT * FROM loans WHERE user_id = ? AND book_id = ? ORDER BY requested_at DESC LIMIT 1',
        userId,
        bookId,
      );
      return row ? toLoan(row) : null;
    },

    async clear(): Promise<void> {
      await db.runAsync('DELETE FROM loans');
    },
  };
}

// ─────────────────────────────── Cola (outbox) ────────────────────────────────

export interface OutboxItem {
  id: number;
  kind: 'request_loan';
  payload: { bookId: string; clientRequestId: string };
  attempts: number;
  lastError: string | null;
  createdAt: string;
}
```

```
interface OutboxRow {
  id: number;
  kind: 'request_loan';
  payload: string;
  attempts: number;
  last_error: string | null;
  created_at: string;
}

export function createOutboxRepo(db: SQLiteDatabase) {
  return {
    async enqueue(kind: OutboxItem['kind'], payload: OutboxItem['payload']): Promise<void> {
      await db.runAsync(
        'INSERT INTO outbox (kind, payload, created_at) VALUES (?, ?, ?)',
        kind,
        JSON.stringify(payload),
        new Date().toISOString(),
      );
    },

    /** En orden de creación (FIFO). */
    async pending(): Promise<OutboxItem[]> {
      const rows = await db.getAllAsync<OutboxRow>('SELECT * FROM outbox ORDER BY id');
      return rows.map((r) => ({
        id: r.id,
        kind: r.kind,
        payload: JSON.parse(r.payload) as OutboxItem['payload'],
        attempts: r.attempts,
        lastError: r.last_error,
        createdAt: r.created_at,
      }));
    },

    async count(): Promise<number> {
      const row = await db.getFirstAsync<{ n: number }>('SELECT COUNT(*) AS n FROM outbox');
      return row?.n ?? 0;
    },

    async remove(id: number): Promise<void> {
      await db.runAsync('DELETE FROM outbox WHERE id = ?', id);
    },

    async recordFailure(id: number, message: string): Promise<void> {
      await db.runAsync(
        'UPDATE outbox SET attempts = attempts + 1, last_error = ? WHERE id = ?',
        message,
        id,
      );
    },

    async clear(): Promise<void> {
      await db.runAsync('DELETE FROM outbox');
    },
  };
}

// ─────────────────────────────── Metadatos y bitácora ──────────────────────────────

export function createMetaRepo(db: SQLiteDatabase) {
  return {
    async get(key: string): Promise<string | null> {
      const row = await db.getFirstAsync<{ value: string }>(
        'SELECT value FROM sync_meta WHERE key = ?',
        key,
      );
      return row?.value ?? null;
    },
    async set(key: string, value: string): Promise<void> {
      await db.runAsync(
        'INSERT INTO sync_meta (key, value) VALUES (?, ?) ON CONFLICT(key) DO UPDATE SET value = excluded.value',
        key,
        value,
      );
    },
    async clear(): Promise<void> {
      await db.runAsync('DELETE FROM sync_meta');
    },
```

```
  };
}

export interface ActivityEntry {
  id: number;
  kind: string;
  detail: string;
  createdAt: string;
}

export function createActivityRepo(db: SQLiteDatabase) {
  return {
    /** Nunca registrar tokens, contraseñas ni datos personales en la bitácora. */
    async log(kind: string, detail: string): Promise<void> {
      await db.runAsync(
        'INSERT INTO activity_log (kind, detail, created_at) VALUES (?, ?, ?)',
        kind,
        detail,
        new Date().toISOString(),
      );
      // Se conservan solo los últimos 200 eventos.
      await db.runAsync(
        'DELETE FROM activity_log WHERE id NOT IN (SELECT id FROM activity_log ORDER BY id DESC LIMIT 200)',
      );
    },
    async recent(limit = 20): Promise<ActivityEntry[]> {
      const rows = await db.getAllAsync<{
        id: number;
        kind: string;
        detail: string;
        created_at: string;
      }>('SELECT * FROM activity_log ORDER BY id DESC LIMIT ?', limit);
      return rows.map((r) => ({
        id: r.id,
        kind: r.kind,
        detail: r.detail,
        createdAt: r.created_at,
      }));
    },
  };
}

export function createRepositories(db: SQLiteDatabase) {
  return {
    books: createBooksRepo(db),
    loans: createLoansRepo(db),
    outbox: createOutboxRepo(db),
    meta: createMetaRepo(db),
    activity: createActivityRepo(db),
  };
}

export type Repositories = ReturnType<typeof createRepositories>;
```

Paso 11.4 · Solicitar un préstamo, con o sin internet

El servicio aplica primero las reglas que ya puede comprobar **en el teléfono** y luego guarda; no espera a la red. Fíjate en el orden: primero la cola y después el préstamo local. Si la app se cierra entre ambos pasos, la sincronización igual envía la solicitud y trae el préstamo desde el servidor; nunca queda un préstamo "fantasma" sin cola.

▸ **CREAR · library_app/src/services/loans/loan-service.ts**

```
import type { Repositories } from '@/db/repositories';
import type { ApiClient } from '@/services/api/client';
import type { Book, Loan, LoanAction, LocalLoan, User } from '@/types';

/** Regla de negocio detectada en el dispositivo, antes de tocar la red. */
export class LoanRuleError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'LoanRuleError';
  }
}

interface Deps {
```

```
  api: Pick<ApiClient, 'patch'>;
  repos: Pick<Repositories, 'loans' | 'outbox' | 'activity'>;
  newId: () => string;
  now?: () => Date;
}

export function createLoanService({ api, repos, newId, now = () => new Date() }: Deps) {
  return {
    /**
     * Estudiante: solicita un libro. Funciona con o sin conexión:
     * guarda la solicitud en SQLite + cola y devuelve de inmediato; el envío lo hace la sincronización.
     */
    async requestLoan(user: User, book: Book): Promise<LocalLoan> {
      if (await repos.loans.hasActiveForBook(user.uid, book.id)) {
        throw new LoanRuleError('Ya tienes una solicitud o un préstamo activo de este libro.');
      }
      if (book.availableCopies <= 0) {
        throw new LoanRuleError('No hay ejemplares disponibles de este libro.');
      }

      const clientRequestId = newId();
      const timestamp = now().toISOString();
      const loan: Loan = {
        id: `local-${clientRequestId}`,
        bookId: book.id,
        bookTitle: book.title,
        userId: user.uid,
        userEmail: user.email,
        userName: user.name,
        status: 'requested',
        clientRequestId,
        requestedAt: timestamp,
        resolvedAt: null,
        dueDate: null,
        returnedAt: null,
        updatedAt: timestamp,
      };

      // Primero la cola y después el préstamo local: si la app se cierra entre ambos pasos,
      // la sincronización igual envía la solicitud y trae el préstamo desde el servidor.
      await repos.outbox.enqueue('request_loan', { bookId: book.id, clientRequestId });
      await repos.loans.insertPending(loan);
      await repos.activity.log('loan.request', `Solicitud local de "${book.title}"`);
      return { ...loan, syncState: 'pending', syncError: null };
    },

    /** Bibliotecario: aprobar, rechazar o registrar devolución. Requiere conexión (decisión de diseño). */
    async transition(loanId: string, action: LoanAction): Promise<Loan> {
      const { loan } = await api.patch<{ loan: Loan }>(`/api/loans/${loanId}`, { action });
      await repos.loans.upsertFromServer(loan);
      await repos.activity.log(
        'loan.' + action,
        `Préstamo de "${loan.bookTitle}" → ${loan.status}`,
      );
      return loan;
    },

    /** Descarta un préstamo local que quedó en conflicto. */
    dismiss(loanId: string): Promise<void> {
      return repos.loans.remove(loanId);
    },
  };
}

export type LoanService = ReturnType<typeof createLoanService>;
```

Paso 11.5 · El servicio de sincronización

Es el archivo central de la semana. Primero **envía** la cola; después **descarga** libros (incremental) y préstamos. Qué hace ante cada respuesta al enviar una solicitud:

Respuesta	Qué hace la app
200 / 201	Reemplaza la copia local por la del servidor y quita el elemento de la cola

Respuesta	Qué hace la app
`409` (`NO_COPIES`, `ALREADY_ACTIVE`) o `404`	**Conflicto**: marca el préstamo `conflict` con un mensaje claro y lo quita de la cola. No bloquea las demás solicitudes
Sin conexión	Conserva **toda** la cola y propaga el error
`5xx` o `429`	Suma un intento y reintenta en la próxima sincronización; tras 5 intentos lo marca como no procesable
`401`	No es un conflicto del préstamo: se conserva la cola y se renueva la sesión

El cursor de libros se guarda con **1 segundo de solape**: pedir un poco "hacia atrás" evita perder un cambio ocurrido justo en el borde, y como el upsert es idempotente, repetir un libro no causa daño.

▸ **CREAR · library_app/src/services/sync/sync-service.ts**

```
/**
 * Sincronización local ⇄ remota (semana 11). Estrategia offline-first:
 *
 *  1. La UI SIEMPRE lee de SQLite; nunca espera a la red.
 *  2. Las solicitudes de préstamo se guardan primero en local (sync_state = 'pending') y en la cola
 *     `outbox`. Este servicio las envía cuando hay conexión.
 *  3. Después se traen los cambios del servidor: libros desde el último cursor (incremental)
 *     y los préstamos del usuario.
 *
 * Consistencia eventual y conflictos: el servidor es la fuente de verdad ("gana el servidor").
 * Si una solicitud hecha offline ya no es válida al llegar (otro estudiante tomó el último
 * ejemplar, o ya existía una solicitud activa) la API responde 409 y el préstamo local queda
 * marcado como 'conflict' con una explicación para el usuario.
 */
import type { Repositories } from '@/db/repositories';
import { ApiRequestError, NetworkError, type ApiClient } from '@/services/api/client';
import type { Book, Loan } from '@/types';

/** Tras este número de fallos 5xx se abandona el envío y se informa al usuario. */
export const MAX_ATTEMPTS = 5;

/** Se pide al servidor un poco "hacia atrás" del cursor: el upsert es idempotente y evita perder cambios en el borde. */
const CURSOR_OVERLAP_MS = 1000;

const KEYS = { booksCursor: 'books.cursor', lastSyncAt: 'sync.lastSyncAt' } as const;

export interface SyncResult {
  pushed: number;
  conflicts: number;
  booksPulled: number;
  loansPulled: number;
}

export interface SyncDeps {
  api: Pick<ApiClient, 'get' | 'post'>;
  repos: Pick<Repositories, 'books' | 'loans' | 'outbox' | 'meta' | 'activity'>;
  now?: () => Date;
}

/** Explica al usuario, en lenguaje claro, por qué el servidor rechazó una solicitud. */
export function conflictMessage(err: ApiRequestError): string {
  switch (err.code) {
    case 'ALREADY_ACTIVE':
      return 'Ya tenías una solicitud o un préstamo activo de este libro.';
    case 'NO_COPIES':
      return 'Cuando se envió tu solicitud ya no quedaban ejemplares disponibles.';
    case 'NOT_FOUND':
      return 'El libro ya no existe en el catálogo.';
    case 'FORBIDDEN':
      return 'Tu cuenta no tiene permiso para solicitar libros.';
    default:
      return err.message;
  }
}
```

```
export function createSyncService({ api, repos, now = () => new Date() }: SyncDeps) {
  const { books, loans, outbox, meta, activity } = repos;

  /** Envía la cola en orden. Devuelve cuántas salieron y cuántas terminaron en conflicto. */
  async function pushOutbox(): Promise<{ pushed: number; conflicts: number }> {
    let pushed = 0;
    let conflicts = 0;

    for (const item of await outbox.pending()) {
      try {
        // Idempotente: la API usa clientRequestId, así que un reintento tras perder la respuesta no duplica.
        const { loan } = await api.post<{ loan: Loan }>('/api/loans', item.payload);
        await loans.upsertFromServer(loan);
        await outbox.remove(item.id);
        pushed++;
        await activity.log('sync.push', `Solicitud de "${loan.bookTitle}" enviada`);
      } catch (err) {
        if (err instanceof NetworkError) throw err; // sigue sin conexión: se conserva toda la cola

        if (err instanceof ApiRequestError) {
          if (err.status === 401) throw err; // sesión inválida: no es un conflicto del préstamo

          const retryable = err.status >= 500 || err.status === 429;
          if (retryable && item.attempts + 1 < MAX_ATTEMPTS) {
            await outbox.recordFailure(item.id, err.message);
            throw err; // se reintenta en la próxima sincronización
          }

          const message = retryable
            ? 'El servidor no pudo procesar la solicitud. Inténtalo de nuevo.'
            : conflictMessage(err);
          await loans.markConflict(item.payload.clientRequestId, message);
          await outbox.remove(item.id);
          conflicts++;
          await activity.log('sync.conflict', `${err.code}: ${message}`);
          continue;
        }
        throw err;
      }
    }
    return { pushed, conflicts };
  }

  async function pullBooks(): Promise<number> {
    const cursor = await meta.get(KEYS.booksCursor);
    const query = cursor ? `?updatedSince=${encodeURIComponent(cursor)}` : '';
    const res = await api.get<{ books: Book[]; serverTime: string }>(`/api/books${query}`);

    await books.upsertMany(res.books);
    const next = new Date(new Date(res.serverTime).getTime() - CURSOR_OVERLAP_MS).toISOString();
    await meta.set(KEYS.booksCursor, next);
    return res.books.length;
  }

  async function pullLoans(): Promise<number> {
    const res = await api.get<{ loans: Loan[] }>('/api/loans');
    await loans.upsertManyFromServer(res.loans);
    return res.loans.length;
  }

  return {
    /** Ciclo completo. Lanza NetworkError/ApiRequestError si no pudo terminar (lo ya hecho se conserva). */
    async run(): Promise<SyncResult> {
      const { pushed, conflicts } = await pushOutbox();
      const booksPulled = await pullBooks();
      const loansPulled = await pullLoans();
      await meta.set(KEYS.lastSyncAt, now().toISOString());
      return { pushed, conflicts, booksPulled, loansPulled };
    },

    async lastSyncAt(): Promise<string | null> {
      return meta.get(KEYS.lastSyncAt);
    },
  };
}

export type SyncService = ReturnType<typeof createSyncService>;
```

Paso 11.6 · Cuándo sincronizar: conectividad y ciclo de vida

`DataProvider` decide los momentos de sincronización: al iniciar sesión, al volver la red (NetInfo), al volver la app a primer plano, al deslizar para refrescar y justo después de solicitar un préstamo. Garantiza **un solo ciclo a la vez**. Además, los datos privados pertenecen a **una cuenta**: si cambió el usuario, se descartan los préstamos y la cola anteriores para no enviar solicitudes a nombre de otra persona; y al cerrar sesión se borran los datos privados del dispositivo (el catálogo público se conserva).

▸ **CREAR · library_app/src/providers/data-provider.tsx**

```
import NetInfo from '@react-native-community/netinfo';
import { randomUUID } from 'expo-crypto';
import { useSQLiteContext } from 'expo-sqlite';
import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useRef,
  useState,
  type ReactNode,
} from 'react';
import { AppState } from 'react-native';

import { createRepositories, type Repositories } from '@/db/repositories';
import { api } from '@/services/api';
import { describeError } from '@/services/api/client';
import { createLoanService } from '@/services/loans/loan-service';
import { createSyncService } from '@/services/sync/sync-service';
import type { Book, LoanAction, LocalLoan } from '@/types';

import { useAuth } from './auth-provider';

interface DataContextValue {
  repos: Repositories;
  /** Hay red y el dispositivo puede salir a internet. */
  online: boolean;
  syncing: boolean;
  lastSyncAt: string | null;
  /** Mensaje del último intento fallido de sincronización, o null. */
  lastError: string | null;
  /** Cuántos cambios hechos sin conexión esperan ser enviados. */
  pendingCount: number;
  /** Se incrementa cada vez que cambia SQLite; los hooks de lectura lo usan para volver a consultar. */
  dataVersion: number;
  syncNow: () => Promise<void>;
  requestLoan: (book: Book) => Promise<LocalLoan>;
  transitionLoan: (loanId: string, action: LoanAction) => Promise<void>;
  dismissLoan: (loanId: string) => Promise<void>;
  notifyChanged: () => void;
}

const DataContext = createContext<DataContextValue | null>(null);

export function DataProvider({ children }: { children: ReactNode }) {
  const db = useSQLiteContext();
  const { status, user } = useAuth();
  const repos = useMemo(() => createRepositories(db), [db]);
  const syncService = useMemo(() => createSyncService({ api, repos }), [repos]);
  const loanService = useMemo(() => createLoanService({ api, repos, newId: randomUUID }), [repos]);

  const [online, setOnline] = useState(true);
  const [syncing, setSyncing] = useState(false);
  const [lastSyncAt, setLastSyncAt] = useState<string | null>(null);
  const [lastError, setLastError] = useState<string | null>(null);
  const [pendingCount, setPendingCount] = useState(0);
  const [dataVersion, setDataVersion] = useState(0);

  const inFlight = useRef<Promise<void> | null>(null);
  const wasOnline = useRef(true);
  const uid = user?.uid ?? null;
```

```
  const refreshMeta = useCallback(async () => {
    setPendingCount(await repos.outbox.count());
    setLastSyncAt(await syncService.lastSyncAt());
  }, [repos, syncService]);

  const notifyChanged = useCallback(() => {
    setDataVersion((v) => v + 1);
    void refreshMeta();
  }, [refreshMeta]);

  /** Un solo ciclo de sincronización a la vez: llamadas simultáneas comparten la misma promesa. */
  const syncNow = useCallback(async () => {
    if (!uid) return;
    if (inFlight.current) return inFlight.current;

    const run = (async () => {
      setSyncing(true);
      try {
        const result = await syncService.run();
        setLastError(null);
        if (result.pushed + result.conflicts > 0) {
          await repos.activity.log(
            'sync',
            `Enviadas ${result.pushed}, en conflicto ${result.conflicts}`,
          );
        }
      } catch (err) {
        setLastError(describeError(err));
      } finally {
        setSyncing(false);
        inFlight.current = null;
        notifyChanged();
      }
    })();
    inFlight.current = run;
    return run;
  }, [uid, syncService, repos, notifyChanged]);

  // Conectividad: al volver la red se vacía la cola automáticamente.
  useEffect(() => {
    const unsubscribe = NetInfo.addEventListener((state) => {
      const isOnline = !!state.isConnected && state.isInternetReachable !== false;
      setOnline(isOnline);
      if (isOnline && !wasOnline.current) void syncNow();
      wasOnline.current = isOnline;
    });
    return unsubscribe;
  }, [syncNow]);

  // También al volver a primer plano.
  useEffect(() => {
    const sub = AppState.addEventListener('change', (next) => {
      if (next === 'active') void syncNow();
    });
    return () => sub.remove();
  }, [syncNow]);

  // Al iniciar sesión: proteger los datos de otra cuenta y sincronizar.
  useEffect(() => {
    if (status === 'loading') return;
    let cancelled = false;

    (async () => {
      if (uid) {
        // Los préstamos y la cola pertenecen a UNA cuenta. Si cambió el usuario (p. ej. se cerró la
        // sesión a la fuerza con cambios pendientes), se descartan para no enviar solicitudes a nombre de otro.
        const owner = await repos.meta.get('owner.uid');
        if (owner && owner !== uid) {
          await repos.loans.clear();
          await repos.outbox.clear();
        }
        await repos.meta.set('owner.uid', uid);
        if (cancelled) return;
        await refreshMeta();
        notifyChanged();
        await syncNow();
      } else {
        // Sesión cerrada: se borran los datos privados del dispositivo. El catálogo (público) se conserva.
```

```
        await repos.loans.clear();
        await repos.outbox.clear();
        await repos.meta.clear();
        if (cancelled) return;
        setLastSyncAt(null);
        setLastError(null);
        notifyChanged();
      }
    })();

    return () => {
      cancelled = true;
    };
  }, [status, uid, repos, refreshMeta, notifyChanged, syncNow]);

  const requestLoan = useCallback(
    async (book: Book) => {
      if (!user) throw new Error('Sesión no iniciada');
      const loan = await loanService.requestLoan(user, book);
      notifyChanged();
      void syncNow(); // intenta enviar ya; si no hay red queda en la cola
      return loan;
    },
    [user, loanService, notifyChanged, syncNow],
  );

  const transitionLoan = useCallback(
    async (loanId: string, action: LoanAction) => {
      await loanService.transition(loanId, action);
      notifyChanged();
      void syncNow(); // el stock del libro cambió: se actualiza el catálogo
    },
    [loanService, notifyChanged, syncNow],
  );

  const dismissLoan = useCallback(
    async (loanId: string) => {
      await loanService.dismiss(loanId);
      notifyChanged();
    },
    [loanService, notifyChanged],
  );

  const value = useMemo<DataContextValue>(
    () => ({
      repos,
      online,
      syncing,
      lastSyncAt,
      lastError,
      pendingCount,
      dataVersion,
      syncNow,
      requestLoan,
      transitionLoan,
      dismissLoan,
      notifyChanged,
    }),
    [
      repos,
      online,
      syncing,
      lastSyncAt,
      lastError,
      pendingCount,
      dataVersion,
      syncNow,
      requestLoan,
      transitionLoan,
      dismissLoan,
      notifyChanged,
    ],
  );

  return <DataContext value={value}>{children}</DataContext>;
}

export function useData(): DataContextValue {
```

```
  const ctx = useContext(DataContext);
  if (!ctx) throw new Error('useData debe usarse dentro de <DataProvider>');
  return ctx;
}
```

Paso 11.7 · La interfaz solo lee de SQLite

Los hooks consultan la base local y vuelven a consultar cuando `dataVersion` cambia (lo incrementa el proveedor tras cada sincronización o escritura).

▸ **CREAR · library_app/src/hooks/use-library-data.ts**

```
/**
 * Hooks de lectura. La UI nunca llama a la API directamente para listar: lee SQLite, y la
 * sincronización (DataProvider) mantiene SQLite al día. Cuando cambia `dataVersion` se vuelve a consultar.
 */
import { useEffect, useState } from 'react';

import { deriveListStatus, type ListStatus } from '@/lib/list-status';
import { useAuth } from '@/providers/auth-provider';
import { useData } from '@/providers/data-provider';
import type { Book, LocalLoan } from '@/types';

/** Ejecuta una consulta local y la repite cuando cambian los datos. */
function useLocalQuery<T>(
  query: () => Promise<T>,
  deps: readonly unknown[],
): { data: T | null; loaded: boolean } {
  const { dataVersion } = useData();
  const [state, setState] = useState<{ data: T | null; loaded: boolean }>({
    data: null,
    loaded: false,
  });

  useEffect(() => {
    let active = true;
    query().then((data) => {
      if (active) setState({ data, loaded: true });
    });
    return () => {
      active = false;
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [...deps, dataVersion]);

  return state;
}

export function useBooks(filters: { q: string; category: string | null }) {
  const { repos, syncing, lastError, lastSyncAt } = useData();
  const { data, loaded } = useLocalQuery(
    () => repos.books.list({ q: filters.q, category: filters.category }),
    [repos, filters.q, filters.category],
  );
  const books: Book[] = data ?? [];
  const status: ListStatus = deriveListStatus({
    loaded,
    count: books.length,
    syncing,
    error: lastError,
    everSynced: lastSyncAt !== null,
  });
  return { books, status };
}

export function useCategories(): string[] {
  const { repos } = useData();
  const { data } = useLocalQuery(() => repos.books.categories(), [repos]);
  return data ?? [];
}

export function useBook(id: string) {
  const { repos } = useData();
  const { data, loaded } = useLocalQuery(() => repos.books.getById(id), [repos, id]);
  return { book: data, loaded };
}
```

```
/** Estudiante: sus préstamos. Bibliotecario: todos. */
export function useLoans() {
  const { repos, syncing, lastError, lastSyncAt } = useData();
  const { user } = useAuth();
  const scope = user?.role === 'librarian' ? null : (user?.uid ?? '');
  const { data, loaded } = useLocalQuery(() => repos.loans.list(scope), [repos, scope]);
  const loans: LocalLoan[] = data ?? [];
  const status: ListStatus = deriveListStatus({
    loaded,
    count: loans.length,
    syncing,
    error: lastError,
    everSynced: lastSyncAt !== null,
  });
  return { loans, status };
}

/** Último préstamo del usuario actual para un libro (para mostrar "ya solicitado"). */
export function useMyLoanForBook(bookId: string) {
  const { repos } = useData();
  const { user } = useAuth();
  const uid = user?.uid ?? '';
  const { data } = useLocalQuery(
    () => repos.loans.latestForBook(uid, bookId),
    [repos, uid, bookId],
  );
  return data;
}
```

Paso 11.8 · Comunicar el estado al usuario

El aviso no bloquea nada; informa y permite reintentar con un toque. La tarjeta de préstamo muestra el estado real: **Pendiente de envío**, **Solicitado**, **Prestado**, **Devuelto** o **No se pudo solicitar** (con el motivo y un botón Descartar).

▸ **CREAR · library_app/src/components/connection-banner.tsx**

```
import { Pressable, StyleSheet, View } from 'react-native';

import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import { formatDateTime } from '@/lib/text';
import { useData } from '@/providers/data-provider';

import { ThemedText } from './themed-text';

/**
 * Informa el estado de conectividad y de la cola sin bloquear la pantalla:
 * sin conexión → se muestran datos guardados; con cambios pendientes → se avisa cuántos.
 */
export function ConnectionBanner() {
  const theme = useTheme();
  const { online, syncing, lastError, pendingCount, lastSyncAt, syncNow } = useData();

  let message: string | null = null;
  let color: string = theme.warning;

  if (!online) {
    message = `Sin conexión · Mostrando datos guardados (última sincronización: ${formatDateTime(lastSyncAt)})`;
  } else if (lastError) {
    message = `${lastError} Toca para reintentar.`;
    color = theme.danger;
  } else if (pendingCount > 0) {
    const many = pendingCount !== 1;
    const label = `${pendingCount} solicitud${many ? 'es' : ''} pendiente${many ? 's' : ''}`;
    message = `${label} de enviar${syncing ? '…' : ''}`;
  }

  if (!message) return null;
  return (
    <Pressable
      onPress={() => void syncNow()}
      accessibilityRole="button"
      accessibilityLabel={message}
      style={[styles.banner, { borderColor: color }]}>
      <View style={[styles.dot, { backgroundColor: color }]} />
```

```
      <ThemedText type="small" style={styles.text}>
        {message}
      </ThemedText>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  banner: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: Spacing.two,
    borderWidth: 1,
    borderRadius: 12,
    paddingVertical: Spacing.two,
    paddingHorizontal: Spacing.three,
  },
  dot: { width: 10, height: 10, borderRadius: 5 },
  text: { flex: 1 },
});
```

▸ **CREAR · library_app/src/components/loan-card.tsx**

```
import { StyleSheet, View } from 'react-native';

import { Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import { formatDate } from '@/lib/text';
import type { LoanAction, LocalLoan, LoanStatus } from '@/types';

import { Button } from './button';
import { ThemedText } from './themed-text';

const STATUS_LABEL: Record<LoanStatus, string> = {
  requested: 'Solicitado',
  approved: 'Prestado',
  rejected: 'Rechazado',
  returned: 'Devuelto',
};

export function actionsFor(status: LoanStatus): LoanAction[] {
  if (status === 'requested') return ['approve', 'reject'];
  if (status === 'approved') return ['return'];
  return [];
}

const ACTION_LABEL: Record<LoanAction, string> = {
  approve: 'Prestar',
  reject: 'Rechazar',
  return: 'Registrar devolución',
};

interface LoanCardProps {
  loan: LocalLoan;
  /** Bibliotecario: muestra quién pidió el libro y los botones de gestión. */
  librarian: boolean;
  online: boolean;
  busy: boolean;
  onAction: (loan: LocalLoan, action: LoanAction) => void;
  onDismiss: (loan: LocalLoan) => void;
}

export function LoanCard({ loan, librarian, online, busy, onAction, onDismiss }: LoanCardProps) {
  const theme = useTheme();
  const conflict = loan.syncState === 'conflict';
  const pending = loan.syncState === 'pending';

  const statusColor = conflict
    ? theme.danger
    : pending
      ? theme.warning
      : loan.status === 'approved'
        ? theme.success
        : loan.status === 'rejected'
          ? theme.danger
          : theme.textSecondary;
  const statusText = conflict
    ? 'No se pudo solicitar'
```

```
    : pending
      ? 'Pendiente de envío'
      : STATUS_LABEL[loan.status];

  const lines = [
    `Solicitado ${formatDate(loan.requestedAt)}`,
    loan.status === 'approved' && loan.dueDate ? `Entrega ${formatDate(loan.dueDate)}` : null,
    loan.status === 'returned' ? `Devuelto ${formatDate(loan.returnedAt)}` : null,
  ].filter(Boolean);

  return (
    <View
      style={[
        styles.card,
        {
          backgroundColor: theme.backgroundElement,
          borderColor: conflict ? theme.danger : theme.border,
        },
      ]}
      accessible
      accessibilityLabel={`${loan.bookTitle}. ${statusText}. ${lines.join('. ')}`}>
      <View style={styles.row}>
        <ThemedText type="smallBold" style={[styles.title, { fontSize: 16 }]} numberOfLines={2}>
          {loan.bookTitle}
        </ThemedText>
        <ThemedText type="smallBold" style={{ color: statusColor }}>
          {statusText}
        </ThemedText>
      </View>

      {librarian ? (
        <ThemedText type="small" themeColor="textSecondary">
          {loan.userName} · {loan.userEmail}
        </ThemedText>
      ) : null}
      <ThemedText type="small" themeColor="textSecondary">
        {lines.join(' · ')}
      </ThemedText>

      {conflict ? (
        <>
          <ThemedText type="small" style={{ color: theme.danger }}>
            {loan.syncError}
          </ThemedText>
          <Button title="Descartar" variant="secondary" onPress={() => onDismiss(loan)} />
        </>
      ) : null}

      {librarian && !pending && !conflict && actionsFor(loan.status).length > 0 ? (
        <View style={styles.actions}>
          {actionsFor(loan.status).map((action) => (
            <View key={action} style={styles.actionItem}>
              <Button
                title={ACTION_LABEL[action]}
                variant={action === 'approve' ? 'primary' : 'secondary'}
                disabled={!online || busy}
                onPress={() => onAction(loan, action)}
              />
            </View>
          ))}
        </View>
      ) : null}
      {librarian && !online && actionsFor(loan.status).length > 0 ? (
        <ThemedText type="small" themeColor="textSecondary">
          Conéctate a internet para gestionar préstamos.
        </ThemedText>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  card: { padding: Spacing.three, borderRadius: 14, borderWidth: 1, gap: Spacing.two },
  row: { flexDirection: 'row', justifyContent: 'space-between', gap: Spacing.two },
  title: { flex: 1 },
  actions: { flexDirection: 'row', gap: Spacing.two, flexWrap: 'wrap' },
  actionItem: { flexGrow: 1 },
});
```

> **Idea. Decisión de diseño documentada:** el estudiante puede solicitar sin conexión, pero el bibliotecario necesita conexión para prestar o devolver. Prestar modifica el inventario compartido y debe decidirse con datos actuales; permitirlo sin conexión generaría conflictos mucho más difíciles de resolver. Los botones se deshabilitan sin internet con una explicación.

Paso 11.9 · Pantallas definitivas: pestañas, préstamos y perfil

Ya no necesitamos la pantalla provisional ni el hook de la semana 9. Elimínalos y crea las pestañas:

▸ ELIMINAR · `library_app/src/app/index.tsx`

▸ ELIMINAR · `library_app/src/hooks/use-resource.ts`

▸ **CREAR · library_app/src/app/(tabs)/_layout.tsx**

```
import { NativeTabs } from 'expo-router/unstable-native-tabs';
import { useColorScheme } from 'react-native';

import { Colors } from '@/constants/theme';
import { useAuth } from '@/providers/auth-provider';

export default function TabsLayout() {
  const scheme = useColorScheme();
  const colors = Colors[scheme === 'dark' ? 'dark' : 'light'];
  const { user } = useAuth();

  return (
    <NativeTabs
      backgroundColor={colors.background}
      indicatorColor={colors.backgroundElement}
      labelStyle={{ selected: { color: colors.text } }}>
      <NativeTabs.Trigger name="index">
        <NativeTabs.Trigger.Label>Libros</NativeTabs.Trigger.Label>
        <NativeTabs.Trigger.Icon sf="books.vertical" md="menu_book" />
      </NativeTabs.Trigger>

      <NativeTabs.Trigger name="loans">
        <NativeTabs.Trigger.Label>
          {user?.role === 'librarian' ? 'Gestión' : 'Mis préstamos'}
        </NativeTabs.Trigger.Label>
        <NativeTabs.Trigger.Icon sf="list.bullet.clipboard" md="assignment" />
      </NativeTabs.Trigger>

      <NativeTabs.Trigger name="profile">
        <NativeTabs.Trigger.Label>Perfil</NativeTabs.Trigger.Label>
        <NativeTabs.Trigger.Icon sf="person.crop.circle" md="person" />
      </NativeTabs.Trigger>
    </NativeTabs>
  );
}
```

El catálogo ahora lee de SQLite, con búsqueda sin tildes, filtro por categoría y deslizar para sincronizar:

▸ **CREAR · library_app/src/app/(tabs)/index.tsx**

```
import { useDeferredValue, useState } from 'react';
import {
  FlatList,
  Pressable,
  RefreshControl,
  ScrollView,
  StyleSheet,
  TextInput,
  View,
} from 'react-native';

import { BookCard } from '@/components/book-views';
import { ConnectionBanner } from '@/components/connection-banner';
```

```
import { Screen } from '@/components/screen';
import { EmptyView, ErrorView, LoadingView } from '@/components/state-views';
import { ThemedText } from '@/components/themed-text';
import { BottomTabInset, Spacing } from '@/constants/theme';
import { useBooks, useCategories } from '@/hooks/use-library-data';
import { useTheme } from '@/hooks/use-theme';
import { useData } from '@/providers/data-provider';

export default function BooksScreen() {
  const theme = useTheme();
  const { syncNow, syncing, lastError } = useData();
  const [search, setSearch] = useState('');
  const [category, setCategory] = useState<string | null>(null);
  const q = useDeferredValue(search);

  const { books, status } = useBooks({ q, category });
  const categories = useCategories();
  const filtering = q.trim().length > 0 || category !== null;

  return (
    <Screen>
      <View style={styles.header}>
        <ThemedText type="subtitle" accessibilityRole="header">
          Catálogo
        </ThemedText>
        <ConnectionBanner />
        <TextInput
          value={search}
          onChangeText={setSearch}
          placeholder="Buscar por título o autor"
          placeholderTextColor={theme.textSecondary}
          accessibilityLabel="Buscar libros por título o autor"
          returnKeyType="search"
          clearButtonMode="while-editing"
          autoCorrect={false}
          style={[
            styles.search,
            {
              color: theme.text,
              borderColor: theme.border,
              backgroundColor: theme.backgroundElement,
            },
          ]}
        />
        {categories.length > 0 ? (
          <ScrollView
            horizontal
            showsHorizontalScrollIndicator={false}
            contentContainerStyle={styles.chips}>
            {[null, ...categories].map((c) => {
              const selected = c === category;
              return (
                <Pressable
                  key={c ?? 'all'}
                  onPress={() => setCategory(c)}
                  accessibilityRole="button"
                  accessibilityState={{ selected }}
                  accessibilityLabel={c ? `Categoría ${c}` : 'Todas las categorías'}
                  style={[
                    styles.chip,
                    {
                      borderColor: selected ? theme.primary : theme.border,
                      backgroundColor: selected ? theme.primary : 'transparent',
                    },
                  ]}>
                  <ThemedText
                    type="smallBold"
                    style={{ color: selected ? theme.onPrimary : theme.text }}>
                    {c ?? 'Todas'}
                  </ThemedText>
                </Pressable>
              );
            })}
          </ScrollView>
        ) : null}
      </View>

      <FlatList
```

```
        data={books}
        keyExtractor={(b) => b.id}
        renderItem={({ item }) => <BookCard book={item} />}
        ItemSeparatorComponent={() => <View style={{ height: Spacing.two }} />}
        contentContainerStyle={[styles.list, { paddingBottom: BottomTabInset + Spacing.four }]}
        keyboardShouldPersistTaps="handled"
        refreshControl={
          <RefreshControl
            refreshing={syncing && books.length > 0}
            onRefresh={() => void syncNow()}
          />
        }
        ListEmptyComponent={
          status === 'loading' ? (
            <LoadingView message="Cargando catálogo…" />
          ) : status === 'error' ? (
            <ErrorView message={lastError ?? 'Error desconocido'} onRetry={() => void syncNow()} />
          ) : (
            <EmptyView
              title={filtering ? 'Sin resultados' : 'El catálogo está vacío'}
              message={
                filtering
                  ? 'Prueba con otro título, autor o categoría.'
                  : 'Aún no hay libros registrados.'
              }
            />
          )
        }
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  header: { gap: Spacing.two, paddingTop: Spacing.two, paddingBottom: Spacing.two },
  search: {
    minHeight: 48,
    borderWidth: 1,
    borderRadius: 12,
    paddingHorizontal: Spacing.three,
    fontSize: 16,
  },
  chips: { gap: Spacing.two, paddingVertical: Spacing.one },
  chip: {
    minHeight: 36,
    paddingHorizontal: Spacing.three,
    borderRadius: 18,
    borderWidth: 1,
    justifyContent: 'center',
  },
  list: { flexGrow: 1 },
});
```

Mis préstamos (estudiante) o Gestión (bibliotecario, con las acciones prestar, rechazar y devolver):

▸ **CREAR · library_app/src/app/(tabs)/loans.tsx**

```
import { useState } from 'react';
import { Alert, FlatList, Pressable, RefreshControl, StyleSheet, View } from 'react-native';

import { ConnectionBanner } from '@/components/connection-banner';
import { LoanCard } from '@/components/loan-card';
import { Screen } from '@/components/screen';
import { EmptyView, ErrorView, LoadingView } from '@/components/state-views';
import { ThemedText } from '@/components/themed-text';
import { BottomTabInset, Spacing } from '@/constants/theme';
import { useLoans } from '@/hooks/use-library-data';
import { useTheme } from '@/hooks/use-theme';
import { useAuth } from '@/providers/auth-provider';
import { useData } from '@/providers/data-provider';
import { describeError } from '@/services/api/client';
import type { LoanAction, LocalLoan } from '@/types';

type Filter = 'open' | 'all';

export default function LoansScreen() {
```

```
const theme = useTheme();
const { user } = useAuth();
const { online, syncing, lastError, syncNow, transitionLoan, dismissLoan } = useData();
const { loans, status } = useLoans();
const librarian = user?.role === 'librarian';

const [filter, setFilter] = useState<Filter>('open');
const [busyId, setBusyId] = useState<string | null>(null);

// Bibliotecario: por defecto solo lo que requiere acción (solicitados y prestados).
const visible =
  librarian && filter === 'open'
    ? loans.filter((l) => l.status === 'requested' || l.status === 'approved')
    : loans;

async function onAction(loan: LocalLoan, action: LoanAction) {
  setBusyId(loan.id);
  try {
    await transitionLoan(loan.id, action);
  } catch (err) {
    Alert.alert('No se pudo completar la acción', describeError(err));
  } finally {
    setBusyId(null);
  }
}

return (
  <Screen>
    <View style={styles.header}>
      <ThemedText type="subtitle" accessibilityRole="header">
        {librarian ? 'Gestión de préstamos' : 'Mis préstamos'}
      </ThemedText>
      <ConnectionBanner />
      {librarian ? (
        <View style={styles.filters}>
          {(['open', 'all'] as const).map((f) => {
            const selected = f === filter;
            return (
              <Pressable
                key={f}
                onPress={() => setFilter(f)}
                accessibilityRole="button"
                accessibilityState={{ selected }}
                style={[
                  styles.chip,
                  {
                    borderColor: selected ? theme.primary : theme.border,
                    backgroundColor: selected ? theme.primary : 'transparent',
                  },
                ]}>
                <ThemedText
                  type="smallBold"
                  style={{ color: selected ? theme.onPrimary : theme.text }}>
                  {f === 'open' ? 'Por gestionar' : 'Todos'}
                </ThemedText>
              </Pressable>
            );
          })}
        </View>
      ) : null}
    </View>

    <FlatList
      data={visible}
      keyExtractor={(l) => l.id}
      renderItem={({ item }) => (
        <LoanCard
          loan={item}
          librarian={librarian}
          online={online}
          busy={busyId === item.id}
          onAction={onAction}
          onDismiss={(l) => void dismissLoan(l.id)}
        />
      )}
      ItemSeparatorComponent={() => <View style={{ height: Spacing.two }} />}
      contentContainerStyle={[styles.list, { paddingBottom: BottomTabInset + Spacing.four }]}
      refreshControl={
```

```
          <RefreshControl
            refreshing={syncing && visible.length > 0}
            onRefresh={() => void syncNow()}
          />
        }
        ListEmptyComponent={
          status === 'loading' ? (
            <LoadingView message="Cargando préstamos…" />
          ) : status === 'error' ? (
            <ErrorView message={lastError ?? 'Error desconocido'} onRetry={() => void syncNow()} />
          ) : (
            <EmptyView
              title={librarian ? 'Nada por gestionar' : 'Aún no has solicitado libros'}
              message={
                librarian
                  ? 'Las nuevas solicitudes aparecerán aquí.'
                  : 'Entra a un libro y pulsa “Solicitar préstamo”.'
              }
            />
          )
        }
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  header: { gap: Spacing.two, paddingTop: Spacing.two, paddingBottom: Spacing.two },
  filters: { flexDirection: 'row', gap: Spacing.two },
  chip: {
    minHeight: 36,
    paddingHorizontal: Spacing.three,
    borderRadius: 18,
    borderWidth: 1,
    justifyContent: 'center',
  },
  list: { flexGrow: 1 },
});
```

El perfil muestra el estado de sincronización, la actividad reciente y cierra sesión (avisando si hay solicitudes sin enviar):

▸ **CREAR · library_app/src/app/(tabs)/profile.tsx**

```
import { useEffect, useState } from 'react';
import { Alert, ScrollView, StyleSheet, View } from 'react-native';

import { Button } from '@/components/button';
import { Screen } from '@/components/screen';
import { ThemedText } from '@/components/themed-text';
import { BottomTabInset, Spacing } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';
import type { ActivityEntry } from '@/db/repositories';
import { formatDateTime } from '@/lib/text';
import { useAuth } from '@/providers/auth-provider';
import { useData } from '@/providers/data-provider';

export default function ProfileScreen() {
  const theme = useTheme();
  const { user, signOut } = useAuth();
  const { repos, online, syncing, lastSyncAt, lastError, pendingCount, dataVersion, syncNow } =
    useData();
  const [activity, setActivity] = useState<ActivityEntry[]>([]);

  useEffect(() => {
    let active = true;
    repos.activity.recent(15).then((rows) => active && setActivity(rows));
    return () => {
      active = false;
    };
  }, [repos, dataVersion]);

  function confirmSignOut() {
    const warning =
      pendingCount > 0
        ? `Tienes ${pendingCount} solicitud(es) sin enviar que se perderán si cierras sesión ahora. `
        : '';
```

```
    Alert.alert('Cerrar sesión', `${warning}¿Quieres continuar?`, [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Cerrar sesión', style: 'destructive', onPress: () => void signOut() },
    ]);
  }

  return (
    <Screen>
      <ScrollView
        contentContainerStyle={[styles.content, { paddingBottom: BottomTabInset + Spacing.four }]}>
        <ThemedText type="subtitle" accessibilityRole="header">
          Perfil
        </ThemedText>

        <View style={[styles.card, { backgroundColor: theme.backgroundElement }]}>
          <ThemedText type="smallBold" style={{ fontSize: 18 }}>
            {user?.name}
          </ThemedText>
          <ThemedText themeColor="textSecondary">{user?.email}</ThemedText>
          <ThemedText type="smallBold" style={{ color: theme.primary }}>
            {user?.role === 'librarian' ? 'Bibliotecario' : 'Estudiante'}
          </ThemedText>
        </View>

        <View style={[styles.card, { backgroundColor: theme.backgroundElement }]}>
          <ThemedText type="smallBold">Sincronización</ThemedText>
          <ThemedText type="small">Conexión: {online ? 'en línea' : 'sin conexión'}</ThemedText>
          <ThemedText type="small">Última sincronización: {formatDateTime(lastSyncAt)}</ThemedText>
          <ThemedText type="small">Cambios pendientes de enviar: {pendingCount}</ThemedText>
          {lastError ? (
            <ThemedText type="small" style={{ color: theme.danger }}>
              {lastError}
            </ThemedText>
          ) : null}
          <Button
            title={syncing ? 'Sincronizando…' : 'Sincronizar ahora'}
            variant="secondary"
            loading={syncing}
            onPress={() => void syncNow()}
          />
        </View>

        <View style={[styles.card, { backgroundColor: theme.backgroundElement }]}>
          <ThemedText type="smallBold">Actividad reciente</ThemedText>
          {activity.length === 0 ? (
            <ThemedText type="small" themeColor="textSecondary">
              Sin actividad registrada.
            </ThemedText>
          ) : (
            activity.map((a) => (
              <ThemedText key={a.id} type="small" themeColor="textSecondary">
                {formatDateTime(a.createdAt)} · {a.kind} · {a.detail}
              </ThemedText>
            ))
          )}
        </View>

        <Button title="Cerrar sesión" variant="danger" onPress={confirmSignOut} />
      </ScrollView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  content: { gap: Spacing.three, paddingTop: Spacing.two },
  card: { padding: Spacing.three, borderRadius: 14, gap: Spacing.one },
});
```

El detalle del libro, ahora con el botón **Solicitar préstamo** y el estado de la solicitud (la gestión de portadas llega en la semana 12):

▸ **CREAR · library_app/src/app/book/[id].tsx**

```
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { availabilityLabel, BookCover } from '@/components/book-views';
import { Button } from '@/components/button';
import { Screen } from '@/components/screen';
import { EmptyView, LoadingView } from '@/components/state-views';
import { ThemedText } from '@/components/themed-text';
import { Spacing } from '@/constants/theme';
import { useBook, useMyLoanForBook } from '@/hooks/use-library-data';
import { useTheme } from '@/hooks/use-theme';
import { useAuth } from '@/providers/auth-provider';
import { useData } from '@/providers/data-provider';
import { describeError } from '@/services/api/client';
import { LoanRuleError } from '@/services/loans/loan-service';

export default function BookDetailScreen() {
  const theme = useTheme();
  const { id } = useLocalSearchParams<{ id: string }>();
  const { user } = useAuth();
  const { online, requestLoan } = useData();
  const { book, loaded } = useBook(id);
  const myLoan = useMyLoanForBook(id);

  const [busy, setBusy] = useState(false);
  const [feedback, setFeedback] = useState<{ text: string; error: boolean } | null>(null);

  if (!loaded) return <LoadingView />;
  if (!book)
    return <EmptyView title="Libro no encontrado" message="Puede que ya no esté en el catálogo." />;

  const librarian = user?.role === 'librarian';
  const hasActive =
    myLoan &&
    myLoan.syncState !== 'conflict' &&
    (myLoan.status === 'requested' || myLoan.status === 'approved');

  async function onRequest() {
    if (!book) return;
    setBusy(true);
    setFeedback(null);
    try {
      await requestLoan(book);
      setFeedback({
        error: false,
        text: online
          ? 'Solicitud registrada. Se está enviando al servidor.'
          : 'Sin conexión: tu solicitud quedó guardada y se enviará automáticamente cuando vuelva internet.',
      });
    } catch (err) {
      setFeedback({
        error: true,
        text: err instanceof LoanRuleError ? err.message : describeError(err),
      });
    } finally {
      setBusy(false);
    }
  }

  return (
    <Screen edges={['left', 'right', 'bottom']}>
      <ScrollView contentContainerStyle={styles.content}>
        <View style={styles.cover}>
          <BookCover book={book} width={160} height={220} />
        </View>

        <ThemedText type="subtitle" accessibilityRole="header">
          {book.title}
        </ThemedText>
        <ThemedText themeColor="textSecondary">{book.author}</ThemedText>
        <ThemedText type="small" themeColor="textSecondary">
          {book.category} · {book.year}
        </ThemedText>
        <ThemedText
          type="smallBold"
```

```
          style={{ color: book.availableCopies > 0 ? theme.success : theme.danger }}>
          {availabilityLabel(book)}
        </ThemedText>
        <ThemedText style={styles.description}>{book.description}</ThemedText>

        {!librarian ? (
          <View style={styles.actions}>
            {myLoan && myLoan.syncState === 'pending' ? (
              <ThemedText type="smallBold" style={{ color: theme.warning }}>
                Solicitud guardada · pendiente de envío
              </ThemedText>
            ) : null}
            {myLoan && myLoan.syncState === 'conflict' ? (
              <ThemedText
                type="smallBold"
                style={{ color: theme.danger }}
                accessibilityRole="alert">
                Tu última solicitud no se pudo procesar: {myLoan.syncError}
              </ThemedText>
            ) : null}
            {myLoan?.syncState === 'synced' && myLoan.status === 'requested' ? (
              <ThemedText type="smallBold" style={{ color: theme.primary }}>
                Solicitud enviada · esperando al bibliotecario
              </ThemedText>
            ) : null}
            {myLoan?.syncState === 'synced' && myLoan.status === 'approved' ? (
              <ThemedText type="smallBold" style={{ color: theme.success }}>
                Tienes este libro prestado
              </ThemedText>
            ) : null}
            <Button
              title={book.availableCopies > 0 ? 'Solicitar préstamo' : 'Sin ejemplares disponibles'}
              disabled={book.availableCopies <= 0 || !!hasActive}
              loading={busy}
              onPress={onRequest}
              accessibilityHint="Envía una solicitud de préstamo al bibliotecario"
            />
          </View>
        ) : null}

        {feedback ? (
          <ThemedText
            accessibilityLiveRegion="polite"
            accessibilityRole={feedback.error ? 'alert' : undefined}
            style={{ color: feedback.error ? theme.danger : theme.success }}>
            {feedback.text}
          </ThemedText>
        ) : null}
      </ScrollView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  content: { gap: Spacing.two, paddingVertical: Spacing.three },
  cover: { alignItems: 'center', paddingBottom: Spacing.two },
  description: { marginTop: Spacing.two, lineHeight: 24 },
  actions: { gap: Spacing.two, marginTop: Spacing.three },
});
```

Por último, el **layout raíz definitivo**: abre y migra la base de datos antes que todo lo demás (`SQLiteProvider` con `onInit`), y envuelve la app con la sesión y los datos. El orden importa: la sesión y los datos dependen de la base de datos.

▸ **CREAR · library_app/src/app/_layout.tsx**

```
import { DarkTheme, DefaultTheme, Stack, ThemeProvider } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { SQLiteProvider } from 'expo-sqlite';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';
import { useColorScheme } from 'react-native';

import { migrateDbIfNeeded } from '@/db/migrations';
import { AuthProvider, useAuth } from '@/providers/auth-provider';
import { DataProvider } from '@/providers/data-provider';
```

```
// La pantalla de carga nativa permanece hasta saber si hay una sesión guardada.
SplashScreen.preventAutoHideAsync();

function RootStack() {
  const { status } = useAuth();

  useEffect(() => {
    if (status !== 'loading') SplashScreen.hideAsync();
  }, [status]);

  if (status === 'loading') return null;
  const signedIn = status === 'signedIn';

  return (
    <Stack screenOptions={{ headerShown: false }}>
      {/* Rutas protegidas (semana 10): el guard decide qué pantallas existen según la sesión.
          Al cerrar sesión se redirige solo; al iniciarla, se entra a (tabs). */}
      <Stack.Protected guard={!signedIn}>
        <Stack.Screen name="sign-in" />
        <Stack.Screen name="sign-up" options={{ headerShown: true, title: 'Crear cuenta' }} />
      </Stack.Protected>
      <Stack.Protected guard={signedIn}>
        <Stack.Screen name="(tabs)" />
        <Stack.Screen
          name="book/[id]"
          options={{ headerShown: true, title: 'Detalle del libro' }}
        />
      </Stack.Protected>
    </Stack>
  );
}

export default function RootLayout() {
  const colorScheme = useColorScheme();
  return (
    <ThemeProvider value={colorScheme === 'dark' ? DarkTheme : DefaultTheme}>
      <StatusBar style="auto" />
      {/* Orden importa: la BD se migra primero; la sesión y los datos dependen de ella. */}
      <SQLiteProvider databaseName="library.db" onInit={migrateDbIfNeeded}>
        <AuthProvider>
          <DataProvider>
            <RootStack />
          </DataProvider>
        </AuthProvider>
      </SQLiteProvider>
    </ThemeProvider>
  );
}
```

Paso 11.10 · Pruebas de la semana

SQLite no corre dentro de Jest, pero Node 24 incluye `node:sqlite`. El archivo `node-sqlite.ts` expone la misma interfaz que `expo-sqlite` sobre ese motor, así que **las consultas reales** se prueban sin dispositivo. `fakes.ts` aporta datos de prueba y repositorios en memoria.

▸ **CREAR · library_app/test-utils/node-sqlite.ts**

```
/**
 * Adaptador de pruebas: expone la parte de la API async de expo-sqlite que usa la app
 * sobre `node:sqlite` (SQLite real incluido en Node). Así se prueban migraciones y consultas
 * sin dispositivo. El SQL es el mismo; solo cambia el "motor" que lo ejecuta.
 */
import { DatabaseSync } from 'node:sqlite';

import type { SQLiteDatabase } from 'expo-sqlite';

type Param = string | number | null;

function args(params: unknown[]): Param[] {
  return (params.length === 1 && Array.isArray(params[0]) ? params[0] : params) as Param[];
}

export function createTestDb(): SQLiteDatabase {
  const db = new DatabaseSync(':memory:');
  let depth = 0;
```

```
  const adapter = {
    async execAsync(sql: string) {
      db.exec(sql);
    },
    async runAsync(sql: string, ...params: unknown[]) {
      const r = db.prepare(sql).run(...args(params));
      return { lastInsertRowId: Number(r.lastInsertRowid), changes: Number(r.changes) };
    },
    async getAllAsync(sql: string, ...params: unknown[]) {
      return db.prepare(sql).all(...args(params));
    },
    async getFirstAsync(sql: string, ...params: unknown[]) {
      return db.prepare(sql).get(...args(params)) ?? null;
    },
    async withTransactionAsync(task: () => Promise<void>) {
      // expo-sqlite no anida transacciones; aquí se ignoran las internas para imitarlo.
      if (depth > 0) return task();
      depth++;
      db.exec('BEGIN');
      try {
        await task();
        db.exec('COMMIT');
      } catch (err) {
        db.exec('ROLLBACK');
        throw err;
      } finally {
        depth--;
      }
    },
  };
  return adapter as unknown as SQLiteDatabase;
}
```

▸ **CREAR · library_app/test-utils/fakes.ts**

```
import type { SyncDeps } from '@/services/sync/sync-service';
import type { OutboxItem } from '@/db/repositories';
import type { Book, Loan } from '@/types';

export function makeLoan(overrides: Partial<Loan> = {}): Loan {
  return {
    id: 'srv-1',
    bookId: 'b1',
    bookTitle: 'Clean Code',
    userId: 'u1',
    userEmail: 'a@b.co',
    userName: 'Ana',
    status: 'requested',
    clientRequestId: 'req-00000001',
    requestedAt: '2026-10-12T10:00:00.000Z',
    resolvedAt: null,
    dueDate: null,
    returnedAt: null,
    updatedAt: '2026-10-12T10:00:00.000Z',
    ...overrides,
  };
}

export function makeBook(overrides: Partial<Book> = {}): Book {
  return {
    id: 'b1',
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Tecnología',
    year: 2008,
    description: 'x',
    totalCopies: 2,
    availableCopies: 2,
    coverUrl: null,
    updatedAt: '2026-10-12T10:00:00.000Z',
    ...overrides,
  };
}

/** Repositorios en memoria con la misma forma que los de SQLite, para probar la sincronización sin base de datos. */
export function makeFakeRepos(initialOutbox: OutboxItem[] = []) {
  const state = {
```

```
    outbox: [...initialOutbox],
    books: [] as Book[],
    serverLoans: [] as Loan[],
    conflicts: [] as { clientRequestId: string; message: string }[],
    meta: new Map<string, string>(),
    log: [] as string[],
  };

  const repos = {
    books: {
      upsertMany: async (b: Book[]) => void state.books.push(...b),
    },
    loans: {
      upsertFromServer: async (l: Loan) => void state.serverLoans.push(l),
      upsertManyFromServer: async (l: Loan[]) => void state.serverLoans.push(...l),
      markConflict: async (clientRequestId: string, message: string) =>
        void state.conflicts.push({ clientRequestId, message }),
    },
    outbox: {
      pending: async () => [...state.outbox],
      remove: async (id: number) => void (state.outbox = state.outbox.filter((i) => i.id !== id)),
      recordFailure: async (id: number, message: string) =>
        void (state.outbox = state.outbox.map((i) =>
          i.id === id ? { ...i, attempts: i.attempts + 1, lastError: message } : i,
        )),
    },
    meta: {
      get: async (k: string) => state.meta.get(k) ?? null,
      set: async (k: string, v: string) => void state.meta.set(k, v),
    },
    activity: { log: async (_k: string, d: string) => void state.log.push(d) },
  };

  return { state, repos: repos as unknown as SyncDeps['repos'] };
}

export function outboxItem(id: number, clientRequestId: string, attempts = 0): OutboxItem {
  return {
    id,
    kind: 'request_loan',
    payload: { bookId: 'b1', clientRequestId },
    attempts,
    lastError: null,
    createdAt: '2026-10-12T10:00:00.000Z',
  };
}
```

- `database.test.ts`: migración desde cero, idempotente, y **actualización v1 → v2 sin perder datos**; búsqueda sin tildes; un `%` o una comilla no rompen el SQL; reemplazo de la copia local; restricción `CHECK` de estados.
- `sync-service.test.ts`: orden FIFO, sin conexión conserva la cola, conflicto `409`, `5xx` con reintentos, `401`, cursor incremental, el cursor no avanza si la descarga falla.
- `offline-flow.test.ts`: los escenarios de reconexión, conflicto e idempotencia de punta a punta con SQL real.

▸ **CREAR · library_app/__tests__/text.test.ts**

```
import { escapeLike, normalizeSearch } from '@/lib/text';

describe('normalizeSearch', () => {
  it('ignora tildes y mayúsculas', () => {
    expect(normalizeSearch('  El amor en los tiempos del CÓLERA ')).toBe(
      'el amor en los tiempos del colera',
    );
    expect(normalizeSearch('García Márquez')).toBe('garcia marquez');
  });
});

describe('escapeLike', () => {
  it('escapa comodines para que se busquen de forma literal', () => {
    expect(escapeLike('100%_real\\')).toBe('100\\%\\_real\\\\');
  });
});
```

▸ **CREAR · library_app/__tests__/database.test.ts**

```
import { DATABASE_VERSION, migrateDbIfNeeded } from '@/db/migrations';
import { createRepositories, type Repositories } from '@/db/repositories';

import { createTestDb } from '../test-utils/node-sqlite';
import { makeBook, makeLoan } from '../test-utils/fakes';

async function freshDb() {
  const db = createTestDb();
  await migrateDbIfNeeded(db);
  return { db, repos: createRepositories(db) };
}

describe('migraciones', () => {
  it('crea el esquema y fija user_version', async () => {
    const { db } = await freshDb();
    const v = await db.getFirstAsync<{ user_version: number }>('PRAGMA user_version');
    expect(v?.user_version).toBe(DATABASE_VERSION);
    const tables = await db.getAllAsync<{ name: string }>(
      `SELECT name FROM sqlite_master WHERE type = 'table'`,
    );
    expect(tables.map((t) => t.name)).toEqual(
      expect.arrayContaining(['books', 'loans', 'outbox', 'sync_meta', 'activity_log']),
    );
  });

  it('es idempotente: ejecutarla otra vez no falla ni pierde datos', async () => {
    const { db, repos } = await freshDb();
    await repos.books.upsertMany([makeBook()]);
    await migrateDbIfNeeded(db);
    expect(await repos.books.list()).toHaveLength(1);
  });

  it('aplica solo la migración que falta (v1 → v2)', async () => {
    const db = createTestDb();
    // Simula un teléfono que ya tenía la app en la versión 1: esquema v1 sin índices ni activity_log.
    await db.execAsync(`
      CREATE TABLE books (id TEXT PRIMARY KEY NOT NULL, title TEXT NOT NULL,
        author TEXT NOT NULL, category TEXT NOT NULL, year INTEGER NOT NULL,
        description TEXT NOT NULL, total_copies INTEGER NOT NULL,
        available_copies INTEGER NOT NULL, cover_url TEXT, updated_at TEXT NOT NULL,
        search_text TEXT NOT NULL);
      CREATE TABLE loans (id TEXT PRIMARY KEY NOT NULL, book_id TEXT NOT NULL,
        book_title TEXT NOT NULL, user_id TEXT NOT NULL, user_email TEXT NOT NULL,
        user_name TEXT NOT NULL, status TEXT NOT NULL,
        client_request_id TEXT NOT NULL UNIQUE, requested_at TEXT NOT NULL,
        resolved_at TEXT, due_date TEXT, returned_at TEXT, updated_at TEXT NOT NULL,
        sync_state TEXT NOT NULL DEFAULT 'synced', sync_error TEXT);
      CREATE TABLE outbox (id INTEGER PRIMARY KEY AUTOINCREMENT, kind TEXT NOT NULL,
        payload TEXT NOT NULL, attempts INTEGER NOT NULL DEFAULT 0, last_error TEXT,
        created_at TEXT NOT NULL);
      CREATE TABLE sync_meta (key TEXT PRIMARY KEY NOT NULL, value TEXT NOT NULL);
      INSERT INTO books VALUES ('b1', 'Rayuela', 'Julio Cortázar', 'Novela', 1963, 'd',
        1, 1, NULL, '2026-01-01T00:00:00.000Z', 'rayuela julio cortazar');
      PRAGMA user_version = 1;
    `);

    await migrateDbIfNeeded(db);

    const v = await db.getFirstAsync<{ user_version: number }>('PRAGMA user_version');
    expect(v?.user_version).toBe(DATABASE_VERSION);
    expect(await db.getAllAsync('SELECT * FROM books')).toHaveLength(1); // los datos existentes sobreviven
    await createRepositories(db).activity.log('x', 'y'); // activity_log ya existe
  });
});

describe('libros', () => {
  let repos: Repositories;
  beforeEach(async () => {
    ({ repos } = await freshDb());
    await repos.books.upsertMany([
      makeBook({
        id: 'cholera',
        title: 'El amor en los tiempos del cólera',
        author: 'Gabriel García Márquez',
        category: 'Novela',
      }),
```

```
      makeBook({ id: 'rayuela', title: 'Rayuela', author: 'Julio Cortázar', category: 'Novela' }),
      makeBook({
        id: 'clean',
        title: 'Clean Code',
        author: 'Robert C. Martin',
        category: 'Tecnología',
      }),
    ]);
  });

  it('lista ordenada por título', async () => {
    expect((await repos.books.list()).map((b) => b.id)).toEqual(['clean', 'cholera', 'rayuela']);
  });

  it('busca sin importar tildes ni mayúsculas, en título y autor', async () => {
    expect((await repos.books.list({ q: 'COLERA' })).map((b) => b.id)).toEqual(['cholera']);
    expect((await repos.books.list({ q: 'cortazar' })).map((b) => b.id)).toEqual(['rayuela']);
    expect((await repos.books.list({ q: 'garcia marquez' })).map((b) => b.id)).toEqual(['cholera']);
  });

  it('los comodines del usuario se tratan como texto (no como LIKE)', async () => {
    expect(await repos.books.list({ q: '%' })).toHaveLength(0);
    expect(await repos.books.list({ q: '_' })).toHaveLength(0);
  });

  it('una consulta con comilla no rompe el SQL (parametrizada)', async () => {
    expect(await repos.books.list({ q: "'; DROP TABLE books; --" })).toHaveLength(0);
    expect(await repos.books.list()).toHaveLength(3);
  });

  it('filtra por categoría y combina con la búsqueda', async () => {
    expect(await repos.books.list({ category: 'Novela' })).toHaveLength(2);
    expect(await repos.books.list({ category: 'Novela', q: 'rayuela' })).toHaveLength(1);
    expect(await repos.books.list({ category: 'Tecnología', q: 'rayuela' })).toHaveLength(0);
  });

  it('upsert actualiza un libro existente sin duplicarlo', async () => {
    await repos.books.upsertMany([
      makeBook({
        id: 'rayuela',
        title: 'Rayuela',
        availableCopies: 0,
        coverUrl: '/api/books/rayuela/cover?v=1',
      }),
    ]);
    const all = await repos.books.list();
    expect(all).toHaveLength(3);
    const rayuela = await repos.books.getById('rayuela');
    expect(rayuela).toMatchObject({ availableCopies: 0, coverUrl: '/api/books/rayuela/cover?v=1' });
  });

  it('categorías sin repetir', async () => {
    expect(await repos.books.categories()).toEqual(['Novela', 'Tecnología']);
  });

  it('getById de un libro inexistente → null', async () => {
    expect(await repos.books.getById('nada')).toBeNull();
  });
});

describe('préstamos y cola', () => {
  let repos: Repositories;
  beforeEach(async () => {
    ({ repos } = await freshDb());
  });

  const local = (clientRequestId = 'req-00000001') =>
    makeLoan({ id: `local-${clientRequestId}`, clientRequestId, status: 'requested' });

  it('un préstamo pendiente cuenta como activo (evita solicitudes duplicadas offline)', async () => {
    await repos.loans.insertPending(local());
    expect(await repos.loans.hasActiveForBook('u1', 'b1')).toBe(true);
    expect(await repos.loans.hasActiveForBook('u2', 'b1')).toBe(false);
    expect(await repos.loans.hasActiveForBook('u1', 'otro')).toBe(false);
  });

  it('upsertFromServer reemplaza la copia local (mismo clientRequestId) sin duplicar', async () => {
    await repos.loans.insertPending(local());
```

```
    await repos.loans.upsertFromServer(makeLoan({ id: 'srv-1', clientRequestId: 'req-00000001' }));

    const loans = await repos.loans.list('u1');
    expect(loans).toHaveLength(1);
    expect(loans[0]).toMatchObject({ id: 'srv-1', syncState: 'synced', syncError: null });
  });

  it('un conflicto deja de contar como activo y conserva el motivo', async () => {
    await repos.loans.insertPending(local());
    await repos.loans.markConflict('req-00000001', 'Sin ejemplares');

    const [loan] = await repos.loans.list('u1');
    expect(loan).toMatchObject({ syncState: 'conflict', syncError: 'Sin ejemplares' });
    expect(await repos.loans.hasActiveForBook('u1', 'b1')).toBe(false);
    await repos.loans.remove(loan.id);
    expect(await repos.loans.list('u1')).toHaveLength(0);
  });

  it('list(null) devuelve todos (bibliotecario); list(uid) solo los del usuario; más recientes primero', async () => {
    await repos.loans.upsertManyFromServer([
      makeLoan({
        id: 'a',
        clientRequestId: 'req-aaaaaaaa',
        userId: 'u1',
        requestedAt: '2026-10-01T00:00:00.000Z',
      }),
      makeLoan({
        id: 'b',
        clientRequestId: 'req-bbbbbbbb',
        userId: 'u2',
        requestedAt: '2026-10-03T00:00:00.000Z',
      }),
      makeLoan({
        id: 'c',
        clientRequestId: 'req-cccccccc',
        userId: 'u1',
        requestedAt: '2026-10-02T00:00:00.000Z',
      }),
    ]);
    expect((await repos.loans.list(null)).map((l) => l.id)).toEqual(['b', 'c', 'a']);
    expect((await repos.loans.list('u1')).map((l) => l.id)).toEqual(['c', 'a']);
  });

  it('la base de datos rechaza estados inválidos (CHECK)', async () => {
    await expect(
      repos.loans.upsertFromServer(makeLoan({ status: 'hackeado' as never })),
    ).rejects.toThrow();
  });

  it('la cola es FIFO, cuenta intentos y permite quitar elementos', async () => {
    await repos.outbox.enqueue('request_loan', { bookId: 'b1', clientRequestId: 'req-aaaaaaaa' });
    await repos.outbox.enqueue('request_loan', { bookId: 'b2', clientRequestId: 'req-bbbbbbbb' });
    expect(await repos.outbox.count()).toBe(2);

    const [first, second] = await repos.outbox.pending();
    expect(first.payload.clientRequestId).toBe('req-aaaaaaaa');
    await repos.outbox.recordFailure(first.id, 'timeout');
    expect((await repos.outbox.pending())[0]).toMatchObject({ attempts: 1, lastError: 'timeout' });

    await repos.outbox.remove(first.id);
    expect((await repos.outbox.pending()).map((i) => i.id)).toEqual([second.id]);
  });

  it('meta guarda y actualiza valores; activity conserva solo los últimos 200 eventos', async () => {
    await repos.meta.set('k', '1');
    await repos.meta.set('k', '2');
    expect(await repos.meta.get('k')).toBe('2');
    expect(await repos.meta.get('nada')).toBeNull();

    for (let i = 0; i < 210; i++) await repos.activity.log('t', `evento ${i}`);
    const recent = await repos.activity.recent(500);
    expect(recent).toHaveLength(200);
    expect(recent[0].detail).toBe('evento 209');
  });
});
```

▸ **CREAR · library_app/__tests__/sync-service.test.ts**

```
import { ApiRequestError, NetworkError } from '@/services/api/client';
import { createSyncService, MAX_ATTEMPTS } from '@/services/sync/sync-service';

import { makeBook, makeFakeRepos, makeLoan, outboxItem } from '../test-utils/fakes';

const NOW = new Date('2026-10-12T12:00:00.000Z');

function setup(
  opts: { outbox?: ReturnType<typeof outboxItem>[]; post?: jest.Mock; get?: jest.Mock } = {},
) {
  const { state, repos } = makeFakeRepos(opts.outbox ?? []);
  const get =
    opts.get ??
    jest.fn(async (path: string) =>
      path.startsWith('/api/books')
        ? { books: [makeBook()], categories: [], serverTime: '2026-10-12T11:59:59.000Z' }
        : { loans: [makeLoan()] },
    );
  const post = opts.post ?? jest.fn();
  const sync = createSyncService({ api: { get, post } as never, repos, now: () => NOW });
  return { state, get, post, sync };
}

describe('sync: enviar la cola (outbox)', () => {
  it('envía la solicitud pendiente, la quita de la cola y guarda la versión del servidor', async () => {
    const post = jest.fn().mockResolvedValue({ loan: makeLoan({ id: 'srv-9' }) });
    const { state, sync } = setup({ outbox: [outboxItem(1, 'req-00000001')], post });

    const result = await sync.run();

    expect(post).toHaveBeenCalledWith('/api/loans', {
      bookId: 'b1',
      clientRequestId: 'req-00000001',
    });
    expect(result.pushed).toBe(1);
    expect(state.outbox).toHaveLength(0);
    expect(state.serverLoans.some((l) => l.id === 'srv-9')).toBe(true);
  });

  it('respeta el orden de creación (FIFO)', async () => {
    const post = jest.fn().mockImplementation(async (_p, body) => ({
      loan: makeLoan({ clientRequestId: body.clientRequestId }),
    }));
    const { sync } = setup({
      outbox: [outboxItem(1, 'req-aaaaaaaa'), outboxItem(2, 'req-bbbbbbbb')],
      post,
    });
    await sync.run();
    expect(post.mock.calls.map((c) => c[1].clientRequestId)).toEqual([
      'req-aaaaaaaa',
      'req-bbbbbbbb',
    ]);
  });

  it('sin conexión conserva TODA la cola y propaga NetworkError', async () => {
    const post = jest.fn().mockRejectedValue(new NetworkError());
    const { state, sync } = setup({
      outbox: [outboxItem(1, 'req-00000001'), outboxItem(2, 'req-00000002')],
      post,
    });

    await expect(sync.run()).rejects.toBeInstanceOf(NetworkError);
    expect(state.outbox).toHaveLength(2);
    expect(state.conflicts).toHaveLength(0);
    expect(post).toHaveBeenCalledTimes(1); // no insiste con las demás
  });

  it('409 NO_COPIES → conflicto: gana el servidor, se informa y se saca de la cola', async () => {
    const post = jest
      .fn()
      .mockRejectedValue(new ApiRequestError(409, 'NO_COPIES', 'No hay ejemplares disponibles'));
    const { state, sync } = setup({ outbox: [outboxItem(1, 'req-00000001')], post });

    const result = await sync.run();

    expect(result.conflicts).toBe(1);
```

```
    expect(state.outbox).toHaveLength(0);
    expect(state.conflicts[0]).toMatchObject({ clientRequestId: 'req-00000001' });
    expect(state.conflicts[0].message).toMatch(/ejemplares/i);
  });

  it('un conflicto no bloquea las solicitudes siguientes', async () => {
    const post = jest
      .fn()
      .mockRejectedValueOnce(new ApiRequestError(409, 'ALREADY_ACTIVE', 'x'))
      .mockResolvedValueOnce({ loan: makeLoan({ id: 'srv-2', clientRequestId: 'req-bbbbbbbb' }) });
    const { state, sync } = setup({
      outbox: [outboxItem(1, 'req-aaaaaaaa'), outboxItem(2, 'req-bbbbbbbb')],
      post,
    });

    const result = await sync.run();
    expect(result).toMatchObject({ pushed: 1, conflicts: 1 });
    expect(state.outbox).toHaveLength(0);
  });

  it('error 5xx: reintenta después (suma intentos) sin descartar la solicitud', async () => {
    const post = jest.fn().mockRejectedValue(new ApiRequestError(503, 'INTERNAL_ERROR', 'caído'));
    const { state, sync } = setup({ outbox: [outboxItem(1, 'req-00000001')], post });

    await expect(sync.run()).rejects.toBeInstanceOf(ApiRequestError);
    expect(state.outbox).toHaveLength(1);
    expect(state.outbox[0].attempts).toBe(1);
    expect(state.conflicts).toHaveLength(0);
  });

  it(`tras ${MAX_ATTEMPTS} fallos 5xx se abandona y se informa al usuario`, async () => {
    const post = jest.fn().mockRejectedValue(new ApiRequestError(500, 'INTERNAL_ERROR', 'x'));
    const { state, sync } = setup({
      outbox: [outboxItem(1, 'req-00000001', MAX_ATTEMPTS - 1)],
      post,
    });

    const result = await sync.run();
    expect(result.conflicts).toBe(1);
    expect(state.outbox).toHaveLength(0);
  });

  it('401 (sesión inválida) no se confunde con un conflicto del préstamo', async () => {
    const post = jest.fn().mockRejectedValue(new ApiRequestError(401, 'UNAUTHENTICATED', 'x'));
    const { state, sync } = setup({ outbox: [outboxItem(1, 'req-00000001')], post });

    await expect(sync.run()).rejects.toMatchObject({ status: 401 });
    expect(state.outbox).toHaveLength(1);
    expect(state.conflicts).toHaveLength(0);
  });
});

describe('sync: traer cambios del servidor', () => {
  it('la primera vez pide todos los libros y guarda el cursor (con 1 s de solape)', async () => {
    const { get, state, sync } = setup();
    await sync.run();

    expect(get).toHaveBeenCalledWith('/api/books');
    expect(state.books).toHaveLength(1);
    expect(state.meta.get('books.cursor')).toBe('2026-10-12T11:59:58.000Z');
    expect(state.meta.get('sync.lastSyncAt')).toBe(NOW.toISOString());
  });

  it('después pide solo lo modificado desde el cursor (sincronización incremental)', async () => {
    const { get, sync, state } = setup();
    state.meta.set('books.cursor', '2026-10-12T09:00:00.000Z');
    await sync.run();
    expect(get).toHaveBeenCalledWith('/api/books?updatedSince=2026-10-12T09%3A00%3A00.000Z');
  });

  it('si la descarga falla NO avanza el cursor ni marca la sincronización como exitosa', async () => {
    const get = jest.fn().mockRejectedValue(new NetworkError());
    const { state, sync } = setup({ get });
    state.meta.set('books.cursor', 'viejo');

    await expect(sync.run()).rejects.toBeInstanceOf(NetworkError);
    expect(state.meta.get('books.cursor')).toBe('viejo');
    expect(state.meta.has('sync.lastSyncAt')).toBe(false);
```

```
  });
});
```

▸ **CREAR · library_app/__tests__/offline-flow.test.ts**

```
/**
 * Escenario completo de la semana 11 con SQL real (node:sqlite) y una API simulada:
 *   sin conexión → solicitar → se guarda local + cola → vuelve la red → se envía → llega la versión del servidor.
 */
import { migrateDbIfNeeded } from '@/db/migrations';
import { createRepositories } from '@/db/repositories';
import { ApiRequestError, NetworkError } from '@/services/api/client';
import { createLoanService, LoanRuleError } from '@/services/loans/loan-service';
import { createSyncService } from '@/services/sync/sync-service';
import type { Loan, User } from '@/types';

import { makeBook, makeLoan } from '../test-utils/fakes';
import { createTestDb } from '../test-utils/node-sqlite';

const student: User = { uid: 'u1', email: 'a@b.co', name: 'Ana', role: 'student' };

async function setup() {
  const db = createTestDb();
  await migrateDbIfNeeded(db);
  const repos = createRepositories(db);
  let n = 0;
  const loanService = createLoanService({
    api: { patch: jest.fn() },
    repos,
    newId: () => `req-${String(++n).padStart(8, '0')}`,
  });
  return { repos, loanService };
}

/** API simulada que se comporta como la real: idempotente por clientRequestId y con control de ejemplares. */
function fakeServer(options: { online: boolean; copies: number }) {
  const created = new Map<string, Loan>();
  const server = {
    options,
    post: jest.fn(async (_path: string, body: { bookId: string; clientRequestId: string }) => {
      if (!options.online) throw new NetworkError();
      const existing = created.get(body.clientRequestId);
      if (existing) return { loan: existing };
      if (options.copies <= 0)
        throw new ApiRequestError(409, 'NO_COPIES', 'No hay ejemplares disponibles');
      const loan = makeLoan({
        id: `srv-${created.size + 1}`,
        clientRequestId: body.clientRequestId,
        bookId: body.bookId,
      });
      created.set(body.clientRequestId, loan);
      return { loan };
    }),
    get: jest.fn(async (path: string) => {
      if (!options.online) throw new NetworkError();
      return path.startsWith('/api/books')
        ? {
            books: [makeBook({ availableCopies: options.copies })],
            serverTime: '2026-10-12T12:00:00.000Z',
          }
        : { loans: [...created.values()] };
    }),
  };
  return server;
}

describe('flujo offline-first', () => {
  it('solicitar sin red → pendiente → al reconectar se envía y se reemplaza por la del servidor', async () => {
    const { repos, loanService } = await setup();
    await repos.books.upsertMany([makeBook()]);
    const server = fakeServer({ online: false, copies: 2 });
    const sync = createSyncService({ api: server as never, repos });

    // 1) Sin conexión: la solicitud se acepta de inmediato.
    const loan = await loanService.requestLoan(student, (await repos.books.getById('b1'))!);
    expect(loan.syncState).toBe('pending');
    expect(await repos.outbox.count()).toBe(1);
```

```
    // 2) Intento de sincronizar sin red: nada se pierde.
    await expect(sync.run()).rejects.toBeInstanceOf(NetworkError);
    expect(await repos.outbox.count()).toBe(1);
    expect((await repos.loans.list('u1'))[0].syncState).toBe('pending');

    // 3) Vuelve la red.
    server.options.online = true;
    const result = await sync.run();

    expect(result.pushed).toBe(1);
    expect(await repos.outbox.count()).toBe(0);
    const loans = await repos.loans.list('u1');
    expect(loans).toHaveLength(1); // sin duplicados
    expect(loans[0]).toMatchObject({ id: 'srv-1', syncState: 'synced' });
  });

  it('reintentar tras perder la respuesta no duplica la solicitud en el servidor (idempotencia)', async () => {
    const { repos, loanService } = await setup();
    await repos.books.upsertMany([makeBook()]);
    const server = fakeServer({ online: true, copies: 2 });
    const sync = createSyncService({ api: server as never, repos });

    await loanService.requestLoan(student, (await repos.books.getById('b1'))!);
    await sync.run();
    // Simula que la respuesta se perdió: la cola vuelve a tener el mismo clientRequestId.
    const [loan] = await repos.loans.list('u1');
    await repos.outbox.enqueue('request_loan', {
      bookId: 'b1',
      clientRequestId: loan.clientRequestId,
    });
    await sync.run();

    expect(await repos.loans.list('u1')).toHaveLength(1);
  });

  it('conflicto: otro estudiante tomó el último ejemplar mientras estabas offline → gana el servidor', async () => {
    const { repos, loanService } = await setup();
    await repos.books.upsertMany([makeBook({ availableCopies: 1 })]); // el catálogo guardado aún muestra 1
    const server = fakeServer({ online: false, copies: 1 });
    const sync = createSyncService({ api: server as never, repos });

    await loanService.requestLoan(student, (await repos.books.getById('b1'))!);

    server.options.online = true;
    server.options.copies = 0; // mientras tanto se agotaron
    const result = await sync.run();

    expect(result.conflicts).toBe(1);
    expect(await repos.outbox.count()).toBe(0);
    const [loan] = await repos.loans.list('u1');
    expect(loan.syncState).toBe('conflict');
    expect(loan.syncError).toMatch(/ejemplares/i);
    // El catálogo ya refleja la realidad del servidor.
    expect((await repos.books.getById('b1'))?.availableCopies).toBe(0);
    // Y el usuario puede volver a intentarlo cuando haya stock (el conflicto no bloquea).
    expect(await repos.loans.hasActiveForBook('u1', 'b1')).toBe(false);
  });

  it('reglas locales: no se puede pedir dos veces el mismo libro ni uno sin ejemplares', async () => {
    const { repos, loanService } = await setup();
    await repos.books.upsertMany([
      makeBook({ id: 'b1' }),
      makeBook({ id: 'b2', availableCopies: 0 }),
    ]);

    await loanService.requestLoan(student, (await repos.books.getById('b1'))!);
    await expect(
      loanService.requestLoan(student, (await repos.books.getById('b1'))!),
    ).rejects.toBeInstanceOf(LoanRuleError);
    await expect(
      loanService.requestLoan(student, (await repos.books.getById('b2'))!),
    ).rejects.toThrow(/disponibles/);
    expect(await repos.outbox.count()).toBe(1);
  });
});
```

Nota. **Punto de control de la semana 11.** `npm run typecheck`, `npx expo lint` y `npm test` en verde (8 archivos de pruebas).

5.3 Escenarios de prueba en el teléfono

Escenario A · Reconexión

1. Inicia sesión como estudiante con internet y espera a que cargue el catálogo.
2. Activa el **modo avión**. Verás el aviso Sin conexión · Mostrando datos guardados.
3. Abre un libro y pulsa **Solicitar préstamo**. Aparece "Solicitud guardada… se enviará cuando vuelva internet".
4. En Mis préstamos la solicitud dice **Pendiente de envío**; en Perfil, Cambios pendientes: 1.
5. Desactiva el modo avión. En unos segundos la solicitud pasa a **Solicitado** y el contador vuelve a 0.

Escenario B · Conflicto

1. En el libro **Rayuela** (1 ejemplar), entra como estudiante con internet y espera la sincronización.
2. Activa el modo avión y solicita Rayuela: queda Pendiente de envío.
3. Desde otro dispositivo o desde el sitio web, entra como **otro estudiante**, solicita Rayuela y haz que el **bibliotecario la apruebe**. Ahora no quedan ejemplares.
4. Desactiva el modo avión en el primer teléfono. El servidor responde `409 NO_COPIES`.
5. Resultado esperado: tu solicitud queda **No se pudo solicitar** con el motivo "Cuando se envió tu solicitud ya no quedaban ejemplares disponibles", el catálogo muestra Rayuela sin ejemplares y puedes Descartar el aviso.

Escenario C · Reintento sin duplicados

Si el teléfono pierde la señal justo después de enviar, la app reenvía la solicitud con el mismo `clientRequestId`; el servidor devuelve la existente y en la base de datos **no** aparecen dos préstamos (lo prueba `offline-flow.test.ts`).

5.4 Evidencia y criterios

Evidencia para el docente. Modo avión funcionando (catálogo visible, solicitud en cola); video corto o capturas de los escenarios A y B; migraciones y repositorios; pruebas de sincronización en verde; documento breve que justifique la estrategia de conflictos ("gana el servidor").

- ☐ La app arranca y muestra el catálogo sin internet después de la primera sincronización
- ☐ Una solicitud hecha sin conexión se envía sola al volver la red
- ☐ Reintentar nunca crea préstamos duplicados
- ☐ Un conflicto se explica al usuario con un mensaje claro
- ☐ Cerrar sesión borra préstamos y cola del teléfono

6. Semana 12 · Servicio del dispositivo integrado con la API

Del microcurrículo: Integración de un servicio del dispositivo con la API: imágenes, ubicación o notificaciones. Permisos, validación, manejo de fallos y trazabilidad. Presencial: demostración de integración y pruebas en dispositivo. Entrega y sustentación del **hito 3**.

Elegimos **imágenes**: el bibliotecario toma una foto (o elige una de la galería) y la usa como portada de un libro. Es una integración completa: permiso → captura → procesamiento → subida a la API → actualización de la interfaz.

6.1 Conceptos clave

- **Mínimo privilegio.** Se declara solo el permiso de cámara; el micrófono se desactiva. La galería usa el selector del sistema, que **no requiere permiso de almacenamiento**.
- **Permiso en contexto.** No se pide al abrir la app: se pide cuando el usuario pulsa Tomar foto, momento en el que entiende por qué.
- **Permiso denegado para siempre.** Si el usuario marcó "No volver a preguntar", el sistema no muestra más el diálogo; la única salida es enviarlo a **Ajustes**.
- **Validar antes y después.** La app reduce y comprime; la API revisa el tamaño y los **bytes reales** del archivo.
- **Trazabilidad.** Cada paso queda en `activity_log` y se muestra en Perfil → Actividad reciente, sin guardar la imagen ni datos personales.

6.2 Paso a paso

Paso 12.1 · Declarar los permisos (plugin de Expo)

Los textos que ve el usuario se configuran en `app.json`. Esta es la lista **completa** de plugins del proyecto (se reemplaza el arreglo completo; los de SQLite y SecureStore los agregó `expo install`). Cambiarlos exige una nueva compilación nativa; en Expo Go ya vienen configurados por Expo.

▸ **AGREGAR A · library_app/app.json**

```
{
  "expo": {
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#208AEF",
          "image": "./assets/images/splash-icon.png",
          "imageWidth": 76
        }
      ],
      "expo-sqlite",
      "expo-secure-store",
      [
        "expo-image-picker",
        {
          "cameraPermission": "Biblioteca usa la cámara para tomar la foto de portada de un libro.",
          "photosPermission": "Biblioteca accede a tus fotos solo cuando eliges una portada.",
          "microphonePermission": false
        }
      ]
    ]
```

```
  }
}
```

Paso 12.2 · El servicio de cámara y subida

La foto original puede pesar varios MB: se reduce a 600 px de ancho y se prueban calidades decrecientes hasta que cabe en el límite de la API (≈ 520 KB). `PermissionDeniedError` distingue si se puede volver a preguntar (`canAskAgain`).

▸ **CREAR · library_app/src/services/device/cover-service.ts**

```
/**
 * Servicio del dispositivo integrado con la API (semana 12): foto de portada.
 *
 * Flujo: permiso → cámara/galería → reducir y comprimir → PUT /api/books/:id/cover.
 *
 * Buenas prácticas aplicadas:
 *  - Permisos mínimos: solo CAMERA, solo cuando el bibliotecario pulsa el botón (no al abrir la app).
 *    La galería usa el selector del sistema, que no requiere permiso de almacenamiento.
 *  - Se reduce la imagen ANTES de subirla (ahorra datos y cumple el límite de la API).
 *  - Cada paso queda en la bitácora local (activity_log) sin guardar la imagen ni datos personales.
 */
import * as ImageManipulator from 'expo-image-manipulator';
import * as ImagePicker from 'expo-image-picker';

import type { Repositories } from '@/db/repositories';
import type { ApiClient } from '@/services/api/client';
import type { Book } from '@/types';

export type CoverSource = 'camera' | 'library';

/** La API acepta hasta 700 000 caracteres en base64 (~520 KB de imagen). */
export const MAX_BASE64_LENGTH = 700_000;
const COVER_WIDTH = 600;
/** Se prueban calidades decrecientes hasta que entre en el límite. */
const QUALITIES = [0.7, 0.5, 0.35, 0.2];

export class PermissionDeniedError extends Error {
  constructor(
    /** false si el sistema ya no volverá a mostrar el diálogo: hay que enviar al usuario a Ajustes. */
    readonly canAskAgain: boolean,
  ) {
    super('Permiso de cámara denegado');
    this.name = 'PermissionDeniedError';
  }
}

export class ImageTooLargeError extends Error {
  constructor() {
    super('La imagen es demasiado grande incluso después de comprimirla');
    this.name = 'ImageTooLargeError';
  }
}

export interface PickedCover {
  base64: string;
  contentType: 'image/jpeg';
}

/** Devuelve null si el usuario cancela. */
export async function pickCover(
  source: CoverSource,
  activity: Pick<Repositories['activity'], 'log'>,
): Promise<PickedCover | null> {
  if (source === 'camera') {
    const permission = await ImagePicker.requestCameraPermissionsAsync();
    await activity.log('cover.permission', `cámara: ${permission.status}`);
    if (!permission.granted) throw new PermissionDeniedError(permission.canAskAgain);
  }

  const options: ImagePicker.ImagePickerOptions = {
    mediaTypes: 'images',
    allowsEditing: true,
    aspect: [3, 4],
    quality: 0.8,
  };
```

```
  const result =
    source === 'camera'
      ? await ImagePicker.launchCameraAsync(options)
      : await ImagePicker.launchImageLibraryAsync(options);

  if (result.canceled || result.assets.length === 0) {
    await activity.log('cover.pick', `${source}: cancelado`);
    return null;
  }

  const asset = result.assets[0];
  for (const compress of QUALITIES) {
    const context = ImageManipulator.ImageManipulator.manipulate(asset.uri);
    context.resize({ width: COVER_WIDTH });
    const rendered = await context.renderAsync();
    const saved = await rendered.saveAsync({
      format: ImageManipulator.SaveFormat.JPEG,
      compress,
      base64: true,
    });
    if (saved.base64 && saved.base64.length <= MAX_BASE64_LENGTH) {
      await activity.log(
        'cover.pick',
        `${source}: ${Math.round(saved.base64.length / 1024)} KB base64, calidad ${compress}`,
      );
      return { base64: saved.base64, contentType: 'image/jpeg' };
    }
  }
  await activity.log('cover.pick', `${source}: imagen demasiado grande`);
  throw new ImageTooLargeError();
}

/** Sube la portada. Requiere conexión: si falla por red, el error llega a la pantalla. */
export async function uploadCover(
  api: Pick<ApiClient, 'put'>,
  activity: Pick<Repositories['activity'], 'log'>,
  bookId: string,
  cover: PickedCover,
): Promise<Book> {
  try {
    const { book } = await api.put<{ book: Book }>(`/api/books/${bookId}/cover`, {
      contentType: cover.contentType,
      data: cover.base64,
    });
    await activity.log('cover.upload', `OK libro ${bookId}`);
    return book;
  } catch (err) {
    await activity.log('cover.upload', `ERROR libro ${bookId}: ${(err as Error).name}`);
    throw err;
  }
}
```

Paso 12.3 · La pantalla del libro, versión definitiva

Reemplaza el detalle de la semana 11. Añade los botones del bibliotecario y traduce cada fallo a una respuesta útil. Cuando se sube la portada se actualiza **también la base local** (`upsertMany`): así aparece en la lista sin esperar otra sincronización, y los demás dispositivos la reciben en la suya porque el servidor sube `updatedAt` del libro.

▸ **CREAR · library_app/src/app/book/[id].tsx**

```
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { Alert, Linking, ScrollView, StyleSheet, View } from 'react-native';

import { availabilityLabel, BookCover } from '@/components/book-views';
import { Button } from '@/components/button';
import { Screen } from '@/components/screen';
import { EmptyView, LoadingView } from '@/components/state-views';
import { ThemedText } from '@/components/themed-text';
import { Spacing } from '@/constants/theme';
import { useBook, useMyLoanForBook } from '@/hooks/use-library-data';
import { useTheme } from '@/hooks/use-theme';
import { useAuth } from '@/providers/auth-provider';
import { useData } from '@/providers/data-provider';
import { api } from '@/services/api';
```

```
import { describeError } from '@/services/api/client';
import {
  ImageTooLargeError,
  pickCover,
  PermissionDeniedError,
  uploadCover,
  type CoverSource,
} from '@/services/device/cover-service';
import { LoanRuleError } from '@/services/loans/loan-service';

export default function BookDetailScreen() {
  const theme = useTheme();
  const { id } = useLocalSearchParams<{ id: string }>();
  const { user } = useAuth();
  const { repos, online, requestLoan, notifyChanged } = useData();
  const { book, loaded } = useBook(id);
  const myLoan = useMyLoanForBook(id);

  const [busy, setBusy] = useState(false);
  const [feedback, setFeedback] = useState<{ text: string; error: boolean } | null>(null);

  if (!loaded) return <LoadingView />;
  if (!book)
    return <EmptyView title="Libro no encontrado" message="Puede que ya no esté en el catálogo." />;

  const librarian = user?.role === 'librarian';
  const hasActive =
    myLoan &&
    myLoan.syncState !== 'conflict' &&
    (myLoan.status === 'requested' || myLoan.status === 'approved');

  async function onRequest() {
    if (!book) return;
    setBusy(true);
    setFeedback(null);
    try {
      await requestLoan(book);
      setFeedback({
        error: false,
        text: online
          ? 'Solicitud registrada. Se está enviando al servidor.'
          : 'Sin conexión: tu solicitud quedó guardada y se enviará automáticamente cuando vuelva internet.',
      });
    } catch (err) {
      setFeedback({
        error: true,
        text: err instanceof LoanRuleError ? err.message : describeError(err),
      });
    } finally {
      setBusy(false);
    }
  }

  async function onChangeCover(source: CoverSource) {
    if (!book) return;
    if (!online) {
      setFeedback({ error: true, text: 'Necesitas conexión para subir la portada.' });
      return;
    }
    setBusy(true);
    setFeedback(null);
    try {
      const cover = await pickCover(source, repos.activity);
      if (!cover) return; // el usuario canceló
      const updated = await uploadCover(api, repos.activity, book.id, cover);
      await repos.books.upsertMany([updated]);
      notifyChanged();
      setFeedback({ error: false, text: 'Portada actualizada.' });
    } catch (err) {
      if (err instanceof PermissionDeniedError) {
        // Si el sistema ya no vuelve a preguntar, la única salida es Ajustes.
        Alert.alert(
          'Permiso de cámara necesario',
          'Para tomar la foto de la portada, permite el acceso a la cámara. ' +
            'También puedes elegir una imagen de la galería.',
          err.canAskAgain
            ? [{ text: 'Entendido' }]
            : [
```

```
              { text: 'Cancelar', style: 'cancel' },
              { text: 'Abrir ajustes', onPress: () => void Linking.openSettings() },
            ],
      );
    } else if (err instanceof ImageTooLargeError) {
      setFeedback({ error: true, text: 'La imagen es demasiado grande. Prueba con otra.' });
    } else {
      setFeedback({ error: true, text: describeError(err) });
    }
  } finally {
    setBusy(false);
  }
}

return (
  <Screen edges={['left', 'right', 'bottom']}>
    <ScrollView contentContainerStyle={styles.content}>
      <View style={styles.cover}>
        <BookCover book={book} width={160} height={220} />
      </View>

      <ThemedText type="subtitle" accessibilityRole="header">
        {book.title}
      </ThemedText>
      <ThemedText themeColor="textSecondary">{book.author}</ThemedText>
      <ThemedText type="small" themeColor="textSecondary">
        {book.category} · {book.year}
      </ThemedText>
      <ThemedText
        type="smallBold"
        style={{ color: book.availableCopies > 0 ? theme.success : theme.danger }}>
        {availabilityLabel(book)}
      </ThemedText>
      <ThemedText style={styles.description}>{book.description}</ThemedText>

      {!librarian ? (
        <View style={styles.actions}>
          {myLoan && myLoan.syncState === 'pending' ? (
            <ThemedText type="smallBold" style={{ color: theme.warning }}>
              Solicitud guardada · pendiente de envío
            </ThemedText>
          ) : null}
          {myLoan && myLoan.syncState === 'conflict' ? (
            <ThemedText
              type="smallBold"
              style={{ color: theme.danger }}
              accessibilityRole="alert">
              Tu última solicitud no se pudo procesar: {myLoan.syncError}
            </ThemedText>
          ) : null}
          {myLoan?.syncState === 'synced' && myLoan.status === 'requested' ? (
            <ThemedText type="smallBold" style={{ color: theme.primary }}>
              Solicitud enviada · esperando al bibliotecario
            </ThemedText>
          ) : null}
          {myLoan?.syncState === 'synced' && myLoan.status === 'approved' ? (
            <ThemedText type="smallBold" style={{ color: theme.success }}>
              Tienes este libro prestado
            </ThemedText>
          ) : null}
          <Button
            title={book.availableCopies > 0 ? 'Solicitar préstamo' : 'Sin ejemplares disponibles'}
            disabled={book.availableCopies <= 0 || !!hasActive}
            loading={busy}
            onPress={onRequest}
            accessibilityHint="Envía una solicitud de préstamo al bibliotecario"
          />
        </View>
      ) : (
        <View style={styles.actions}>
          <ThemedText type="smallBold">Portada del libro</ThemedText>
          <Button title="Tomar foto" disabled={busy} onPress={() => onChangeCover('camera')} />
          <Button
            title="Elegir de la galería"
            variant="secondary"
            disabled={busy}
            onPress={() => onChangeCover('library')}
          />
```

```
          </View>
        )}

        {feedback ? (
          <ThemedText
            accessibilityLiveRegion="polite"
            accessibilityRole={feedback.error ? 'alert' : undefined}
            style={{ color: feedback.error ? theme.danger : theme.success }}>
            {feedback.text}
          </ThemedText>
        ) : null}
      </ScrollView>
    </Screen>
  );
}

const styles = StyleSheet.create({
  content: { gap: Spacing.two, paddingVertical: Spacing.three },
  cover: { alignItems: 'center', paddingBottom: Spacing.two },
  description: { marginTop: Spacing.two, lineHeight: 24 },
  actions: { gap: Spacing.two, marginTop: Spacing.three },
});
```

Nota. **Punto de control de la semana 12.** `npm run typecheck`, `npx expo lint`, `npm test` y la exportación del paquete (`npx expo export`) sin errores.

6.3 Manejo de fallos: qué ve el usuario

Situación	Comportamiento
Permiso de cámara denegado (se puede volver a preguntar)	Alerta explicativa; sigue disponible Elegir de la galería
Denegado definitivamente	Alerta con botón **Abrir ajustes**
Usuario cancela la cámara	No pasa nada, sin mensajes de error
Sin conexión	Mensaje Necesitas conexión para subir la portada (esta acción no se encola)
Imagen demasiado grande tras comprimir	Mensaje pidiendo otra imagen
Servidor rechaza el archivo (`400`/`403`)	Se muestra el mensaje del servidor
Un estudiante intenta la acción	La API responde `403`; la interfaz ni siquiera muestra los botones

6.4 Trazabilidad

Eventos que quedan registrados en SQLite (visibles en Perfil → Actividad reciente):

```
cover.permission · cámara: granted
cover.pick       · camera: 148 KB base64, calidad 0.7
cover.upload     · OK libro clean-code
sync             · Enviadas 1, en conflicto 0
loan.approve     · Préstamo de "Clean Code" → approved
```

Ojo. La bitácora **nunca** guarda la imagen, tokens ni datos personales: solo el tipo de evento y datos técnicos.

6.5 Pruebas en dispositivo (lista de verificación)

El servicio de cámara no se puede probar en un simulador (el de iOS no tiene cámara). Marca cada prueba en un teléfono real:

- ☐ Como bibliotecario, Tomar foto pide permiso la primera vez
- ☐ Rechazar el permiso muestra la alerta y la galería sigue funcionando
- ☐ Conceder el permiso abre la cámara; cancelar no produce errores
- ☐ La portada tomada aparece en el detalle y en la lista de libros
- ☐ Otro dispositivo ve la portada tras sincronizar
- ☐ Con modo avión, intentar subir muestra el mensaje de conexión
- ☐ Como estudiante no existen los botones de portada
- ☐ Perfil → Actividad reciente muestra los eventos `cover.*`

6.6 Evidencia del hito 3

> **Evidencia para el docente. Hito 3 (semana 12, 15 %):** aplicación integrada con API REST, autenticación y almacenamiento seguro de sesión, sincronización local-remota, manejo de conectividad y un servicio del dispositivo; pruebas de escenarios críticos y documentación; sustentación individual.

Prepárate para explicar y **modificar en vivo**:

- Cómo viaja una solicitud de préstamo desde el toque del botón hasta Firestore, y qué pasa si falla cada tramo.
- Por qué los tokens están en SecureStore y qué ocurre con un `401`.
- Qué es `clientRequestId` y qué problema resuelve.
- Qué pasa con una solicitud hecha sin conexión cuando el ejemplar se agota.
- Cómo agregarías una columna a `books` con una migración nueva.

7. Pruebas de integración y calidad

7.1 La app real contra la API real

Esta prueba ejecuta **el código de la app** (cliente HTTP, sesión, SQLite, cola y sincronización) contra tu API y Firebase. Verifica el flujo estudiante → bibliotecario → estudiante y la recuperación automática de un token vencido. Solo corre si defines la URL de la API **y** las credenciales de un estudiante y un bibliotecario tuyos, por variables de entorno (así no queda ninguna credencial en el repositorio). `node-fetch.ts` es un `fetch` real sobre `node:http` (el entorno de Jest sustituye el global por un simulacro).

▸ **CREAR · library_app/test-utils/node-fetch.ts**

```
/** `fetch` real sobre node:http: el entorno de jest-expo sustituye el fetch global por un mock. */
import http from 'node:http';
import https from 'node:https';

export const nodeFetch = ((input: string, init: RequestInit = {}) =>
  new Promise<Response>((resolve, reject) => {
    const url = new URL(input);
    const transport = url.protocol === 'https:' ? https : http;
    const req = transport.request(
      url,
      { method: init.method ?? 'GET', headers: init.headers as Record<string, string> },
      (res) => {
        const chunks: Buffer[] = [];
        res.on('data', (c) => chunks.push(c));
        res.on('end', () => {
          const headers = new Headers();
          for (const [k, v] of Object.entries(res.headers))
            if (typeof v === 'string') headers.set(k, v);
          resolve(new Response(Buffer.concat(chunks), { status: res.statusCode, headers }));
        });
      },
    );
    req.on('error', reject);
    init.signal?.addEventListener('abort', () => req.destroy(new Error('aborted')));
    if (init.body) req.write(init.body as string);
    req.end();
  })) as unknown as typeof fetch;
```

▸ **CREAR · library_app/__tests__/e2e-api.test.ts**

```
/**
 * Integración REAL: el código de la app (cliente HTTP, sesión, SQLite, cola y sincronización)
 * contra library_api + Firebase. Solo se ejecuta si defines la URL de la API y las credenciales de
 * un estudiante y un bibliotecario TUYOS (por variables de entorno: nada queda escrito en el repo):
 *
 *   (en library_api)  pnpm seed && pnpm dev
 *   (en library_app)  E2E_API_URL=http://localhost:3000 \
 *                     E2E_STUDENT_EMAIL=... E2E_STUDENT_PASSWORD=... \
 *                     E2E_LIBRARIAN_EMAIL=... E2E_LIBRARIAN_PASSWORD=... npx jest e2e-api
 *   (en library_api)  pnpm reset-demo      ← deja los préstamos como al inicio
 */
import { randomUUID } from 'node:crypto';

import { migrateDbIfNeeded } from '@/db/migrations';
import { createRepositories } from '@/db/repositories';
import { createApiClient } from '@/services/api/client';
import { createAuthService } from '@/services/auth/auth-service';
import { SessionManager, type SecureStorage } from '@/services/auth/session-manager';
import { createLoanService } from '@/services/loans/loan-service';
import { createSyncService } from '@/services/sync/sync-service';

import { nodeFetch } from '../test-utils/node-fetch';
import { createTestDb } from '../test-utils/node-sqlite';

const BASE = process.env.E2E_API_URL;
const STUDENT = {
  email: process.env.E2E_STUDENT_EMAIL ?? '',
  password: process.env.E2E_STUDENT_PASSWORD ?? '',
```

```
};
const LIBRARIAN = {
  email: process.env.E2E_LIBRARIAN_EMAIL ?? '',
  password: process.env.E2E_LIBRARIAN_PASSWORD ?? '',
};
const configured =
  BASE && STUDENT.email && STUDENT.password && LIBRARIAN.email && LIBRARIAN.password;
const maybe = configured ? describe : describe.skip;

class MemoryStorage implements SecureStorage {
  data = new Map<string, string>();
  getItemAsync = async (k: string) => this.data.get(k) ?? null;
  setItemAsync = async (k: string, v: string) => void this.data.set(k, v);
  deleteItemAsync = async (k: string) => void this.data.delete(k);
}

/** Construye una "instalación" completa de la app: sesión + API + BD local + servicios. */
async function installApp(credentials: { email: string; password: string }) {
  const publicApi = createApiClient({ baseUrl: BASE!, timeoutMs: 15_000, fetchImpl: nodeFetch });
  const session = new SessionManager(new MemoryStorage(), publicApi);
  const api = createApiClient({
    baseUrl: BASE!,
    timeoutMs: 15_000,
    fetchImpl: nodeFetch,
    auth: {
      getAccessToken: () => session.getAccessToken(),
      refreshSession: () => session.refresh(),
    },
  });
  const db = createTestDb();
  await migrateDbIfNeeded(db);
  const repos = createRepositories(db);
  const user = await createAuthService(publicApi, session).signIn(
    credentials.email,
    credentials.password,
  );
  return {
    user,
    session,
    repos,
    api,
    sync: createSyncService({ api, repos }),
    loans: createLoanService({ api, repos, newId: randomUUID }),
  };
}

maybe('app ⇄ API real', () => {
  jest.setTimeout(60_000);

  it('el estudiante inicia sesión, sincroniza el catálogo y solicita un libro "sin conexión"', async () => {
    const app = await installApp(STUDENT);
    expect(app.user.role).toBe('student');

    // Primera sincronización: catálogo completo en SQLite.
    const first = await app.sync.run();
    expect(first.booksPulled).toBe(10);
    expect(await app.repos.books.list()).toHaveLength(10);
    expect((await app.repos.books.list({ q: 'colera' })).map((b) => b.title)).toEqual([
      'El amor en los tiempos del cólera',
    ]);

    // Segunda: incremental, no vuelve a bajar nada.
    expect((await app.sync.run()).booksPulled).toBe(0);

    // "Sin conexión": solo se guarda local + cola (no se llama a sync).
    const book = (await app.repos.books.getById('pragmatic-programmer'))!;
    const pending = await app.loans.requestLoan(app.user, book);
    expect(pending.syncState).toBe('pending');
    expect(await app.repos.outbox.count()).toBe(1);

    // "Vuelve la red": se envía y el préstamo local es reemplazado por el del servidor.
    const result = await app.sync.run();
    expect(result.pushed).toBe(1);
    expect(await app.repos.outbox.count()).toBe(0);
    const mine = await app.repos.loans.list(app.user.uid);
    expect(mine).toHaveLength(1);
    expect(mine[0].id.startsWith('local-')).toBe(false);
    expect(mine[0]).toMatchObject({
```

```
      bookId: 'pragmatic-programmer',
      status: 'requested',
      syncState: 'synced',
    });

    // El bibliotecario aprueba desde su propia instalación; el estudiante lo ve al sincronizar.
    const librarian = await installApp(LIBRARIAN);
    expect(librarian.user.role).toBe('librarian');
    await librarian.sync.run();
    const toManage = (await librarian.repos.loans.list(null)).find((l) => l.id === mine[0].id);
    expect(toManage?.status).toBe('requested');

    const approved = await librarian.loans.transition(mine[0].id, 'approve');
    expect(approved.status).toBe('approved');

    await app.sync.run();
    expect((await app.repos.loans.list(app.user.uid))[0]).toMatchObject({ status: 'approved' });
    expect((await app.repos.books.getById('pragmatic-programmer'))?.availableCopies).toBe(
      book.availableCopies - 1,
    );

    await librarian.loans.transition(mine[0].id, 'return');
    await app.sync.run();
    expect((await app.repos.loans.list(app.user.uid))[0].status).toBe('returned');
    expect((await app.repos.books.getById('pragmatic-programmer'))?.availableCopies).toBe(
      book.availableCopies,
    );
  });

  it('un estudiante no puede ejecutar acciones de bibliotecario (403 desde el cliente de la app)', async () => {
    const app = await installApp(STUDENT);
    await expect(
      app.api.patch('/api/loans/lo-que-sea', { action: 'approve' }),
    ).rejects.toMatchObject({ status: 403 });
  });

  it('con un token vencido la app se recupera sola (401 → refresh → reintento)', async () => {
    const app = await installApp(STUDENT);
    // Corrompe el token guardado en memoria para forzar un 401 real del servidor.
    (app.session as unknown as { idToken: string }).idToken = 'token.invalido.xyz';
    (app.session as unknown as { expiresAt: number }).expiresAt = Date.now() + 3_600_000; // "parece" vigente
    await expect(app.api.get('/api/auth/me')).resolves.toMatchObject({
      user: { email: STUDENT.email.toLowerCase() },
    });
  });
});

# (en library_api)  pnpm seed && pnpm dev
E2E_API_URL=http://localhost:3000 \
E2E_STUDENT_EMAIL=estudiante@tucorreo.com E2E_STUDENT_PASSWORD=TuClaveEstudiante \
E2E_LIBRARIAN_EMAIL=bibliotecario@tucorreo.com E2E_LIBRARIAN_PASSWORD=TuClaveBibliotecario \
npx jest e2e-api
# (en library_api)  pnpm reset-demo      ← deja los datos como al inicio
```

7.2 Verificación completa

▸ **TERMINAL · library_app**

```
npm run typecheck
npx expo lint
npm test
```

> **Nota. Punto de control final.** Los comandos sin errores (las pruebas de integración se omiten si no defines las variables `E2E_*`). Tu app está completa.

7.3 Qué cubren las pruebas

Archivo	Qué protege
`api-client.test.ts`	Errores tipados, timeout, reintento tras 401, respuestas sin JSON
`session-manager.test.ts`	Almacenamiento seguro, renovación única, sesión sin red
`sync-service.test.ts`	Cola, conflictos, reintentos, cursor incremental
`database.test.ts`	Migraciones, consultas, búsqueda, restricciones (SQL real)
`offline-flow.test.ts`	Reconexión, conflicto e idempotencia de punta a punta
`components.test.tsx`	Estados de pantalla, botones, campos y accesibilidad
`list-status.test.ts`, `text.test.ts`	Reglas puras de estados y búsqueda
`e2e-api.test.ts`	La app real contra la API real (solo con las variables `E2E_*`)

Nota. Las pruebas automáticas **no sustituyen** la prueba en un teléfono real: no cubren permisos del sistema, cámara, rendimiento ni la experiencia táctil. Hazlas siempre antes de entregar.

8. Problemas frecuentes

Síntoma	Causa probable	Solución
`Network request failed` o No hay conexión con el servidor	`EXPO_PUBLIC_API_URL` usa `localhost` en un teléfono, IP incorrecta, otra red Wi-Fi o firewall	Usa la IP del PC, misma red, `pnpm dev:lan`, abre el puerto 3000; reinicia con `npx expo start -c`
Funciona en el navegador del PC pero no en el teléfono	La API escucha solo en `localhost`	Arranca con `pnpm dev:lan`
En WSL2 no conecta desde el teléfono	WSL2 está en red NAT	`networkingMode=mirrored` o `netsh interface portproxy` (ver 1.4)
El cambio de `.env` no se nota	Metro guarda caché	`npx expo start -c`
`Correo o contraseña incorrectos` con tus usuarios de prueba	No creaste el usuario con `pnpm create-user` o Authentication no está habilitado	Revisa la Guía 1 (secciones 2.5 y 4.4)
Un bibliotecario recién asignado sigue viendo opciones de estudiante	Su token aún no se renovó	Cerrar sesión y volver a entrar
La app pide sesión cada vez	`expo-secure-store` falló o hubo `401` al renovar	Revisa los logs de la API; vuelve a iniciar sesión
Las portadas no se ven	La URL de la API es inalcanzable desde el teléfono	Es la misma URL de `.env`; prueba abrirla en el navegador del teléfono
`typecheck` no encuentra `expo-env.d.ts` o las rutas tipadas	Aún no se generaron los tipos de Expo	Ejecuta `npx expo start` unos segundos
`expo-sqlite` no funciona en web	SQLite nativo no está soportado en esta app para web	Prueba en Android o iOS
`Error al subir portada: imagen demasiado grande`	Fotografía muy pesada	La app ya comprime; prueba con otra imagen

9. Bitácora de uso de IA generativa

El microcurrículo exige registrar cada uso de IA con **propósito, evidencia, validación y decisión**. Úsala para explorar, explicar, depurar, generar casos de prueba o documentar, pero tú debes poder explicar y modificar cada línea que entregues.

Fecha	Herramienta y prompt (resumido)	Qué respondió	Cómo lo validé	Decisión
(ej.) Sem. 10	"¿Dónde guardo el refreshToken en Expo?"	Sugirió `AsyncStorage`	Contrasté con la guía de MASVS y la documentación de Expo: no es almacenamiento seguro	Rechazada; usé `expo-secure-store`
(ej.) Sem. 11	"Genera casos de prueba para una cola offline"	8 casos	Ejecuté los casos; 2 fallaron por un error de mi servicio, no del caso	Aceptada con ajustes; corregí el servicio

Buenas prácticas: no pegues credenciales, el `.env` ni datos personales en un asistente; verifica siempre contra la **documentación oficial** y ejecutando en un dispositivo; guarda el enlace o la captura como evidencia.

10. Referencias

- React Native. Accessibility. https://reactnative.dev/docs/accessibility
- Expo. Expo Router: protected routes. https://docs.expo.dev/router/advanced/protected/
- Expo. Expo SQLite. https://docs.expo.dev/versions/latest/sdk/sqlite/
- Expo. Expo SecureStore. https://docs.expo.dev/versions/latest/sdk/securestore/
- Expo. NetInfo. https://docs.expo.dev/versions/latest/sdk/netinfo/
- Expo. ImagePicker e ImageManipulator. https://docs.expo.dev/versions/latest/sdk/imagepicker/
- Expo. Unit testing with Jest. https://docs.expo.dev/develop/unit-testing/
- OWASP. Mobile Application Security Verification Standard (MASVS). https://mas.owasp.org/MASVS/
- UNESCO (2023). Guidance for generative AI in education and research. https://unesdoc.unesco.org/ark:/48223/pf0000386693